

CURSO

PROGRAMACIÓN ORIENTADA A OBJETOS

Prólogo

La necesidad de reducir la brecha existente entre las etapas de aprendizaje en una institución educativa del nivel técnico, técnico superior universitario y superior y la de aplicación productiva en el mercado laboral, llevó a la ANIEI y al ILCE a diseñar el Modelo Paracurricular, en el que se procura la actualización inmediata y permanente de los profesionistas de las áreas de informática y computación. Motivo por el cual la tarea de “Formación de Entrenadores en Tecnologías de Información”, se sustenta en el Modelo Paracurricular.

El modelo paracurricular es producto de diversos análisis y consultas, de las cuales surgieron las áreas de conocimiento a ser consideradas en la formación o entrenamiento del capital humano en nuestro país: Programación, Calidad, Sistemas Distribuidos y Desarrollo Empresarial, dichas áreas están incluidas en los cuatro perfiles del capital humano para la Industria de Software, definidos en el modelo paracurricular: Emprendedor y Administrador de Proyectos de Software, Arquitecto de Software, Ingeniero de Software y Desarrollador de Software.

Para la generación de contenidos de los cursos se capacitó a los docentes para generar y revisar los contenidos de los cursos del modelo paracurricular y tutores de los mismos, en la modalidad e- learning.

En la primera fase de este subproyecto se inició con el perfil básico de la estructura de la industria del software, el *Desarrollador de Software*, y se atendió el área de programación con el curso Programación Orientada a Objetos.

El presente material, ha sido diseñado para aprender Programación Orientada a Objetos; se abordarán tres temas fundamentales que conforman el paradigma orientado a objetos, los cuales son: abstracción, herencia y polimorfismo. La comprensión de estos temas proporcionará las bases para la generación y desarrollo de software con diferentes perspectivas, que permitirá la solución o atención de servicios demandados por diferentes sectores de la sociedad.

Las etapas que se cubrirán en este curso corresponden al desarrollo de software e implementación de proyectos, apoyándose en técnicas de modelado que utilizan el Lenguaje Unificado de Modelado (Unified Model Language – UML).

Agradecimiento y Reconocimiento

Después de una ardua tarea de investigación se ha logrado la creación de una obra vasta en conocimiento en el desarrollo de las Tecnologías de la Información y Comunicación.

La presente obra no hubiera sido posible sin la valiosa aportación de destacados autores y especialistas en la materia. Es por ello que a manera de reconocimiento queremos agradecer su participación:

PROGRAMACIÓN ORIENTADA A OBJETOS

M. en C. Eiso Jorge Kashiwamoto Yabuta
Instituto Latinoamericano de la Comunicación Educativa,
Universidad La Salle

Lic. Martín Ibarra Romero
CECYT No.9 “Juan de Dios Bátiz Paredes” – IPN

Mtro. Alejandro Canales Cruz
Centro de Investigación en Computación – IPN

Introducción al curso

Este material fue estructurado con la finalidad de enseñar programación orientada a objetos y sus conceptos, basándose en los tres temas del paradigma orientada a objetos: abstracción, herencia y polimorfismo.

Estos tres temas conformarán una base sólida para estructuración, generación y desarrollo de software; dando soluciones y/o servicios que cumplan las perspectivas de diversos segmentos de la sociedad actual.

A lo largo de este curso quedarán cubiertas, a través de técnicas de modelado que usan Lenguaje Unificado de Modelado – UML (Unified Model Language), las etapas de desarrollo de software e implementación de proyectos.

En el tema 1 se trata el contexto de desarrollo de software así como los lenguajes de programación orientada a objetos.

Para el tema 2 estudiaremos los conceptos fundamentales de la POO.

El tema 3 estudiará los elementos adicionales del lenguaje.

En el tema 4 revisaremos los fundamentos de UML y sus vistas.

Para el tema 5 examinaremos la vista lógica que comprende el modelado del diseño y el proceso.

En el tema 6 se examinará la vista de implantación o distribución.

Para que realmente obtengas todos los beneficios de este curso, es muy importante tu compromiso y participación en las distintas actividades que lo conforman.

Intención educativa:

La importancia de este curso radica principalmente en que permite conocer, aplicar y desarrollar el paradigma de la POO en el desarrollo de aplicaciones profesionales para la resolución de problemas en el campo laboral, utilizando las últimas herramientas de software, lo cual facilitará la inserción de los profesionales en el mercado laboral de desarrollo de software.

Objetivos Generales

Al terminar el curso, el alumno será capaz de:

- Conocer algunas técnicas de ingeniería de software usadas en el diseño y la implementación de programas en lenguaje de programación C#
- Aplicar las estructuras de datos más convenientes para solucionar problemas específicos.
- Aplicar técnicas de programación usando estructuras y recursividad.
- Implantar estructuras de datos usando memoria estática y memoria dinámica.
- Utilizar las clases contenedoras de un lenguaje orientado a objetos.

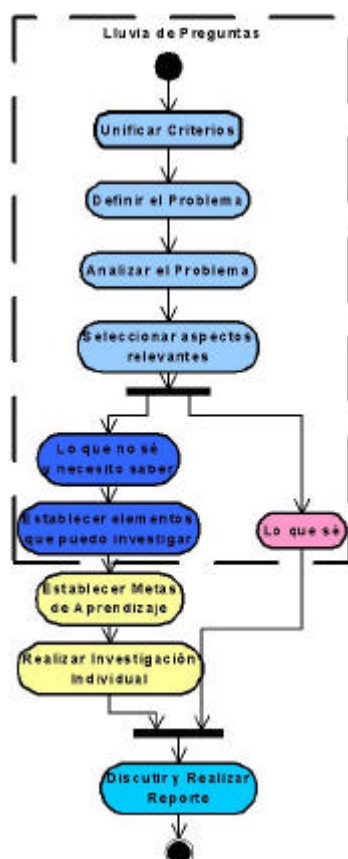
Metodología

Se utilizarán distintos métodos de aprendizaje para cubrir los contenidos. A continuación se describirán dichos métodos. Adicionalmente, en las actividades de aprendizaje se encontrarán las indicaciones específicas en relación con la técnica utilizada en cada una de ellas.

Aprendizaje Basado en Problemas (ABP)

La técnica de Aprendizaje Basado en Problemas (ABP, ó del inglés “PBL-Problem Based Learning”) es una técnica didáctica constructivista, la cual consta de los siguientes pasos:

1. El docente formará grupos de alumnos que trabajarán en equipo para resolver el problema del escenario propuesto. Se nombrará un secretario por equipo, el cual organizará el funcionamiento del mismo y se encargará de enviar los resultados de las actividades realizadas al profesor.
2. El docente presentará el material del escenario, asignará roles y unificará criterios.
3. Los estudiantes elaborarán una serie de preguntas respecto al escenario; posteriormente, procurarán resumir el problema del escenario planteándolo (de ser posible) en forma de una sola pregunta. El docente verificará que tal pregunta sea la apropiada.
4. Una vez establecida la pregunta principal y las preguntas particulares, los estudiantes analizarán y seleccionarán en equipo las preguntas más relevantes a investigar.
5. La primera tarea de cada estudiante será la entrega de un reporte individual correspondiente a una investigación hecha por él mismo.
6. Posteriormente los integrantes discutirán en equipo los resultados de su investigación para establecer en común las respuestas a las preguntas planteadas.
7. En la segunda y última tarea, cada integrante entregará un reporte individual, sin embargo, éste deberá reflejar los aspectos puestos en común en la discusión en grupo. Adicionalmente, en algunos casos de programación, se solicitará la entrega de programas.



Método de Casos

El método de casos difiere de los sistemas de enseñanza tradicionales porque exige que el alumno tome parte activa en el análisis de los problemas y en la toma de decisiones para la solución a situaciones reales muy específicas.

Un aspecto muy importante del método de casos, es el aprender de la experiencia de otros casos similares para la toma de decisiones en el momento de solucionar un problema. La solución de un mayor número de casos incrementará la habilidad para identificar problemas y formular soluciones a problemas reales.

En el método de casos, el proceso que se siga para tomar decisiones y las decisiones mismas, sustentadas en un análisis adecuado, son la clave. Este tipo de ejercicios nos permite aprender a comunicar criterios, defender hechos y opiniones en debates.

Los pasos a seguir en el método de casos se muestran en el siguiente diagrama:



Comprobación de Lectura

La técnica de comprobación de lectura tiene como finalidad fomentar en el alumno la habilidad de leer, analizar y comprender. La comprensión se comprobará al final de cada lección, ya que se presenta una evaluación por medio de preguntas muy puntuales acerca de la lectura.

Los materiales que se utilizarán en este método son una recopilación de diferentes autores de un tema, para homogenizar los conceptos e ideas referentes al tema.

La técnica de comprobación de lectura es una de las más empleadas en los procesos de enseñanza-aprendizaje y tiene como finalidad conformar conceptos e ideas propias al alumno, por lo que no pretende que se memoricen los temas tratados.

Fuentes de Información

APRENDIENDO PROGRAMACION ORIENTADA A OBJETOS EN 21 LECCIONES AVANZADAS.

Sintes, Anthony
Pearson

C++ PROGRAMACION ORIENTADA A OBJETOS

Ruiz Diego
Mega Punto

C++ PROGRAMACION ORIENTADA A OBJETOS

Smith, Jo Ann
Paraninfo

DESARROLLO DE PROYECTOS CON PROGRAMACION ORIENTADA A OBJETOS CON C++

Smith, Jo Ann
Thomson

JAVA CON PROGRAMACION ORIENTADA A OBJETOS Y APLICACIONES EN LA WWW

Wang, Paul
Thomson

PROGRAMACION ORIENTADA A OBJETOS CON C++

Ceballos Sierra, Francisco Javier
Alfaomega

UNIFIED MODELING LANGUAGE. USER GUIDE.

Jacobson, Rumbaugh, Booch
Addison Wesley

Traducción al Español: Lenguaje de Modelado Unificado, Pearson.

Forma de Evaluación

El alumno deberá cumplir con el 100% de asistencia y deberá completar todos los ejercicios y tareas descritos en cada una de las unidades.

La forma de evaluación de este curso es por medio de las rúbricas descritas para cada una de las unidades, siendo 3 el puntaje más alto que se pueda alcanzar, los criterios que se toman para evaluar cada uno de los ejercicios vienen descritos en las tablas de las rúbricas.

Contenido

PRÓLOGO	II
INTRODUCCIÓN AL CURSO	III
INTENCIÓN EDUCATIVA:	IV
OBJETIVOS GENERALES	IV
METODOLOGÍA	IV
FUENTES DE INFORMACIÓN.....	VIII
FORMA DE EVALUACIÓN	VIII
CONTENIDO.....	IX
1.1 CONCEPTO DEL DESARROLLO DE SOFTWARE	1
<i>Ejercicio.....</i>	<i>1</i>
<i>Guía Tutorial del Profesor</i>	<i>4</i>
1.2 LENGUAJES DE PROGRAMACIÓN ORIENTADO A OBJETOS.....	6
<i>Ejercicio.....</i>	<i>6</i>
<i>Páginas Electrónicas de Referencia</i>	<i>7</i>
<i>Ejemplos de la Programación Orientada a Objetos aplicada en la Tecnología Moderna.....</i>	<i>8</i>
2. CONCEPTOS DE LA ORIENTACIÓN A OBJETOS	11
2.1. ORIENTACIÓN A OBJETOS.....	12
<i>Ejercicio</i>	<i>12</i>
<i>Guía Tutorial del Profesor</i>	<i>15</i>
2.2. CLASE, OBJETO E IDENTIDAD.....	17
<i>Ejercicio.....</i>	<i>17</i>
<i>Página Electrónica de Referencia.....</i>	<i>17</i>
<i>Guía Tutorial del Profesor</i>	<i>20</i>
2.3. ENCAPSULAMIENTO.....	23
<i>Ejercicio.....</i>	<i>23</i>
<i>Páginas Electrónicas de Referencia.....</i>	<i>23</i>
<i>Guía Tutorial del Profesor</i>	<i>27</i>
2.4. HERENCIA.....	32
<i>Ejercicio.....</i>	<i>32</i>
<i>Guía Tutorial del Profesor</i>	<i>35</i>
2.5. POLIMORFISMO.....	38
<i>Ejercicio.....</i>	<i>38</i>
2.6. RELACIÓN.....	43
<i>Ejercicio.....</i>	<i>43</i>
2.7. PERSISTENCIA	48
<i>Ejercicio.....</i>	<i>48</i>
<i>Guía Tutorial del Profesor</i>	<i>49</i>
3. ELEMENTOS ADICIONALES DEL LENGUAJE.....	53
3.1 ESTRUCTURA TIPO ARREGLOS.....	53
<i>Ejercicio.....</i>	<i>53</i>
3.2 ESTRUCTURA DE DATOS ESPECIALES	56
<i>Ejercicio.....</i>	<i>56</i>
<i>Ejercicio.....</i>	<i>59</i>
<i>Ejercicio.....</i>	<i>63</i>

Programación Orientada a Objetos

Ejercicio.....	64
3.6 EVENTOS	67
<i>Ejercicio</i>	67
Material 3.6.	68
Creación y Utilización de Delegados	68
Programación de Eventos.....	76
Definir un Evento.....	76
Suscribir a un evento.....	76
Notificar a Suscriptores de un Evento.....	77
<i>Ejercicio</i>	78
3.8 MULTIHILADO.....	78
<i>Ejercicio</i>	78
4. FUNDAMENTOS DE UML.....	80
EJERCICIO	80
Material 1	81
Preguntas I	95
Material II.....	96
DOCUMENTACIÓN.....	99
Preguntas II.....	111
Material III	113
Preguntas III.....	123
5. MODELO DE DISEÑO	128
5.1. DIAGRAMA DE CLASES.....	128
<i>Ejercicio</i>	128
5.2. DIAGRAMA DE SECUENCIA.....	130
<i>Ejercicio</i>	130
5.3. DIAGRAMA DE ESTADO.....	133
<i>Ejercicio</i>	133
5.4. DIAGRAMA DE COLABORACIÓN.....	135
<i>Ejercicio</i>	135
5.5. DIAGRAMA DE ACTIVIDAD.....	138
<i>Ejercicio</i>	138
6. MODELO DE IMPLANTACIÓN O DISTRIBUCIÓN.....	139
6.1. DIAGRAMA DE COMPONENTES (VISTA DE IMPLEMENTACIÓN).....	139
6.2. DIAGRAMA DE DISTRIBUCIÓN (VISTA DE DISTRIBUCIÓN).....	152

1.1 Concepto del Desarrollo de Software

El alumno entenderá el concepto de Desarrollo de Software mediante el siguiente ejercicio aplicando la técnica de aprendizaje basado en problemas.

Objetivos Específicos

- Reconocer el contexto del desarrollo de software que recurre al paradigma de la orientación a objetos.
- Reconocer y evaluar lenguajes de programación orientada a objetos.
- Reconocer las necesidades actuales de la industria del software.
- Comprender la importancia de la orientación a objetos para el desarrollo de sistemas complejos.
- Identificar los beneficios de las tecnologías emergentes.
- Reconocer los paradigmas más importantes en la historia de la orientación a objetos.
- Evaluar las características de los lenguajes orientados a objetos.
- Preparar ambientes de compilación y ejecución de lenguajes orientados a objetos.

Ejercicio

1. Observen detalladamente el siguiente video:



2. Con base en el video, y trabajando en equipo, elaboren una serie de preguntas sobre cómo podrían construir algo así. No olviden que las preguntas deben colocarse en el foro de discusión 1.1.

Al final de la serie de preguntas, elijan las 10 que consideren más relevantes y el secretario del equipo deberá colocarlas en el foro bajo el título "metas de aprendizaje". Una vez que tengan el visto bueno del

profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas.

3. A continuación, se enuncian una serie de ligas que pueden brindarles información acerca de este tema. Cabe aclarar que se puede consultar cualquier otra fuente de información que les pudiera ayudar.
4. Recuerden que, como primera tarea, deben elaborar un reporte individual que incluya las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1).

Rúbrica para evaluar el reporte de investigación individual (máximo 9 puntos).

Contenido del reporte	0	1	2	3	Total
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada.	
Referencias bibliográficas y ligas de Internet	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

5. Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución a

Programación Orientada a Objetos

la pregunta "cómo podrían construir algo así", así como su reporte, en donde se incluirá dicha propuesta y una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2).

Rúbrica para evaluar el reporte final de ABP (en equipo) por parte del tutor. Máximo 9 puntos.

Contenido del reporte	0	1	2	3	Total
Solución o explicación al problema	El equipo no proporciona una solución/explicación al problema.	La solución/explicación del problema es vaga y sin fundamentos.	Dan una solución/explicación detallada al problema, pero no la fundamentan de manera completa.	Dan una solución/explicación detallada al problema y con fundamentos sólidos.	
Síntesis de la información con la cual fundamentan la solución/explicación	No la incluyen.	Únicamente presentan listas de datos y/o copia de lo que cada quien investigó, sin darle coherencia.	Presentan un resumen lógico de la información recopilada por todos.	Sintetizan de manera lógica y coherente la información investigada por todos.	
Referencias bibliográficas y ligas de Internet	No las incluyen.	Las incluyen, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluyen de manera completa, pero no con el formato solicitado.	Las incluyen de manera completa y correcta.	
TOTAL					

Guía Tutorial del Profesor

1. Materia: Programación Orientada a Objetos

2. Tema: Contexto del desarrollo del software.

3. Objetivo de aprendizaje:

Que el alumno identifique la importancia que la POO tiene dentro de las necesidades de la industria, la complejidad de los sistemas y la convergencia de la tecnología.

4. Escenario:

Ver video de Microsoft.

5. Posibles términos que deban clarificarse:

MSN

6. Definición del problema esperado:

- ¿Cómo podría construir esto?
- ¿En que se basan este tipo de aplicaciones?

7. Preguntas esperadas en el análisis del problema:

- ¿Qué otras aplicaciones se podrán hacer?
- ¿Quién construirá esto en México?
- ¿Será factible hacerlo en México?
- ¿Será posible hacerlo hoy?
- ¿Podrá mi plancha, refrigerador, etc., funcionar algún día en la red?
- ¿Será caro este tipo de aplicaciones?
- ¿Perderé mi individualidad?
- ¿Qué rumbo seguirán las tecnologías?

- ¿Cómo se cobra el dinero por internet?
- ¿Cuánta seguridad habrá en los cobros del dinero?
- ¿Los hackers podrán tirar con facilidad este tipo de aplicaciones?

8. Metas de aprendizaje:

- ¿Con que se hace estas aplicaciones?
- ¿Qué características tendrán ese tipo de aplicaciones?
- ¿Será difícil construir ese tipo de aplicaciones?
- ¿Lo hará una persona o un grupo de personas?
- ¿Qué tipos de dispositivos se podrán utilizar con esta tecnología?
- ¿Cuáles son los fundamentos de los lenguajes de programación que se requiere para generar estas aplicaciones?

9. Enviar documentos con ejemplos de clases en cada tipo de aplicación:

- Ejemplo de un web services
- Diagrama de objetos de Excel
- Ejemplo de un formulario tipo windows
- Ejemplo de una clase de una aplicación web

1.2 *Lenguajes de Programación Orientado a Objetos*

El alumno aprenderá la forma de instalación de los lenguajes de programación que utilizarán en este curso y comprenderá el ambiente de trabajo de cada uno de ellos mediante el siguiente ejercicio.

Ejercicio

1. Antes de empezar con las actividades de esta sección, es importante que elijas la tecnología sobre la que quieres trabajar (Java - NetBeans o C# - Visual Studio).

Es posible cursar los dos temas, pero habrá que realizar también ambas tareas. Se recomienda elegir sólo una tecnología para trabajar durante el curso.

2. Observa detalladamente el siguiente video sobre la instalación de NetBeans IDE o Visual Studio.



3. Con base en el video, contesta de manera individual las siguientes preguntas (Preguntas Java - Preguntas C#). No olvides colocar tus respuestas en la sección de Tarea 1.2.

Tarea 1.2

Contesta las siguientes preguntas y entrégalas en un plazo no mayor de 24 horas, a partir de que observaste el video.

1. ¿Cuáles son los comandos que utiliza el J2SE y para qué sirven?
2. ¿Describe la función del compilador del J2SE?
3. ¿Cuál es la función del JVM (Java Virtual Machine)?
4. ¿Describe cómo se desarrolla y ejecuta un programa en JAVA con el J2SE?
5. ¿De dónde se obtiene la documentación del J2SE, qué contiene y dónde se instala?

6. Crear un programa en JAVA, ejecutarlo y utilizar el comando JAVADOC. Entregar en papel el código fuente y archivo generado por el JAVADOC.

7. Describa los 4 opciones (pasos) para la instalación de Visual Studio.Net.

La Tarea debe cumplir con la siguiente rúbrica.

Rúbrica para evaluar el cuestionario de investigación individual Máximo 9 puntos.

Contenido del reporte	0	1	2	3	Total
Respuestas a las preguntas	No responde de manera completa a todas las preguntas.	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada.	
Referencias bibliográficas y ligas de Internet	No las incluye.	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado.	Las incluye de manera completa y correcta.	
TOTAL					

4. A continuación, se enuncian una serie de ligas que te pueden brindar información acerca de este tema. Cabe aclarar que se puede consultar cualquier otra fuente de información que te pueda ayudar.

Páginas Electrónicas de Referencia

Ligas sobre la tecnología Java:

- <http://www.programacion.com/java/>
- <http://acm.escet.urjc.es/~rocapal/acmlux-doc/index.html>
- http://www.netbeans.org/index_es.html

Ejemplos de la Programación Orientada a Objetos aplicada en la Tecnología Moderna

Como podemos apreciar la POO esta contenida en la mayoría de las tecnologías, en el caso de los Web Services en la POO nos ayuda para exponer en el Web los métodos que consideramos necesarios.

Las clases van en color **ROJO**

Las propiedades en **AZUL**

Los métodos en **VERDE**

WEB SERVICES

```
<%@ WebService language="C#" class="Calc" %>
using System;
using System.Web.Services;
using System.Xml.Serialization;
```

```
public class Calc {
    [WebMethod]
    public int Multiplicacion(int a, int b) {
        return a * b;
    }

    [WebMethod]
    public int Resta(int a, int b) {
        return a - b;
    }

} //end class
```

EN APLICACIONES DE OFICINA

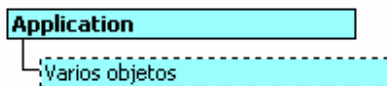
Las Aplicaciones de oficina están repletas de objetos que generalmente son vistos cuando se hace su programación (generalmente de macros)

OBJETO APPLICATION

Representa la aplicación completa Microsoft Excel. El objeto Application contiene: Valores y opciones de toda la aplicación (por ejemplo, muchas de las opciones del cuadro de diálogo Opciones, en el menú Herramientas).

Métodos que devuelven objetos de nivel superior, como ActiveCell, ActiveSheet, etc.

Uso del objeto Application



El ejemplo siguiente crea un objeto libro de Microsoft Excel en otra aplicación y, a continuación, abre un libro en Microsoft Excel y coloca el valor de π (3.14159) en la celda A1.

```
Set xl = CreateObject("Excel.Application")
```

```
xl.Workbooks.add()
```

```
Rem muestra la pantalla del excel
```

```
x1.visible=true
```

```
x1.Range("A1").Value = 3.14159
```

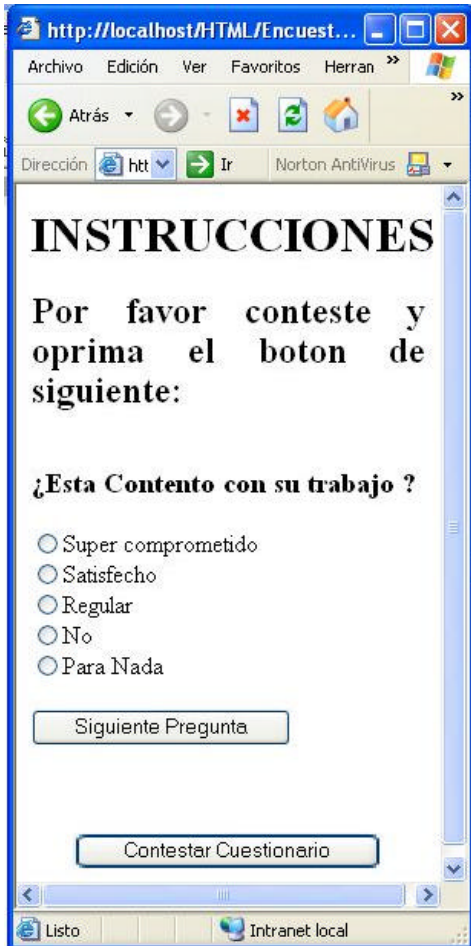
```
set x1 = nothing
```

EJEMPLO DE UNA APLICACIÓN WEB QUE SE PUEDE EJECUTAR EN CUALQUIER NAVEGADOR (PC, PALM, WEBTV, ETC.)

En este ejemplo podemos apreciar el código (clases, métodos y atributos) de la pantalla que se muestra a continuación.

```
public class WebForm1 : System.Web.UI.Page
{
    protected System.Web.UI.HtmlControls.HtmlInputText txtUsuario;
    protected System.Web.UI.WebControls.RequiredFieldValidator
RequiredFieldValidator1;
    protected System.Web.UI.WebControls.RequiredFieldValidator
RequiredFieldValidator2;
    protected System.Web.UI.WebControls.ValidationSummary
ValidationSummary1;
    protected System.Web.UI.HtmlControls.HtmlInputText txtPassword;
```

`protected System.Web.UI.HtmlControls.HtmlInputButton Button1;`



```
private void Page_Load(object sender,
System.EventArgs e)
{
    // Introducir aquí el código de usuario
    para inicializar la página
}
#region Web Form Designer generated code
override protected void OnInit(EventArgs e)
{
    //
    // CODEGEN: llamada requerida por el
    Diseñador de Web Forms ASP.NET.
    //
    InitializeComponent();
    base.OnInit(e);
}
// <summary>
/// Método necesario para admitir el Diseñador,
no se puede modificar
/// el contenido del método con el editor de
código.
/// </summary>
private void InitializeComponent()
{
    this.Button1.ServerClick += new
System.EventHandler(this.Button1_ServerClic
k);
    this.Load += new System.EventHandler(this.Page_Load);
}
```

```
}
#endregion
}
}
```

2. Conceptos de la Orientación a Objetos

El alumno entenderá el concepto de clase y objeto y aplicará las formas

Objetivos Específicos

- Definirá los conceptos de orientación a objetos, clase, objeto, encapsulamiento, herencia, identidad y persistencia.
- Programará y evaluará algunos conceptos de la orientación a objetos.
- Reconocerá los elementos generales implicados en la orientada a objetos.
- Definirá el concepto de Clases considerando los conceptos de abstracción y modularidad.
- Programará objetos considerando sus atributos y métodos.
- Programará el encapsulamiento en objetos considerando los permisos de acceso.
- Programará aplicando el concepto de herencia considerando el polimorfismo.
- Programará aplicando el concepto de identidad y referencias.
- Programará el encapsulamiento en objetos considerando los permisos de acceso.

2.1. Orientación a Objetos

Ejercicio

1. Lee detalladamente el siguiente escenario 2.1.
2. Con base en el escenario, realicen una serie de preguntas en equipo sobre qué fundamentos de Programación Orientada a Objetos se necesitan para desarrollar la aplicación citada. No olviden que las preguntas deben hacerse en el foro de discusión.

Al final de la serie de preguntas, elijan las 10 que consideren más relevantes y el secretario del equipo deberá colocarlas en el foro bajo el título "metas de aprendizaje". Una vez que tengan el visto bueno del profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas.

3. A continuación, se enuncian una serie de ligas que pueden brindarles información acerca de este tema. Cabe aclarar que se puede consultar cualquier otra fuente de información que les pudiera ayudar.

Ligas sobre la Orientación a Objetos

<http://paidoteca.dgsca.unam.mx/neopaidoteca/cursos/becas-java/apuntes.html>

Ligas sobre la tecnología MicroSoft

<http://support.microsoft.com/default.aspx?scid=kb;es;307368>

Ligas sobre la tecnología Java

<http://www.itapizaco.edu.mx/paginas/JavaTut/froufe/parteA/capa-1.html>
<http://paidoteca.dgsca.unam.mx/neopaidoteca/cursos/becas-java/Java2a.html>
http://www.programacion.com/java/tutorial/java_basico/

4. Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1).

Rúbrica para evaluar el reporte de investigación individual Máximo 9 puntos.

Contenido del reporte	0	1	2	3	Total
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada.	
Referencias bibliográficas y ligas de Internet	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte en la sección de Tarea 2.1.

- Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución a la pregunta "qué fundamentos de POO se necesitan para desarrollar la aplicación citada", así como su reporte en donde se incluirá dicha propuesta y una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2).
Rúbrica para evaluar el reporte final de ABP (en equipo) por parte del tutor. Máximo 9 puntos.

Programación Orientada a Objetos

Contenido del reporte	0	1	2	3	Tota
Solución o explicación al problema	El equipo no proporciona una solución/explicación al problema.	La solución/explicación del problema es vaga y sin fundamentos.	Dan una solución/explicación detallada al problema, pero no la fundamentan de manera completa.	Dan una solución/explicación detallada al problema y con fundamentos sólidos.	
Síntesis de la información con la cual fundamentan la solución/explicación	No la incluyen.	Únicamente presentan listas de datos y/o copia de lo que cada quien investigó, sin darle coherencia.	Presentan un resumen lógico de la información recopilada por todos.	Sintetizan de manera lógica y coherente la información investigada por todos.	
Referencias bibliográficas y ligas de Internet	No las incluyen.	Las incluyen, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluyen de manera completa, pero no con el formato solicitado.	Las incluyen de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte final en la sección de Tarea 2.1.

Escenario

Se requiere desarrollar una aplicación que registre las ventas y realice la facturación de una empresa. Se trata de un sistema de tipo personal que debe incluir las siguientes características: calidad, reusabilidad, estabilidad, escalabilidad, etc. Tú eres un desarrollador de software y has visto la potencialidad de la POO para el desarrollo de sistemas, ***¿qué fundamentos de POO crees que deberían tener aquellas personas que pretenden desarrollar esta aplicación?***

Guía Tutorial del Profesor

1. Materia: Programación Orientada a Objetos

2. Tema: Contexto Desarrollo del Software.

3. Objetivos de aprendizaje:

Que el alumno identifique la importancia que la OOP tiene dentro de las necesidades de la industria, la complejidad de los sistemas y la convergencia de la tecnología.

4. Escenario:

Ver video de Microsoft.



5. Posibles términos que deban clarificarse:

MSN

6. Definición del problema esperada:

- ¿Cómo podría construir esto?
- ¿En que se basan este tipo de aplicaciones?

7. Preguntas esperadas en el análisis del problema:

- ¿Qué otras aplicaciones se podrán hacer?
- ¿Quién construirá esto en México?
- ¿Será factible hacerlo en México?
- ¿Será posible hacerlo hoy?
- ¿Podrá mi plancha, refrigerador, etc. funcionar algún día en la red?
- ¿Será caro este tipo de aplicaciones?
- ¿Perderé mi individualidad?
- ¿Qué rumbo seguirán las tecnologías?
- ¿Cómo se cobra el dinero por internet?
- ¿Cuánta seguridad habrá en los cobros del dinero?
- ¿Los hacker podrán tirar con facilidad este tipo de aplicaciones?

8. Metas de aprendizaje:

- ¿Con que se hace estas aplicaciones?
- ¿Qué características tendrán ese tipo de aplicaciones?
- ¿Será difícil construir ese tipo de aplicaciones?
- ¿Lo hará una persona o un grupo de personas?
- ¿Qué tipos de dispositivos se podrán utilizar con esta tecnología?
- ¿Cuáles son los fundamentos de los lenguajes de programación que se requiere para generar estas aplicaciones?

9. Enviar documentos con ejemplos de clases con los siguientes tipos de aplicación:

- Ejemplo de un web services
- Diagrama de objetos de Excel
- Ejemplo de un formulario tipo windows
- Ejemplo de una clase de una aplicación web

2.2. Clase, Objeto e Identidad

El alumno entenderá y aplicará los conceptos de clase, objeto e identidad por medio de la técnica de aprendizaje basado en problemas.

Ejercicio

1. Lee detalladamente el siguiente escenario 2.2.
2. Con base en el escenario 2.2, realicen una serie de preguntas en equipo sobre qué fundamentos de POO se necesitan para desarrollar la aplicación citada. No olviden que las preguntas deben hacerse en el foro de discusión.

Al final de la serie de preguntas, elijan las 10 que consideren más relevantes y el secretario del equipo deberá colocarlas en el foro bajo el título "metas de aprendizaje". Una vez que tengan el visto bueno del profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas.

3. Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1).

Página Electrónica de Referencia

A continuación, se enuncia una liga que puede brindarle información acerca de este tema. Cabe aclarar que se puede consultar cualquier otra fuente de información que les pudiera ayudar.

http://www.programacion.com/articulo/dondo_poo/

Rúbrica 1 para evaluar el reporte de investigación individual Máximo 15 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluye	La incluye, pero no coincide con la del resto del equipo	La incluye, pero difiere en algunos puntos de las del resto del equipo	La incluye y coincide con el resto del equipo.	
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada. Entregar un código	
Referencias bibliográficas	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte en la sección de Tarea individual 2.2.

Programación Orientada a Objetos

4. Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución a la pregunta "qué fundamentos de POO se necesitan para desarrollar la aplicación citada", así como su reporte en donde se incluirá dicha propuesta y una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2).

Rúbrica 2 para evaluar el reporte final de ABP (en equipo) por parte del tutor. Máximo 15 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluyen.	La incluyen, pero no corresponde a las de los reportes individuales.	La incluyen, pero difiere ligeramente de las de los reportes individuales.	La incluyen y coincide al 100% con las de los reportes individuales.	
Solución o explicación al problema	El equipo no proporciona una solución/explicación al problema.	La solución/explicación del problema es vaga y sin fundamentos.	Dan una solución/explicación detallada al problema, pero no la fundamentan de manera completa.	Dan una solución/explicación detallada al problema y con fundamentos sólidos.	
Síntesis de la información con la cual fundamentan la solución/explicación	No la incluyen.	Únicamente presentan listas de datos y/o copia de lo que cada quien investigó, sin darle coherencia.	Presentan un resumen lógico de la información recopilada por todos.	Sintetizan de manera lógica y coherente la información investigada por todos.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Referencias bibliográficas	No las incluyen.	Las incluyen, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluyen de manera completa, pero no con el formato solicitado.	Las incluyen de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte final en la sección de Tarea en equipo 2.2.

Escenario

A un programador su jefe le entrega los diagramas de una aplicación de una calculadora que tiene básicamente 4 operaciones (suma, resta, multiplicación y división) y los operandos de estas operaciones; Por otro lado, la calculadora puede con la que se esté trabajando puede variar en el número de dígitos y de teclas. El jefe le dice al programador que una primera fase debe de codificar estos diagramas utilizando clases. La segunda fase consiste en que debe de instanciar 1 objeto utilizando esta clase y que resuelva la suma (10+20), y la multiplicación (30*10). La tercera fase consiste en utilizar una sola referencia para manejar los objetos e imprimir los resultados de cada uno de los objetos. Todo esto debe estar listo para mañana en la mañana sin excusa ni pretexto.

Guía Tutorial del Profesor

1.- Materia: Programación Orientada a Objetos

2.-Tema: clase, objeto e identidad

3.- Objetivos de aprendizaje:

Al finalizar el tema, el alumno será capaz de:

- Escribir la sintaxis e implementación de:
 - o una clase
 - o los permisos de acceso en una clase
 - o los atributos en una clase
 - o el método en una clase
 - o la instanciación de un objeto
 - o la referencia de un objeto
- Definir instanciación, objeto, identidad de un objeto y referencias de un objeto
- Accesar algunos atributos y métodos de un objeto

4.- Escenario:

A un programador su jefe le entrega los diagramas de una aplicación de una calculadora que tiene básicamente 4 operaciones (suma, resta, multiplicación y división) y los operandos de estas operaciones. El jefe le dice al programador que una primera fase debe de codificar estos diagramas utilizando clases. La segunda

fase consiste en que debe de instanciar 1 objeto utilizando esta clase y que resuelvan la suma (10+20), y la multiplicación (30*10). La tercera fase consiste en utilizar una sola referencia para manejar los objetos e imprimir los resultados de cada uno de los objetos. Todo esto debe estar listo para mañana en la mañana sin excusa ni pretexto.

5.- Posibles términos que deban clarificarse:

Operandos

6.- Definición del problema esperado:

- ¿Cómo codificar la aplicación utilizando clases?
- ¿Cómo instanciar objetos de clases?
- ¿Cómo referenciar a dos objetos utilizando una sola referencia?

7.- Preguntas esperadas en el análisis del problema:

- ¿Qué es un permiso de acceso?
- ¿Qué es la sintaxis de una clase?
- ¿Qué es la sintaxis de un atributo?
- ¿Qué es la sintaxis de un método?
- ¿Qué es la sintaxis de un permiso de acceso?
- ¿Qué es la Implementación de una clase?
- ¿Qué es la Implementación de un atributo?
- ¿Qué es la Implementación de un método?
- ¿Qué es la Implementación de un permiso de acceso?
- ¿Cómo instanciar un objeto?
- ¿Cómo acceder a un atributo de un objeto?

- ¿Cómo acceder a un método de un objeto?
- ¿Cuál es la diferencia entre una clase y un objeto?
- ¿No se necesitan dos referencias para manipular cada uno de los objetos?

8.- Metas de aprendizaje:

- Definir qué es un permiso de acceso
- Declarar la sintaxis de una clase
- Declarar la sintaxis de un atributo
- Declarar la sintaxis de un método
- Declarar la sintaxis de un permiso de acceso
- Implementar una clase
- Implementar un atributo
- Implementar un método
- Implementar un permiso de acceso
- ¿Cómo puede representarse una clase a través de uml?
- Definir que es una instanciación
- Definir que es un objeto
- Instanciación de un objeto
- Acceso a atributos de un objeto
- Acceso a métodos de un objeto
- Definir la identidad de cada objeto instanciado
- Explicar como una variable referencial puede manipular 2 objetos

2.3. Encapsulamiento

El alumno entenderá el concepto de encapsulamiento, también conocido como ocultamiento de datos, es decir, limita el acceso a los atributos y métodos de los objetos que implemente en este capítulo, utilizando la técnica de aprendizaje basado en problemas.

Ejercicio

1. Lee detalladamente el siguiente escenario 2.3.
2. Con base en el escenario 2.3, realicen una serie de preguntas en equipo sobre la situación que en él se presenta. No olviden que las preguntas deben hacerse en el foro 2.3 de discusión, al final de la serie de preguntas:
 - a) De todas ellas, elijan la que consideren que mejor representa la situación a resolver y colóquela bajo el título "definición del problema".
 - b) A partir de dicha definición, elijan las 10 preguntas que consideren más relevantes para poder resolver el problema y colóquenlas en el foro bajo el título "metas de aprendizaje".

Una vez que tengan el visto bueno del profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas. Recuerden que ésta es individual.

3. Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1).

Páginas Electrónicas de Referencia

A continuación, se enuncian una serie de ligas que pueden brindarles información acerca de este tema. Cabe aclarar que se puede consultar cualquier otra fuente de información que les pudiera ayudar.

- <http://www.monografias.com/trabajos/objetos/objetos.shtml>
- <http://www.dcc.uchile.cl/~psalinas/uml/modelo.html#relacion>
- <http://www.inf.udec.cl/~mvaras/estprog/cap41.html#4.4>

Rúbrica 1 para evaluar el reporte de investigación individual Máximo 15 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluye	La incluye, pero no coincide con la del resto del equipo	La incluye, pero difiere en algunos puntos de las del resto del equipo	La incluye y coincide con el resto del equipo.	
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada. Entregar un código	
Referencias bibliográficas	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte en la sección de Tarea individual 2.3.

4. Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución a su pregunta de definición del problema, así como su reporte en donde se incluirá dicha propuesta y una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2).

Rúbrica 2 para evaluar el reporte final de ABP (en equipo) por parte del tutor. Máximo 15 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluyen.	La incluyen, pero no corresponde a las de los reportes individuales.	La incluyen, pero difiere ligeramente de las de los reportes individuales.	La incluyen y coincide al 100% con las de los reportes individuales.	
Solución o explicación al problema	El equipo no proporciona una solución/explicación al problema.	La solución/explicación del problema es vaga y sin fundamentos.	Dan una solución/explicación detallada al problema, pero no la fundamentan de manera completa.	Dan una solución/explicación detallada al problema y con fundamentos sólidos.	
Síntesis de la información con la cual fundamentan la solución/explicación	No la incluyen.	Únicamente presentan listas de datos y/o copia de lo que cada quien investigó, sin darle coherencia.	Presentan un resumen lógico de la información recopilada por todos.	Sintetizan de manera lógica y coherente la información investigada por todos.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Referencias bibliográficas	No las incluyen.	Las incluyen, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluyen de manera completa, pero no con el formato solicitado.	Las incluyen de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte final en la sección de Tarea en Equipo 2.3.

ESCENARIO

Será explicado por el tutor.

Realice sus anotaciones necesarias en el siguiente recuadro:

Guía Tutorial del Profesor

Materia: Programación Orientada a Objetos

Tema: Encapsulamiento

Objetivos de aprendizaje:

Programará el encapsulamiento en objetos considerando los permisos (niveles) de acceso.

Escenario:

El personal de ventas se ha quejado de que las cifras y el estado de las facturas no son las correctas, debido a que no es posible tener un número de factura en un estado de pedido, pues dicho número se asigna en el momento en que el pedido se coloca en el transporte, porque al cliente se le entregan factura y producto simultáneamente. Por otra parte, no es posible tener una factura en números negativos debido a que, para hacer una cancelación o devolución, se utiliza una nota de crédito. Las unidades vendidas de pz (piezas) no pueden llevar decimales.

CIA NACIONAL DE COBRE, S.A. DE C.V.

SISTEMA DE FACTURACION

(listado de ventas por producto)

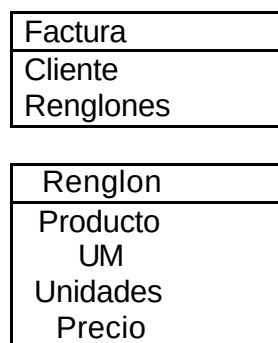
Del 5/abr/04 al 11/abr/04

Producto	Factura	Unidades	(UM)	Estado	Importe
Cobre 15 mm	10250	2	Kg	Pagada	300.00
Cobre 15 mm	10447	3	Kg	Pagada	450.00
Cobre 15 mm	10268	3	Kg	Pedido	450.00
Cobre 15 mm	10943	3	Kg	Pedido	450.00
Subtotal			11		1,650.00

Carrete 150 mm	10264	2	pz	Pagada	200.00
Carrete 150 mm	10565	3.34	pz	Pagada	-300.00
Carrete 150 mm	10746	5	pz	Pedido	500.00
Carrete 150 mm	10847	3	pz	Pagada	300.00
Subtotal		13.34			700.00

Al analizar el sistema nos encontramos con lo siguiente: este sistema cuenta con aproximadamente 50 clases; los errores que hay se deben principalmente a errores que realizan algunos componentes, ya que al utilizar las clases pasan información errónea de otros sistemas; también existen fallas cuando los programadores hacen cambios y por alguna situación en la nueva programación queda la información mal; otros son problemas en la captura de información.

Al revisar la clase nos encontramos con los siguientes métodos y propiedades: Aquí vienen dos clases de uml (factura y detalle)

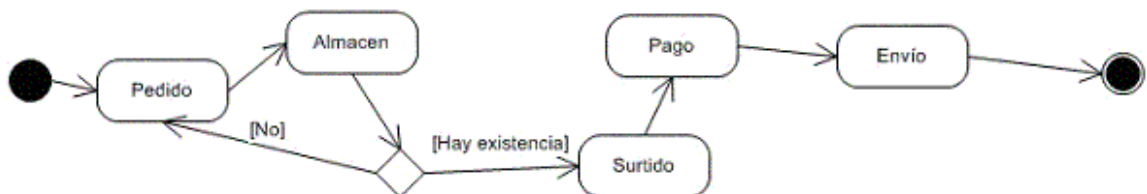


Después de analizar la situación, te solicitan que elabores y entregues la programación necesaria para resolverla.

5. Posibles términos que deban clarificarse:

Informe de facturación.- Es un listado (generalmente en papel) que muestra las ventas que se han tenido en la empresa en un periodo de tiempo el cual se muestra por diferentes agrupaciones.

Estado.- Son los posibles estados por los que puede pasar una factura



Nota de crédito.- Documento fiscal que permite hacer cancelaciones y/o modificaciones a una factura.

6. Definición del problema esperado:

¿Cómo solucionamos esto?

7. Preguntas esperadas en el análisis del problema:

- ¿Qué debo hacer?
- ¿Cómo se hacen las restricciones/permisos/niveles de acceso?
- ¿Cuál es la manera de validar/verificar la información?
- ¿Qué cambios debemos hacer en el código de la clase?

8. Metas de aprendizaje:

- ¿Cómo se hacen las restricciones?
- ¿Cuál es la manera recomendada de validar la información?

Programación Orientada a Objetos

Rúbrica para evaluar el reporte de investigación individual Máximo 12 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluye	La incluye, pero no coincide con la del resto del equipo	La incluye, pero difiere en algunos puntos de las del resto del equipo	La incluye y coincide con el resto del equipo.	
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Código	Esqueleto	Con lógica y compila	Funciona	Funciona, estructuración legible y documentados	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada. Entregar un código	
Referencias bibliográficas	No incluye las	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

**Rúbrica para evaluar el reporte final de ABP (en equipo) por parte del tutor.
Máximo 12 puntos.**

Contenido del reporte	0	1	2	3	Tot
Definición del problema	No la incluyen.	La incluyen, pero no corresponde a las de los reportes individuales.	La incluyen, pero difiere ligeramente de las de los reportes individuales.	La incluyen y coincide al 100% con las de los reportes individuales.	
Solución o explicación al problema	El equipo no proporciona una solución/explicación al problema.	La solución/explicación del problema es vaga y sin fundamentos.	Dan una solución/explicación detallada al problema, pero no la fundamentan de manera completa.	Dan una solución/explicación detallada al problema y con fundamentos sólidos.	
Síntesis de la información con la cual fundamentan la solución/explicación	No la incluyen.	Únicamente presentan listas de datos y/o copia de lo que cada quien investigó, sin darle coherencia.	Presentan un resumen lógico de la información recopilada por todos.	Sintetizan de manera lógica y coherente la información investigada por todos.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Referencias bibliográficas	No las incluyen.	Las incluyen, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluyen de manera completa, pero no con el formato solicitado.	Las incluyen de manera completa y correcta.	
TOTAL					

2.4. Herencia

El alumno entenderá el concepto de herencia y como una clase puede pasar a otra clase algunas de sus características a otra mediante la técnica de aprendizaje basado en problemas.

Ejercicio

1. Lee detalladamente el siguiente escenario 2.4.
2. Con base en el escenario 2.4. realicen una serie de preguntas en equipo sobre la situación que en él se presenta. No olviden que las preguntas deben hacerse en el foro 2.4. al final de la serie de preguntas:
 - a) De todas ellas, elijan la que consideren que mejor representa la situación a resolver y colóquenla bajo el título "definición del problema".
 - b) A partir de dicha definición, elijan las 10 preguntas que consideren más relevantes para poder resolver el problema y colóquenlas en el foro bajo el título "metas de aprendizaje".

Una vez que tengan el visto bueno del profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas. Recuerden que ésta es individual.

3. Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1).

Rúbrica 1 para evaluar el reporte de investigación individual Máximo 15 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluye	La incluye, pero no coincide con la del resto del equipo	La incluye, pero difiere en algunos puntos de las del resto del equipo	La incluye y coincide con el resto del equipo.	
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada. Entregar un código	
Referencias bibliográficas	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte en la sección de Tarea Individual 2.4.

Programación Orientada a Objetos

4. Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución a su pregunta de definición del problema, así como su reporte en donde se incluirá dicha propuesta y una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2).

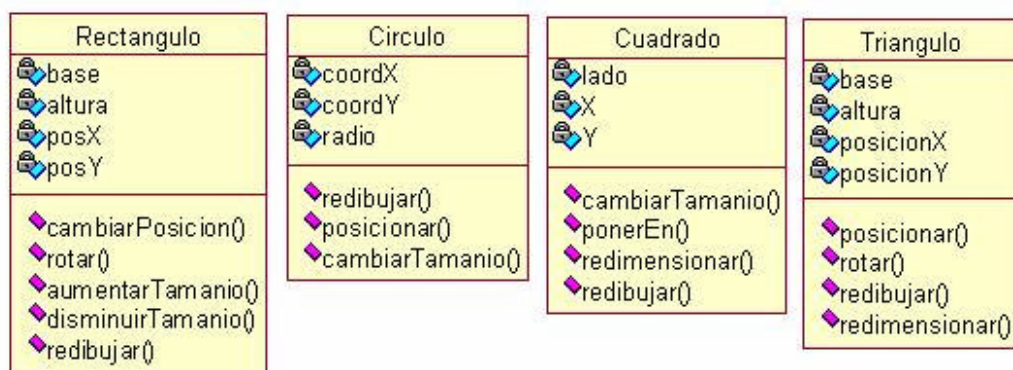
Rúbrica 2 para evaluar el reporte final de ABP (en equipo) por parte del tutor. Máximo 15 puntos.

Contenido del reporte	0	1	2	3	Total
Definición del problema	No la incluyen.	La incluyen, pero no corresponde a las de los reportes individuales.	La incluyen, pero difiere ligeramente de las de los reportes individuales.	La incluyen y coincide al 100% con las de los reportes individuales.	
Solución o explicación al problema	El equipo no proporciona una solución/explicación al problema.	La solución/explicación del problema es vaga y sin fundamentos.	Dan una solución/explicación detallada al problema, pero no la fundamentan de manera completa.	Dan una solución/explicación detallada al problema y con fundamentos sólidos.	
Síntesis de la información con la cual fundamentan la solución/explicación	No la incluyen.	Únicamente presentan listas de datos y/o copia de lo que cada quien investigó, sin darle coherencia.	Presentan un resumen lógico de la información recopilada por todos.	Sintetizan de manera lógica y coherente la información investigada por todos.	
Código	Esqueleto	Con lógica y compila	Funciona	Hace lo que tiene que hacer	
Referencias bibliográficas	No las incluyen.	Las incluyen, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluyen de manera completa, pero no con el formato solicitado.	Las incluyen de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte final en la sección de Tarea en Equipo 2.4.

Escenario 2.4.

Programadores distintos modelaron una clase de una figura geométrica cada uno. Las figuras geométricas fueron cuadrado, círculo, rectángulo y triángulo. Sus propuestas se tomarían como ideas iniciales para comenzar el análisis de un editor gráfico que utiliza figuras geométricas bajo un criterio uniforme. Al reunirse se dieron cuenta de una situación a nivel de atributos y métodos que podía solucionarse al crear una nueva clase llamada 'Figura Geométrica'. ¿Qué situación descubrieron? ¿Cómo la solucionaron?



Guía Tutorial del Profesor

Materia: Programación Orientada a Objetos

Tema : Herencia

Objetivos de aprendizaje

El Alumno:

- Programará aplicando el concepto de herencia
- Reconocerá los tipos de herencia y los problemas de diseño con la misma.

Técnica Didáctica: Aprendizaje Basado en Problemas (ABP)

Escenario A

Programadores distintos modelaron una clase de una figura geométrica cada uno. Las figuras geométricas fueron cuadrado, círculo, rectángulo y triángulo. Sus propuestas se tomarían como ideas iniciales para comenzar el análisis de un editor gráfico que utiliza figuras geométricas bajo un criterio uniforme. Al reunirse se dieron cuenta de una situación a nivel de atributos y métodos que podía solucionarse al crear una nueva clase llamada 'Figura Geométrica'. ¿Qué situación descubrieron? ¿Cómo la solucionaron?

Posibles términos que deban clarificarse

Definición esperada del problema

- ¿Qué es la herencia en la programación orientada a objetos?
- ¿Puedo a través de la herencia optimizar la utilización de atributos y métodos?
- ¿Es la herencia una estrategia de reutilización y una solución poderosa a través de objetos para programar comportamiento genérico de varias clases?

Preguntas esperadas en el análisis del problema

- ¿Por qué se repiten atributos y operaciones?
- ¿Puede abstraerse en una clase base los atributos y operaciones comunes?
- ¿Puede aplicarse la herencia?
- ¿Puede el cuadrado heredar del rectángulo?

Evidencias de Aprendizaje

- Heredar atributos y métodos entre clases
- Puede avanzarse progresivamente en el modelado
 - o Objetos concretos y tangibles de la vida ordinaria

- o Objetos que requieren un nivel mayor de abstracción
- Utilizar la palabra reservada extends
- Utilizar un diagrama de clases de UML
- Simular una interfaz, pero aclarar que tiene su manejo particular
 - o No usar la palabra interface
 - o Crear una situación que se complique al utilizar herencia en vez de interfaces
- Utilizar lenguaje de programación
 - o C#
 - Heredar de clases localmente en código
 - Heredar de clases en namespaces
 - o Java
 - Heredar de clases localmente en código
 - Heredar de clases en packages

Metas de aprendizaje

- ¿Cómo se programa la herencia en un lenguaje OO?
- ¿Cómo se administran los permisos de acceso para heredar atributos y métodos?
- ¿Qué relación tiene la herencia con las 'interfaces de clase'?
- ¿Qué tipos de herencia existen?
- ¿Cómo se representa la herencia en UML?
- ¿Por qué los lenguajes más recientes sólo incorporan la herencia simple?
- ¿Cómo se utiliza la herencia selectiva?

2.5. Polimorfismo

El alumno comprenderá que este concepto se refiere a cuando dentro de una clase pueden contener métodos con el mismo nombre pero se diferencian entre ellos por medio de su definición dentro de la clase por medio de la técnica de aprendizaje basado en problemas.

Ejercicio

1. Lee detalladamente el siguiente escenario 2.5.
2. Con base en el escenario 2.5, realicen una serie de preguntas en equipo sobre el problema a resolver. No olviden que las preguntas deben hacerse en el foro de discusión.

Al final de la serie de preguntas, elijan las 10 que consideren más relevantes y el secretario del equipo deberá colocarlas en el foro bajo el título "metas de aprendizaje". Una vez que tengan el visto bueno del profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas.

3. Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1 – Misma del tema 2.4).

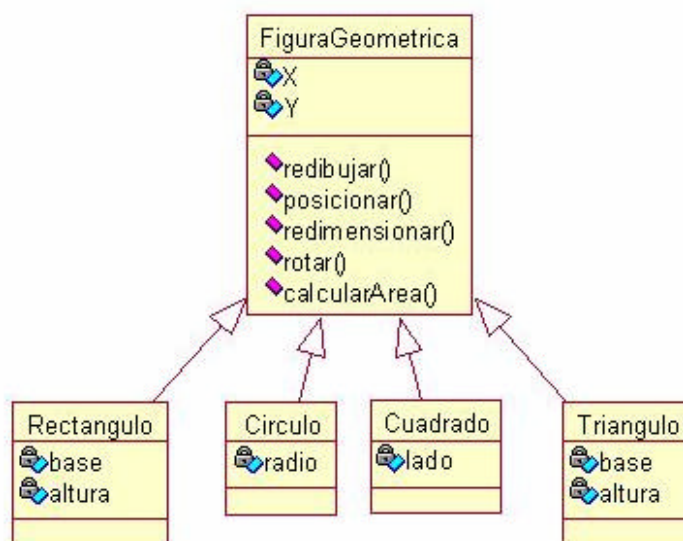
No olviden que deben colocar su reporte en la sección de Tarea individual 2.5.

4. Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución al problema, además de incluir en su reporte una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2 – Misma del tema 2.4).

No olviden que deben colocar su reporte final en la sección de Tarea en equipo 2.5.

ESCENARIO

La programación de un editor gráfico que maneja figuras geométricas (cuadrado, círculo, rectángulo, triángulo) se ha complicado ya que la clase base a todas ellas 'FiguraGeométrica' tiene métodos que se heredan (redibujar, posicionar, redimensionar, rotar, calcularArea); sin embargo cada vez que se agrega una figura geométrica distinta, como por ejemplo "rombo", tiene que modificarse el contenido de estos métodos conforme al tipo de figura, ya que varias de ellas se dibujan en forma distinta. El problema se agravó cuando la clase 'AreaGrafica' cuenta con un método que requiere redibujar las figuras geométricas de un dibujo independientemente del tipo de figura geométrica de que se trate en particular y para tal efecto no se desea estar modificando el código dentro de los métodos de 'AreaGrafica' y 'FiguraGeometrica' cada vez que se agrega una nueva clase representando a un nuevo tipo de figura geométrica. El editor gráfico trabaja tanto con pixeles como con centímetros; y en cuanto a ángulos, tanto con grados como con radianes. Se cuenta con un diagrama de clases. Son conscientes que más adelante incluirán otro tipo de figuras geométricas.



Se proporciona el código fuente ineficiente; que tiene a nivel base una cadena de condicionantes por tipo de clase.(LIGA A CODIGO FUENTE)

Guía Tutorial del Profesor

Materia: Programación Orientada a Objetos

Tema: Polimorfismo

Objetivos de Aprendizaje

El Alumno:

- Describirá los conceptos de polimorfismo
- Comprenderá su representación en UML
- Programará en distintos lenguajes de programación
- Evaluará tales implementaciones ¿cómo?

Técnica Didáctica: Aprendizaje Basado en Problemas (ABP)

Escenario A

La programación de un editor gráfico que maneja figuras geométricas (cuadrado, círculo, rectángulo, triángulo) se ha complicado ya que la clase base a todas ellas 'FiguraGeométrica' tiene métodos que se heredan (redibujar, posicionar, redimensionar, rotar, calcularArea); sin embargo cada vez que se agrega una figura geométrica distinta, como por ejemplo "rombo", tiene que modificarse el contenido de estos métodos conforme al tipo de figura, ya que varias de ellas se dibujan en forma distinta. El problema se agravó cuando la clase 'AreaGrafica' cuenta con un método que requiere redibujar las figuras geométricas de un dibujo independientemente del tipo de figura geométrica de que se trate en particular y para tal efecto no se desea estar modificando el código dentro de los métodos de 'AreaGrafica' y 'FiguraGeometrica' cada vez que se agrega una nueva clase representando a un nuevo tipo de figura geométrica. El editor gráfico trabaja tanto con pixeles como con centímetros; y en cuanto a ángulos, tanto con grados como con radianes. Se cuenta con un diagrama de clases. Son conscientes que más adelante incluirán otro tipo de figuras geométricas.

Se proporciona el código fuente ineficiente; que tiene a nivel base una cadena de condicionantes por tipo de clase (Codigo)

Posibles términos que deban clarificarse

Definición esperada del problema

- ¿Puedo programar métodos genéricos a partir de las clases base y ser a su vez útiles a clases derivadas?

Preguntas esperadas en el análisis del problema

- ¿Es posible utilizar el polimorfismo para alterar el comportamiento en clases derivadas?
- ¿Por qué se están programando las mismas funciones gráficas en cada figura geométrica?
- Si por el concepto de herencia, las clases derivadas heredan los métodos de las clases base; ¿Por qué no se definieron los métodos en la clase 'FiguraGeometrica'?
- ¿Por qué se cuentan con unidades de medida distintas para la misma característica?

Evidencias de aprendizaje

Métodos que puedan utilizar clases actuales y futuras que heredan características y comportamiento.

- Al utilizar clases que se definen en el futuro, se comprenderá la reutilización.
- Crear métodos que utilicen referencias de clases base y que sean instanciadas por clases derivadas
 - o Dentro de un método
 - o Pasando las instancias de clases derivadas como parámetros de tipo clase base
 - o Crear referencias de clases base que manejen instancias de objetos derivados y que accedan a los métodos de las clases derivadas
- Las operaciones son parecidas pero varían ligeramente
- Utilizar los mismos nombres de método tanto en
 - o Clases distintas relacionadas a través de la herencia

- o La misma clase
- Utilizar diagramas de UML
- Utilizar la palabra reservada (permiso de acceso) protected
- Utilizar lenguaje de programación
 - o C#
 - o Java

Metas de aprendizaje

- ¿El polimorfismo es la solución al problema?
- ¿Por qué no hay conflicto al tener métodos con el mismo nombre?
- ¿Por qué puede haber métodos con el mismo nombre mientras la firma del método cambie ligeramente?
- ¿Por qué puede haber nombres idénticos de métodos entre clases bajo una jerarquía de herencia y dentro de la misma clase?
- ¿Cuál es la ventaja de aplicar la 'sobrecarga de métodos' dentro de la misma clase?
- ¿Cómo se interpreta el diagrama de clases proporcionado?
- ¿Cómo se visualiza el polimorfismo en un diagrama de clases de UML?
- ¿Por qué puede instanciarse una clase base con una clase derivada?
- ¿Cómo encuentro el polimorfismo en el código fuente proporcionado?

2.6. Relación

El alumno comprenderá el concepto de relación e identificará las partes de una relación de clases, tales como nombre, roles, multiplicidad.

Ejercicio

1. Trabaje con el siguiente ejercicio 2.6 interactivo.

NOTA: La aplicación interactiva arroja una evaluación como resultado de los datos que se introduzcan. Se puede realizar el ejercicio interactivo el número de veces que se desee. Realícelo hasta obtener una calificación aceptable.

2. Utilice el foro 2.6 de discusión para intercambiar impresiones y comentarios.
3. Complete el recuadro del ejercicio 2.6.

Nombre de Relación	Dibujo	Descripción

Guía Tutorial del Profesor

Materia: Programación Orientada a Objetos

Tema: Relaciones

Objetivos de Aprendizaje

El alumno:

- Comprenderá el concepto de relación y sus tipos
- Evaluará situaciones de aplicación de los tipos de relaciones entre clases

Técnica Didáctica: Simulador

Escenario

Se utilizará una herramienta gráfica que presenta 4 secciones:

- Una situación
- Un diagrama con 2 clases
- Tipos de relación
- Comentarios

Se tendrán 8 reactivos. Para cada reactivo se dará una explicación en la primera sección, se pondrán 2 clases en la segunda sección para las que posteriormente se indicará la forma de relacionarse conforme a la explicación de la sección anterior; la tercera sección tendrá un listado gráfico con las posibles relaciones a utilizar, de la cual se podrá seleccionar una de ellas como opción; y la cuarta sección expresará un puntaje conforme a la relación seleccionada y dará una explicación adicional conforme a ésta. Se le indicará en qué casos es correcta y en cuáles no.

Posibles términos que deban clarificarse

Interfaz (de clase)

Realización

Definición esperada del problema

- ¿Para qué son y como se representan las relaciones entre clases?

Preguntas esperadas en el análisis del problema

- ¿Cuántos tipos de relaciones existen?
- ¿Puede tenerse un objeto como atributo a otra clase, o ser contenido dentro de otro objeto?
- ¿Qué información se le puede poner a la representación de una relación?
- ¿Qué condiciones debe cumplir la clase cuando se usa determinado tipo de relación?
- ¿Puede ponerse nombre a la relación?

Requerimientos para cumplir las metas de aprendizaje

- Agregación
 - o Declarar atributos o variables miembro como objetos
- Dependencia
 - o Utilizar referencias a clases a través de parámetros y variables locales
 - o Vislumbrar al alumno que es necesario, en ciertas ocasiones, crear un atributo que almacene la clave del objeto

- Forzar a la multiplicidad de relaciones
 - o 1
 - o n
 - o 0...n
 - o 1...n
 - o m...n
- Realización
 - o A través de la simulación de interfaces por medio de herencia
- Recordar que la relación de generalización se realiza a través de la herencia
- Presentar situaciones en las que es conveniente utilizar el nombre en la relación
- Presentar situaciones en las que es necesario utilizar los roles
 - o Situaciones comunes
 - o En la autorelación (una clase se relaciona consigo misma pero el nombre de rol cambia)
- Presentar situaciones en las que es necesario indicar la direccionalidad de la relación
- Utilizar lenguaje de programación
 - o C#
 - o Java

Metas de aprendizaje

- ¿Cuáles son los tipos de relaciones de clases en UML?
- ¿Cómo se representan las relaciones en UML?
- ¿Qué es la multiplicidad y cómo se representa?
- ¿Existe la direccionalidad de una relación?
- ¿En qué situaciones es conveniente utilizar las relaciones bidireccionales?
- ¿Qué son los estereotipos en las relaciones?
- ¿Para qué sirven los roles y cómo se representan?
- ¿Qué es, cómo se representa en UML y para qué se usa una relación de tipo asociación?
- ¿Qué es, cómo se representa en UML y para qué se usa una relación de tipo agregación?
- ¿Qué diferencia existe entre una agregación normal y una composición?
- ¿Qué es, cómo se representa en UML y para qué se usa una relación de dependencia?
- ¿Qué es, cómo se representa en UML y para qué se usa una relación de generalización?
- ¿Qué es, cómo se representa en UML y para qué se usa una relación de realización?
- ¿Puede una clase autorelacionarse?

2.7. Persistencia

Ejercicio

1. Lee detalladamente el siguiente escenario 2.7.
2. Con base en el escenario 27, realicen una serie de preguntas en equipo sobre el problema a resolver. No olviden que las preguntas deben hacerse en el foro de discusión.

Al final de la serie de preguntas, elijan las 10 que consideren más relevantes y el secretario del equipo deberá colocarlas en el foro bajo el título "metas de aprendizaje". Una vez que tengan el visto bueno del profesor sobre sus preguntas, lleven a cabo la investigación necesaria para responderlas.

3. Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 1 – Misma del tema 2.5).

No olviden que deben colocar su reporte en la sección de Tarea Individual 2.7.

4. Como segunda tarea, y con base en los reportes individuales de los integrantes de cada equipo, deberán elaborar una propuesta de solución al problema, además de incluir en su reporte una síntesis de las respuestas a sus preguntas relevantes finales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica 2 – Misma del tema 2.5).

No olviden que deben colocar su reporte final en la sección de Tarea en Equipo 2.7.

ESCENARIO

Tienes un amigo(a) que tiene más de 3000 discos compactos diversos de audio y video (CD y DVD). Te pide que le hagas un pequeño programa en el que pueda registrar sus discos compactos tanto actuales como los que vaya comprando. Y por otro lado, tu amigo(a) los presta a todos sus amigos, y desea poder registrar a quién se los presta; así como hacer consultas de quién le debe que disco compacto por diversos criterios de búsqueda (por persona, por tipo de disco compacto – audio o video -, categoría, fecha de préstamo, cantidad de discos prestados a una persona), así como, la combinación de algunos de estos criterios. ¿Cómo le harías si lo quieres hacer con un lenguaje orientado a objetos?

Guía Tutorial del Profesor

Materia: Programación Orientada a Objetos

Tema: Persistencia

Objetivos de Aprendizaje

- Programará el encapsulamiento en objetos considerando los permisos de acceso
- Programará el acceso a base de datos en lenguajes OO particulares

Escenario

Escenario A

Tienes un amigo(a) que tiene más de 3000 discos compactos diversos de audio y video (CD y DVD). Te pide que le hagas un pequeño programa en el que pueda registrar sus discos compactos tanto actuales como los que vaya comprando. Y por otro lado, tu amigo(a) los presta a todos sus amigos, y desea poder registrar a quién se los presta; así como hacer consultas de quién le debe que disco compacto por diversos criterios de búsqueda (por persona, por tipo de disco compacto – audio o video -, categoría, fecha de préstamo, cantidad de discos prestados a una persona), así como, la combinación de algunos de estos criterios. ¿Cómo le harías si lo quieres hacer con un lenguaje orientado a objetos?

Escenario B

Se desea implementar las altas, bajas, cambios y consultas del módulo de pedidos, facturas, entradas y salidas de almacén.

Posibles términos que deban clarificarse

Categoría

Definición esperada del problema

- ¿Cómo puede ayudarme la programación OO sobre una base de datos para administrar los discos compactos y el control de préstamo?

Preguntas esperadas en el análisis del problema

Escenario A

- ¿En que consiste y cómo se implementa la persistencia?
- ¿En dónde voy a grabar los datos?
- ¿Utilizaré una base de datos?
- ¿Una base de datos me facilitará la manipulación de datos, así como las consultas?
- ¿Cuál es la forma de grabar información a través de un lenguaje orientado a objetos?
- ¿Utilizaré SQL?
- ¿Cómo se establece la conexión a la base de datos?
- ¿Qué es un 'recordset' o 'resultset'?

Requerimientos para cumplir las metas de aprendizaje

- Crear la conexión a la base de datos
 - o Considerar las implicaciones de seguridad (identificación y autenticación)

- Permitir compartir una referencia de conexión a base de datos en toda la aplicación
- Utilizar instrucciones de SQL
- Utilización del 'recordset' para la consulta de registros
 - o Validar la no obtención de registros en consultas
- Implantar una arquitectura de 2 ó 3 capas, donde una de ellas sea la capa de datos
 - o Posible e inicialmente, la capa de datos podría estar fusionada con otra u otras capas; posteriormente se separaría.
- Considerar el control de errores
 - o Aunque sea un tema posterior
- Utilizar lenguaje de programación
- C#
 - o Uso de ADO.NET
- Java
 - o Uso de JDBC

Metas de aprendizaje

- ¿Qué es la Persistencia en la POO?
- ¿Qué tecnologías de acceso a datos existen utilizando OO y en qué consisten?
- ¿Qué métodos de acceso a datos existen utilizando OO y en qué consisten?
- ¿Cuál es el patrón o plantilla de código para acceso tradicional de manipulación de datos en una base de datos?

- ¿Cuál es el patrón o plantilla de código para acceso tradicional de consulta de información desde una base de datos?
- ¿Qué elementos en UML representan el acceso a una base de datos?

3. Elementos Adicionales del Lenguaje

Objetivo Específico:

- Comprenderá algunos elementos de los lenguajes orientados a objetos tales como estructuras de datos tipo arreglo, estructuras de dato especiales, interfaces, manejo de errores, archivos, eventos, interfaces gráficas del usuario y multihilado.

3.1 Estructura tipo arreglos

Ejercicio

1. Lee detalladamente el problema que se presenta :

Problema 3.1

2. Con base al problema, realice las preguntas necesarias hasta comprender lo que se necesita realizar coloque las preguntas y dudas en el foro.
3. Desarrolle la siguiente actividad en la que se muestran los conceptos básicos de arreglos de 1 dimensión.

Concepto, Operaciones, Práctica 1, Práctica 2

4. Lleven a cabo la investigación necesaria para responderlas. Recuerden que ésta es individual.
5. A continuación, se enuncian una serie de ligas que pueden brindarles información acerca de este tema. Cabe aclarar que se puede consultar cualquier otra fuente de información que les pudiera ayudar.

Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica – Rúbrica proporcionada por el tutor).

No olviden que deben colocar su reporte en la sección de Tarea Individual.

6. Construya el programa que resuelve el problema verifique su funcionamiento coloque en la sección de tareas los archivos que conforman el código fuente así como el ejecutable en la sección de tareas individuales, recuerde que sera evaluado con respecto a la rubrica 1 (Liga rúbrica – Rúbrica proporcionada por el tutor).
7. Coloque sus respuestas, algunas, sugerencias, escenarios del uso y utilización de arreglos de una dimensión en el foro de discusión, recuerde que será evaluado con la rubrica 1 (Liga rúbrica 1 – Rúbrica proporcionada por el tutor).

¿Cuántas maneras de definir un arreglo existen y en que tipo de situaciones se utiliza en cada una?

¿En que casos utilizaría arreglos de una dimensión?

PROBLEMA 1

Cuando se baraja un mazo de cartas, se toma el mazo completo y se divide en dos, posteriormente se juntan los dos montones en un nuevo mazo poniendo una carta del primer montón, y una carta del segundo, y así posteriormente hasta que no quede ninguna carta en ninguno de los montones. Cuando se “barajea” datos se hace de la misma forma, se toma la lista de datos y se divide en dos, posteriormente se juntan de nuevo pero poniendo el primer dato de la primera mitad y luego el primer dato de la segunda, y así hasta terminar los datos en las dos mitades.

Escribe un programa que “barajee” la lista de datos que se encuentra en las localidades de memoria de la primera columna de la 0 al 9 y escriba la lista “barajeadas” de la segunda columna 0 al 9. Tu programa debe barajear los datos en la forma antes descrita puede utilizar las columnas adicionales que desea.

Los datos de entrada son “x” el valor de datos a leer entre uno $1 \leq x \leq 10$, un número de “n” indicando el numero de barajeadas $1 \leq n \leq 5$

EJEMPLO:

x=7
n=1

Programación Orientada a Objetos

Localidad	Inicial	Final
0	7	7
1	8	10
2	9	8
3	10	12
4	12	9
5	13	13

Rúbrica 1 para evaluar el reporte de investigación individual. Máximo 9 puntos.

Contenido del reporte	0	1	2	3	Total
Metas de aprendizaje	No las incluye	Las incluye, pero no coinciden con las del resto del equipo	Las incluye, pero difieren en algunos puntos de las del resto del equipo	Las incluye y coinciden con el resto del equipo.	
Respuestas a las metas de aprendizaje	No responde de manera completa todas las metas de aprendizaje	Responde superficialmente y/o sin fundamentos.	Responde con cierto detalle, pero no fundamenta todas sus respuestas.	Les da una respuesta clara y fundamentada en la bibliografía consultada.	
Referencias bibliográficas	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

3.2 Estructura de datos especiales

Ejercicio

1. Lee detalladamente el problema que se presenta:
Liga al problema 3.2 (PROBLEMA UNO)
2. Lleve a cabo la investigación para solucionar el problema. Recuerden que ésta es individual.
Recuerden que como primera tarea, se debe elaborar un reporte individual que deberá incluir las respuestas a sus preguntas individuales hechas en el foro (metas de aprendizaje). El contenido del reporte debe cumplir con la siguiente rúbrica (Liga rúbrica).

Rúbrica para evaluar el trabajo individual Máximo 12 puntos.

Contenido del reporte	0	1	2	3	Total
Resumen Máximo una hoja y un programa	No incluye	Incluye los tipos de arreglos	Incluye definiciones y ejemplos de código	Incluye, definiciones y ejemplos de programas ejecutables	
Participación Foro	No participo o participo con "ok", "esta bien", "estoy de acuerdo"	Participo con ideas del concepto	Participo con ideas y código	Participo con ideas de posibles escenarios a utilizar	
Programas	No entrego	El programa funciona	Programas funcionan adecuadamente y tiene una estructuración legible	Los programas funcionan adecuadamente y están auto documentados (comentarios significativos al programa)	
Referencias bibliográficas	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

No olviden que deben colocar su reporte en la sección de Tarea Individual 3.2.

3. Construya el programa que resuelve el problema verifique su funcionamiento coloque en la sección de tareas los archivos que conforman el código fuente así como el ejecutable en la sección de tareas individuales, recuerde que será evaluado con respecto a la rúbrica (Liga rúbrica – Será indicada por el tutor).
4. Una sugerencia de optimizar el proceso sería ordenar la información y hacer una búsqueda binaria o hash sobre ella:
 - Animación de búsqueda binaria
 - Animación de clave hash
5. Modifique el programa para utilizar una construcción basada en clave (numero de palabra) y las palabras ordenadas a fin de optimizar el rendimiento del programa

Recuerde enviar su programa a la sección de tarea individual: Tarea Individual 3.2.

6. Coloque sus puntos de vista, ventajas y desventajas de la utilización de colecciones, ordenación y búsqueda:

¿En qué tipos de situaciones sugiere el uso de colecciones?

¿Qué beneficios se obtiene al ordenar la información para su búsqueda?

PROBLEMA 1

CODIFICACIÓN DE DICCIONARIO DINÁMICO

Un método común de comprensión de dato, es remplazar palabras dentro de un texto, por números indicando su posición en un diccionario. En la codificación de diccionario estático, el diccionario es conocido previamente, y puede ser problemática ya que es necesario tener el diccionario para entender el código. La codificación de diccionario dinámico elimina este problema derivando el diccionario a partir del texto a comprimir. El texto es procesado del inicio al final, iniciando con un diccionario vacío. Cuando encontramos una palabra que ya está en el diccionario, es reemplazada por el número indicando la posición de ésta. Cuando una se encuentra una palabra que no está en el diccionario, aparece tal y como es en el texto y se agrega al diccionario.

ENTRADA:

Cada caso consiste de varias líneas de texto formado tan sólo por letras minúsculas y espacios. Puedes asumir que ninguna palabra es mas larga de 20 letras y que ninguna línea de entrada es más larga de 80 caracteres y que no hay más de 100 líneas de entrada. Los casos de prueba están separados por una línea en blanco, y se deben comprimir cada uno por separado.

SALIDA:

El archivo de salida debe contener los casos de prueba comprimidos, utilizando el método de codificación por diccionario dinámico. Los fines de línea y los espacios deben de ser preservados tal y como en el archivo de entrada, con los diferentes casos de prueba separados por una línea en blanco.

EJEMPLO:

ENTRADA

el gato perseguía a el ratón mientras
el perro perseguía a el gato dentro de la casa del ratón

SALIDA

el gato perseguía a 1 ratón mientras
1 perro 3 4 1 2 dentro de la casa del 5

3.3 Interfaces

Ejercicio

1. Lea la siguiente información del escenario que define un caso de estudio 3.3.
2. Analice el código fuente del escenario y elabore el Reporte de Preparación Individual con base en el siguiente formato.

Rúbrica de Evaluación

Contenido del reporte	0	1	2	3	Total
Análisis de Información	No la incluye	Obtiene la información relevante del caso.	Organiza la información a través de una herramienta conceptual (mapa mental, diagrama de Ishikawa, etc)	Obtiene información relevante a partir del análisis de la información básica.	
Aspectos básicos del caso	No las presenta	Presenta algunos aspectos, pero no son relevantes para la solución del caso.	Adicionalmente establece una jerarquía de aspectos o un conjunto de relaciones entre estos	Obtiene aspectos adicionales a partir del análisis de aspectos inmediatos y básicos.	
Alternativas de Solución, Evaluación, Selección e Implantación.	No presenta ninguna alternativa. O sólo una de ellas.	Presenta más de una alternativa, así como las variables de evaluaciones son claras y medibles mediante un método o criterio bien establecido.	Adicionalmente realiza la evaluación y selección de la alternativa más viable.	Presenta todos los elementos del reporte individual; así como la explicación de una implantación pertinente de la solución.	
Referencias bibliográficas y ligas de Internet	No las incluye	Las incluye, pero de manera incompleta, ya sea en datos o en cantidad.	Las incluye de manera completa, pero no con el formato solicitado	Las incluye de manera completa y correcta.	
TOTAL					

Programación Orientada a Objetos

3. Discuta sus propuestas de solución con el equipo de trabajo en el siguiente foro de discusión.

4. Escriba un programa que elabore un reporte de los recursos de la organización involucrada en el caso. El reporte utiliza las interfaces para imprimir únicamente los nombres de los recursos.

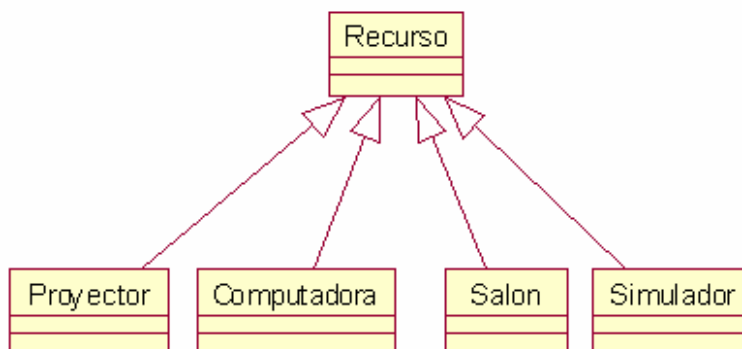
Rúbrica de Evaluación:

Aspectos del Programa	0	1	2	3	Total
Funcionalidad	No es posible apreciar su funcionalidad	Es posible apreciar la lógica de programación en forma concreta para lograr el objetivo	El programa funciona, pero los resultados son parciales o funciona sólo para un caso.	El programa cumple con su objetivo sin ningún problema.	
Usabilidad	El programa no cuenta con la información y presentación apropiada para poder ejecutarlo.	El programa presenta información apropiada para ejecutarlo.	El programa es fácil de usar pero presenta problemas al probar varios casos.	El programa es fácil de usar, con validaciones apropiadas, con ejemplos y ayudas pertinentes para que el usuario pueda utilizarlo rápidamente	
Documentación	No cuenta con documentación o es tan general que no es posible determinar con mayor precisión información del código	Se cuenta con documentación general del código fuente, cómo de los diversos módulos, clases, métodos, atributos, etc.	Se documentan además, líneas o secciones de código importantes	Se utilizan estándares de documentación, así como, posiblemente directivas que faciliten el uso de herramientas automatizadas de documentación	
TOTAL					

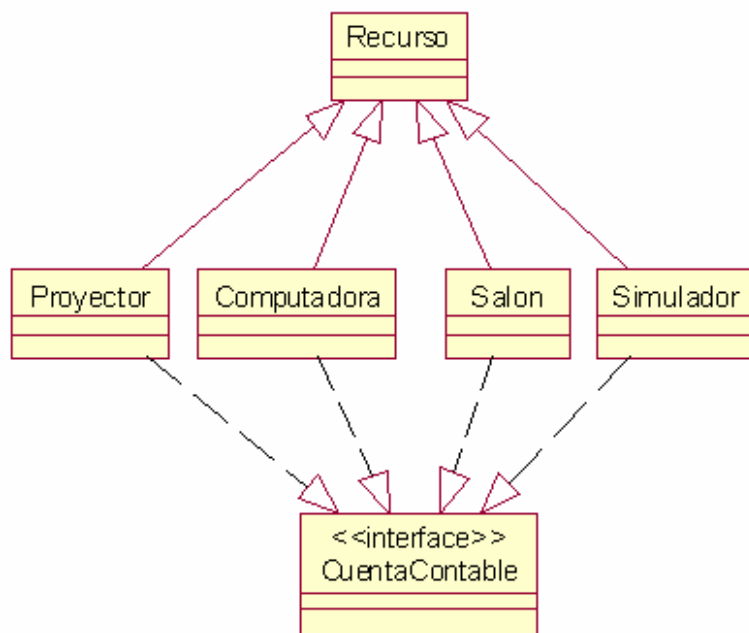
CASO DE ESTUDIO

La compañía Diseño Unificado y Especializado de Software S.A. de C.V. (DEUS) construyó un sistema para la administración de la capacitación llamado Platón a un centro de capacitación de líneas aéreas del país. Platón tiene la facultad de asignar recursos como computadora, cañón de proyección, salón, simulador, etc. a cada uno de los cursos programados. El centro de capacitación manifestó en el mes de agosto del 2001, la necesidad futura de poder asignar un número de cuenta contable a cada uno de estos recursos; así como, las reglas para obtener dinámicamente el número de cuenta contable de cada recurso. Para tal efecto, le fue solicitado a DEUS un preanálisis de la solución a esta situación.

A Mauricio Hidalgo, Director de Operaciones de DEUS, le es encomendada la tarea de elaborar el preanálisis. Como primer paso a realizar, se le solicita presentar un programa prototipo para la implementación de 'CuentaContable' a las clases que heredan de la clase 'Recurso' en el Sistema Platón. Originalmente, por razones de diseño, no es posible hacerlo mediante la aplicación del concepto de herencia, puesto que la clase 'Recurso' no puede ser modificada ya que es utilizada en otros sistemas en los cuales no se requiere tal modificación. Es necesaria la implementación de 'CuentaContable' de tal forma que en un futuro, puedan realizarse sobre clases que heredan o no de 'Recurso'.



Mauricio consideró que la utilización del concepto de 'interfaz' (en inglés interface) es el concepto adecuado a aplicar; realizó un diagrama de clases de la solución, el cual fue el siguiente:



Elaboró un esqueleto de código como punto de partida.
(Se proporcionará el código a través de Internet)

Ahora se pregunta si es la forma de realizarlo en código de programación, y probar algunos casos futuros en los que la interfaz “CuentaContable” se utilice para clases que heredan o no de la clase “Recurso”. De alguna forma se cuestiona si fue la opción más conveniente o existe una forma más óptima.

3.4. Manejo de Errores

Ejercicio

Martín Ibarra se encontraba revisando un reporte de facturación el cual presentaba una falla al leer un archivo tipo texto (txt) decidió aislar el problema en la siguiente pieza de código que lee el archivo y muestra su contenido al revisar el código se encontró con que la falla principal se debe a que a veces falta el archivo y en ocasiones este tenía otro nombre, entonces pensó ¿Será posible modificar el código para controlar esta situación?

```
using System;
using System.IO;

class Retry
{
    static void Main()
    {
        StreamReader sr;
        sr = new StreamReader("entrada.txt");

        string s;
        while (null != (s = sr.ReadLine()))
        {
            Console.WriteLine(s);
        }
        sr.Close();
    }
}
```

3.5 Archivos

Ejercicio

1. Vea detenidamente la siguiente animación y su correspondiente explicación.



2. Consulte la siguiente bibliografía y referencias en Internet:
 - a. Archer, Tom / Whitechapel Andrew. Inside C#. Microsoft Press. 2ª. Edición. Capítulo 11.
3. Transcriba los siguientes programas
 - a. Ejemplo 1. Uso genérico de Streams
 - b. Ejemplo 2. Uso de streams binarios
4. Conteste el siguiente cuestionario:
 - a. ¿Qué es un stream y cuál es su origen?
 - b. ¿Cuáles son las ventajas que ofrece un stream?
 - c. ¿Para qué sirven las clases “Reader” y “Writer”?
 - d. ¿Qué se requiere para poder utilizar nombres y rutas de archivos?
 - e. ¿Qué se requiere para poder obtener información de elementos del sistema de archivos, tales como directorios y archivos?
5. Elabore un programa que haga la copia binaria de un archivo. Como prueba tome un programa ejecutable que Ud. haya creado (inofensivo), cópielo y ejecute la copia.

Programación Orientada a Objetos

Rúbrica de Evaluación:

Aspectos del Programa	0	1	2	3	Total
Funcionalidad	No es posible apreciar su funcionalidad	Es posible apreciar la lógica de programación en forma concreta para lograr el objetivo	El programa funciona, pero los resultados son parciales o funciona sólo para un caso.	El programa cumple con su objetivo sin ningún problema.	
Usabilidad	El programa no cuenta con la información y presentación apropiada para poder ejecutarlo.	El programa presenta información apropiada para ejecutarlo.	El programa es fácil de usar pero presenta problemas al probar varios casos.	El programa es fácil de usar, con validaciones apropiadas, con ejemplos y ayudas pertinentes para que el usuario pueda utilizarlo rápidamente	
Documentación de Código	No cuenta con documentación o es tan general que no es posible determinar con mayor precisión información del código	Se cuenta con documentación general e indentación del código fuente, así como de los diversos módulos, clases, métodos, atributos, etc.	Se documentan además, líneas o secciones de código importantes	Se utilizan estándares de documentación, así como, posiblemente directivas que faciliten el uso de herramientas automatizadas de documentación	
TOTAL					

6. Transcriba el siguiente programa

7. Conteste el siguiente cuestionario:

- Elabore un diagrama de clases conteniendo las clases necesarias para utilizar archivos tanto texto como en forma binaria.
- ¿Cuáles son las diferencias básicas entre la utilización de archivos binarios y de texto?
- ¿Qué otros tipos de streams se pueden utilizar a través del lenguaje de programación que utilizas?

8. Elabore un programa que analice un archivo de código fuente (texto) que reporte los siguientes resultados:

Programación Orientada a Objetos

- a. Líneas de Código
 - i. Totales
 - ii. Comentarios (de línea y multilínea, no hay anidación de comentarios)
 - iii. No vacías
- b. Instrucciones del lenguaje de programación (por ejemplo: if-else son una sola instrucción)
- c. Métodos (Opcional, según indicación del docente o tutor)
 - i. Número promedio de métodos por clase
 - ii. Totales

Rúbrica de Evaluación (por programa):

Aspectos del Programa	0	1	2	3	Total
Funcionalidad	No es posible apreciar su funcionalidad	Es posible apreciar la lógica de programación en forma concreta para lograr el objetivo	El programa funciona, pero los resultados son parciales o funciona sólo para un caso.	El programa cumple con su objetivo sin ningún problema.	
Usabilidad	El programa no cuenta con la información y presentación apropiada para poder ejecutarlo.	El programa presenta información apropiada para ejecutarlo.	El programa es fácil de usar pero presenta problemas al probar varios casos.	El programa es fácil de usar, con validaciones apropiadas, con ejemplos y ayudas pertinentes para que el usuario pueda utilizarlo rápidamente	
Documentación	No cuenta con documentación o es tan general que no es posible determinar con mayor precisión información del código	Se cuenta con documentación general del código fuente, cómo de los diversos módulos, clases, métodos, atributos, etc.	Se documentan además, líneas o secciones de código importantes	Se utilizan estándares de documentación, así como, posiblemente directivas que faciliten el uso de herramientas automatizadas de documentación	
TOTAL					

3.6 Eventos

Ejercicio

1. Analice el siguiente caso

Material 3.6

2. Con base en la alternativa propuesta en el caso transcriba el código fuente y realice pruebas de ejecución
3. Conteste el siguiente cuestionario
 - a. ¿Qué es un delegado y cómo se define en el lenguaje de programación?
 - b. ¿Por qué se utilizan delegados para implantar el control de eventos?
 - c. ¿Qué es un manejador de eventos?
 - d. ¿Qué es la programación asíncrona?
 - e. ¿Qué ventajas ofrece la programación asíncrona contra la síncrona?
 - f. ¿Cuándo no se recomienda utilizar la programación síncrona?
4. (Opcional) Elabore un programa que genere una orden de compra al llegar al inventario mínimo sobre un producto que es elegido para surtir un pedido.

Aspectos del Programa	0	1	2	3	Total
Funcionalidad	No es posible apreciar su funcionalidad	Es posible apreciar la lógica de programación en forma concreta para lograr el objetivo	El programa funciona, pero los resultados son parciales o funciona sólo para un caso.	El programa cumple con su objetivo sin ningún problema.	
Usabilidad	El programa no cuenta con la información y presentación apropiada para poder ejecutarlo.	El programa presenta información apropiada para ejecutarlo.	El programa es fácil de usar pero presenta problemas al probar varios casos.	El programa es fácil de usar, con validaciones apropiadas, con ejemplos y ayudas pertinentes para que el usuario pueda utilizarlo rápidamente	

Documentación	No cuenta con documentación o es tan general que no es posible determinar con mayor precisión información del código	Se cuenta con documentación general del código fuente, cómo de los diversos módulos, clases, métodos, atributos, etc.	Se documentan además, líneas o secciones de código importantes	Se utilizan estándares de documentación, así como, posiblemente directivas que faciliten el uso de herramientas automatizadas de documentación	
TOTAL					

Material 3.6.

Creación y Utilización de Delegados

Los delegados permiten escribir código que dinámicamente cambien los métodos que son llamados. Esta es una característica flexible que permite a un método variar independientemente del código que lo invoca.

En esta lección analizará un escenario donde el uso de delegados es útil, así como, será capaz de definir y utilizar delegados.

Escenario: La Planta de Energía Nuclear

Para entender cómo utilizar delegados, considere el ejemplo de una planta de energía nuclear para la cual la utilización de delegados es una solución adecuada.

El Problema

En una planta de energía nuclear, la temperatura de un reactor nuclear debe mantenerse por debajo de una temperatura crítica. Pruebas internas del núcleo constantemente monitorean la temperatura. Si la temperatura se eleva significativamente, varias bombas necesitarán encenderse para incrementar el flujo de refrigerante a través del núcleo. El software que controla el trabajo del reactor nuclear debe comenzar con las bombas apropiadas en el momento apropiado.

Posibles Soluciones

El software de control puede ser diseñado de varias maneras de tal forma que cumpla con estos requerimientos, dos de estas formas pueden ser:

El software controlador de las bombas de refrigerante puede constantemente medir la temperatura del núcleo e incrementar el flujo de refrigerante como la temperatura lo requiere.

El dispositivo de monitoreo de la temperatura del núcleo puede encender a través del software las bombas de refrigerante apropiadas solamente en el momento que la temperatura cambie a un rango de temperatura crítica.

Ambas técnicas tienen inconvenientes. En la primera, la frecuencia con la que la temperatura es medida o sensada requiere ser determinada en ciertos lapsos de tiempo (por ejemplo, en milisegundos). Medir demasiado frecuente puede afectar la operación de las bombas debido a que el software tiene que controlar las bombas al paso, conforme monitorea la temperatura del núcleo. Medir poco frecuente puede significar que un cambio muy rápido en la temperatura no pueda ser percibido hasta que sea demasiado tarde.

En la segunda, habrá varias bombas y controladores que requerirán ser activados en cada cambio de temperatura. La programación requerida para realizarlo puede ser compleja y difícil de mantener, específicamente si hay diferentes tipos de bombas en el sistema que requieran ser activados de diferentes formas.

Contextualización del Problema

Antes de comenzar a resolver el problema, considere la dinámica involucrada en la implantación de una solución para el escenario de una planta de energía nuclear.

Consideraciones actuales

Un aspecto relevante es que pueden existir varios tipos de bombas de refrigeración provistas por diversos fabricantes, cada una con su propio software de control. El componente de monitoreo de la temperatura del núcleo tendrá que reconocer para cada tipo de bomba, qué método llamar para encender cada una de ellas.

Para este ejemplo suponga 2 tipos de bombas :

- 1.- Eléctricas, y
- 2.- Neumáticas.

Cada tipo de bomba tiene su propio controlador de software que contiene un método, el cual, enciende la bomba.

```
public class ControladorBombaElectrica
{
    ...
    public void IniciarEjecucionBombaElectrica( )
    {
        ...
    }
}

public class ControladorBombaNeumatica
{
    ...
    public void Encender( )
    {
        ...
    }
}
```


El componente de software para el monitoreo de la temperatura del núcleo encenderá las bombas. El código siguiente muestra la parte principal de este componente, la clase **MonitorTempNucleo**. Ésta crea un número de bombas y las almacena en un **ArrayList**, una clase colección que implementa un arreglo de longitud variable. El método **EncenderTodasLasBombas** itera a través del **ArrayList**, determina el tipo de bomba, e invoca al método apropiado para encender la bomba:

```
public class MonitorTempNucleo
{
    public void Add(object bomba)
    {
        bombas.Add(bomba);
    }
    public void EncenderTodasLasBombas()
    {
        foreach (object bomba in bombas) {
            if (bomba is ControladorBombaElectrica) {
                ((ControladorBombaElectrica)bomba).IniciarEjecucionBombaElectrica();
            }
            if (bomba is ControladorBombaNeumatica) {
                ((ControladorBombaNeumatica)bomba).Encender();
            }
            ...
        }
    }
    ...
    private ArrayList bombas = new ArrayList();
}

public class Ejemplo
{
    public static void Main( )
    {
        MonitorTempNucleo ctm = new MonitorTempNucleo();
        ControladorBombaElectrica ed1 =
            new ControladorBombaElectrica();
        ctm.Add(ed1);
        ControladorBombaNeumatica pd1 =
            new ControladorBombaNeumatica();
        ctm.Add(pd1);
        ctm.EncenderTodasLasBombas();
    }
}
```

Consideraciones futuras

Utilizar la estructura como ha sido descrita tiene serios inconvenientes. Si un nuevo tipo de bomba es instalada posteriormente, necesitará cambiar el método **EncenderTodasLasBombas** para incorporar la nueva bomba. Esto implicaría que todo el código volviera a ser probado, con factores de costo y tiempos fuera asociados dado que es una pieza crucial del software.

Una Solución

Para resolver este problema, puede utilizarse un mecanismo referido como delegado. El método **EncenderTodasLasBombas** puede usar el delegado para invocar el método apropiado para encender cada bomba inclusive sin la necesidad de determinar el tipo de bomba.

Creación de Delegados

Un delegado contiene una referencia a un método más que a un nombre de método. Al utilizar delegados, puede invocarse a un método sin conocer su nombre. Invocando a un delegado verdaderamente se ejecutará el método referenciado.

En el ejemplo de la planta de energía nuclear, en lugar que el **ArrayList** tenga objetos tipo **bomba**, puedes utilizarlo para contener delegados que apunten a los métodos requeridos para cada tipo de bomba.

Un delegado es similar a una interfaz. Especifica un contrato entre el llamador y el implementador. Un delegado asociado a un nombre con la especificación de un método. Una implementación de un método puede ser referida a este nombre, y un componente puede llamar al método a través de él. El requerimiento primario de implementar métodos consiste en que deben tener la misma firma y regresar el mismo tipo de parámetros. En el caso del escenario de la planta de energía nuclear, los métodos **IniciarEjecucionBombaElectrica** y **Encender** ambos son de **tipo void** y no tienen parámetros.

Para utilizar un delegado, debe primero definirlo y después instanciarlo.

Definir Delegados

Un delegado especifica un tipo de regreso y parámetros que cada método debe proveer. Se utiliza la siguiente sintaxis para definir a un delegado.

```
public delegate void EncenderBombaCallback( );
```

Advierta que la sintaxis para definir un delegado es similar a la sintaxis para definir un método. En este ejemplo se define un delegado **EncenderBombaCallBack** como si fuera un método que no regresa valor (void) y no tiene parámetros. Esto coincide con las especificaciones de los métodos **IniciarEjecucionBombaElectrica** y **Encender** en los dos tipos de controladores de bombas.

Instanciar Delegados

Después que se han definido los delegados, debe instanciarse y hacerse referencia a un método. Para instanciar un delegado, utilice un constructor de delegado para proveer el método del objeto que debe invocar cuando sea llamado. En el siguiente ejemplo, un **ControladorBombaElectrica**, **ed1**, se crea, y posteriormente un delegado, **callback**, se instancia, referenciado como el método **IniciarEjecucionBombaElectrica** de **ed1**:

```
public delegate void EncenderBombaCallback( );
```

```
void Ejemplo()
{
    ControladorBombaElectrica ed1 =
        new ControladorBombaElectrica( );

    IniciarBombaCallback callback;
    callback =
        ↳ new IniciarBombaCallback
        ↳ (ed1.IniciarEjecucionBombaElectrica);
    ...
}
```

Utilizar Delegados

Un delegado es una variable que invoca un método. Se invoca de la misma manera que un método, excepto que un delegado reemplaza el nombre del método. Ejemplo:

A continuación se muestra la forma de definir, crear e invocar delegados para usarse por la planta de energía nuclear. Se utiliza un **ArrayList** llamado **callbacks** con instancias de los delegados que hacen referencia a los métodos utilizados para encender cada bomba. El método **EncenderTodasLasBombas** itera a través del **ArrayList** e invoca al delegado en turno. Con delegados, el método no requiere verificar tipos de dato; siendo esto, mucho más sencillo que la solución anterior.

```
public delegate void EncenderBombasCallback( );

public class MonitorTempNucleo2
{
    public void Add(EncenderBombasCallback callback)
    {
        callbacks.Add(callback);
    }

    public void EncenderTodasLasBombas( )
    {
        foreach(EncenderBombasCallback callback in callbacks)
        {
            callback( );
        }
    }

    private ArrayList callbacks = new ArrayList( );
}

class Ejemplo
{
    public static void Main( )
    {
        MonitorTempNucleo2 ctm = new MonitorTempNucleo2( );

        ControladorBombaElectrica ed1 =
            new ControladorBombaElectrica( );
        ctm.Add(
            new EncenderBombasCallback(
```

```
        ed1.IniciarEjecucionBombaElectrica)
    );

    ControladorBombaNeumatica pd1 =
        new ControladorBombaNeumatica( );
    ctm.Add(
        new EncenderBombasCallback(pd1.Encender)
    );

    ctm.EncenderTodasLasBombas( );
}
}
```

Definir y Utilizar Eventos

En el ejemplo de la planta de energía nuclear, se empleó el concepto de delegado para resolver el problema de cómo encender los diferentes tipos de bombas en forma genérica. Sin embargo, el componente de software que monitorea la temperatura del núcleo del reactor es todavía responsable de notificar a cada una de las bombas en turno que necesitan encenderse. Una variante para resolver este aspecto de notificación mediante el uso de eventos.

Los eventos permiten a un objeto notificar asincrónicamente otro u otros objetos que un cambio ha ocurrido en el sistema. Los otros objetos pueden registrar un interés en un evento, y ser notificados cuando el evento ocurra.

Forma de Operación de los Eventos

Los eventos permiten a los objetos registrar un interés en cambios en el sistema hacia otros objetos. En otras palabras, los eventos permiten a los objetos que requieren ser notificados acerca de los cambios a otros objetos. Los eventos utilizan el modelo Editor-Suscriptor.

Editor

Un editor es un objeto que mantiene su estado interno. Sin embargo, cuando su estado cambia, puede disparar un evento para alertar a otros objetos interesados acerca del cambio.

Suscriptor

Un suscriptor es un objeto que registra un interés en un evento. Es alertado cuando los editores disparan un evento. Un evento puede tener 0 ó más suscriptores.

Los eventos pueden ser muy complejos. Para facilitar su entendimiento y mantenimiento hay guías que lo permiten.

Programación de Eventos

Los eventos utilizan delegados para invocar métodos de objetos suscritos. Son punto-multipunto. Esto significa que cuando un editor dispara un evento, puede resultar que varios delegados son invocados. Sin embargo, no puede establecerse con precisión el orden en que los delegados son invocados. Si uno de los delegados provoca una excepción, podría colapsar el procesamiento de órdenes, provocando que los delegados restantes por ejecutar no sean invocados.

Definir un Evento

Para definir un evento, un editor primero define un delegado y establece el evento sobre éste. El siguiente código define un delegado llamado **EncenderBombasCallback** y un evento llamado **SobreCalentamientoNucleo** que invoca al delegado **EncenderBombasCallback** cuando se dispara:

```
public delegate void EncenderBombasCallback( );  
private event EncenderBombasCallback SobreCalentamientoNucleo;
```

Suscribir a un evento

Al suscribir objetos se especifica un método a ser llamado cuando un evento se dispara. Si el evento no ha sido instanciado, al suscribir objetos se especifica un delegado que hace referencia al método cuando se crea el evento. Si el evento existe, al suscribir objetos se agrega un delegado que invoca al método cuando el evento se dispara.

Por ejemplo, en el escenario de la estación de potencia, se crean dos controladores de bombas y tenerlos suscritos en el evento **SobreCalentamientoNucleo**:

```
ControladorBombaElectrica ed1 = new ControladorBombaElectrica( );  
ControladorBombaNeumatica pd1 = new ControladorBombaNeumatica( );  
...  
SobreCalentamientoNucleo = new EncenderBombasCallback(  
    ↪ed1.IniciarEjecucionBombaElectrica);  
SobreCalentamientoNucleo += new EncenderBombasCallback(pd1.Encender);
```

Nota: Deben declararse los delegados (y métodos) que son utilizados para suscribirse a un evento como **void**. Esta restricción no aplica cuando un delegado se utiliza sin un evento.

Notificar a Suscriptores de un Evento

Para notificar a los suscriptores, debe dispararse el evento. La sintaxis utilizada es la misma para invocar a un método o un delegado. En el escenario de la planta de energía nuclear, el método **EncenderTodasLasBombas** del componente de monitoreo de la temperatura del núcleo ya no requiere estar iterando en la lista de delegados:

```
public void EncenderTodasLasBombas( )
{
    if (SobreCalentamientoNucleo != null) {
        SobreCalentamientoNucleo( );
    }
}
```

Ejecutar los eventos de esta manera provocará que todos los delegados sean invocados, y en este ejemplo, todas las bombas que están suscritas al evento se activan. Note que el código checa primero que el evento tiene por lo menos un delegado suscrito. Sin esta verificación, el código lanza una excepción si no hay suscriptores.

3.7 Interfaces Graficas del Usuario

Ejercicio

1. El instructor demostrará la manera de insertar controles básicos y comunes en un formulario.
 - a. Creación básica de un formulario con controles básicos
 - b. Utilización de controles básicos
2. Elabore un Programa con las Interfaces Propuestas por el Instructor

3.8 Multihilado

Ejercicio

1. Transcriba el siguiente código y elabore pruebas de ejecución

```
using System;
using System.Threading;

class Sum
{
    public Sum(int op1, int op2)
    {
        Console.WriteLine("[Sum.Sum] Instantiated with " +
            "values of {0} and {1}", op1, op2);
        this.op1 = op1;
        this.op2 = op2;
    }
    int op1;
    int op2;
    int result;
    public int Result{ get { return result; } }
    public void Add()
    {
        // Simulate work
        Thread.Sleep(5000);
        result = op1 + op2;
    }
}
```



```
    }  
};  
  
class ThreadData  
{  
    static void Main()  
    {  
        Console.WriteLine("[Main] Instantiating the Sum " +  
            "object and passing it the values to add");  
        Sum sum = new Sum(6, 42);  
  
        Console.WriteLine("[Main] Starting a thread using " +  
            "a Sum delegate");  
        Thread thread = new Thread(new ThreadStart(sum.Add));  
        thread.Start();  
        // Here we're simulating doing some work before  
        // blocking on the Add method's completion.  
        Console.WriteLine("[Main] Doing other work");  
        for (int i = 0; i < 10; i++)  
        {  
            Thread.Sleep(200);  
            Console.WriteLine(".");  
        }  
        Console.WriteLine("\n[Main] Waiting for Add to finish");  
        thread.Join();  
  
        Console.WriteLine("[Main] The result is {0}",  
            sum.Result);  
        Console.ReadLine();  
    }  
}
```

4. Fundamentos de UML

Ejercicio

1. Lee detalladamente el siguiente material que se encuentra dividido en tres secciones.
 - 4.1 UML (Unified Modeling Language- Lenguaje Unificado de Modelado).
 - 4.2. Diagramas de UML 1a. parte.
 - 4.3. Diagramas de UML 2a. parte.
2. Con base a lo que leíste, se hacen una serie de preguntas que debes contestar con tus propias palabras y de manera individual al término de cada sección. El contenido del reporte debe cumplir con la siguiente rúbrica

Rúbrica para evaluar el cuestionario de investigación individual
Máximo 9 puntos.

Contenido del reporte	0	1	2	3	Total
Respuestas a las preguntas	No responde de manera completa a todas las preguntas y no responde con sus propias palabras.	Responde superficialmente y/o sin fundamentos, además no responde con sus propias palabras.	Responde con cierto detalle, pero no fundamenta todas sus respuestas o no responde con sus propias palabras.	Les da una respuesta clara y fundamentada en la bibliografía consultada, además de responder con sus propias palabras.	
TOTAL					

Material 1

UML (Unified Modeling Language)

1. Introducción

El Lenguaje Unificado de Modelado (UML) es un lenguaje estándar para escribir planos de software. UML puede utilizarse para visualizar, especificar, construir y documentar los artefactos de un sistema que involucra una gran cantidad de software.

UML es apropiado para modelar desde sistemas de información en empresas hasta aplicaciones distribuidas basadas en la web, e incluso para sistemas empujados de tiempo real. Es un lenguaje muy expresivo, que cubre todas las vistas necesarias para desarrollar y luego desplegar tales sistemas.

1.1. LA IMPORTANCIA DE MODELAR

Cuando se requiere construir un software mucho muy grande, el problema es algo más que escribir grandes cantidades de código. De hecho, lo mejor sería crear el software apropiado escribiendo el menor código posible (software de calidad). En ocasiones, muchos proyectos empiezan muy pequeños, pero crecen hasta un tamaño inimaginable, se convierten en un software exitoso. Llega un momento en el que, si no se ha tenido en cuenta la arquitectura, el proceso o las herramientas, el pequeño software, ahora convertido en un mega software, se colapse bajo su propio peso.

Los proyectos de software que fracasan lo hacen por circunstancias propias, pero todos los proyectos con éxito se parecen en muchos aspectos. Hay muchos elementos que contribuyen a una empresa de software con éxito; uno en común es el uso del modelado.

El modelado es una técnica de ingeniería probada y aceptada. Se construyen modelos arquitectónicos desde casas y rascacielos para ayudar a los usuarios a visualizar el producto final. Incluso se construyen modelos matemáticos para analizar los efectos de vientos o terremotos sobre los edificios.

El modelado no es sólo parte de la industria de la construcción. Sería inconcebible crear una nueva aeronave o automóvil sin construir previamente modelos, desde modelos informáticos a modelos físicos para el túnel de viento y prototipos a escala real. Los nuevos dispositivos eléctricos, desde los microprocesadores a las centrales telefónicas, requieren algún grado de modelado para comprender mejor el sistema y comunicar las ideas a otros. En la industria cinematográfica, la técnica del storyboarding (representación de una película con viñetas), que es una forma de modelado, es fundamental en cualquier producción. En los campos de la sociología, la economía y la gestión empresarial, se construyen modelos para poder validar las teorías o probar las nuevas con un costo y riesgos mínimos.

¿Qué es, entonces, un modelo?

Un modelo es una simplificación de la realidad.

Un modelo proporciona los planos de un sistema. Los modelos pueden involucrar planos detallados, así como planos generales que ofrecen una visión global del sistema en consideración. Un buen modelo incluye aquellos elementos que tienen una gran influencia y omite aquellos elementos menores que no son relevantes para el nivel de abstracción dado. Todo sistema puede ser descrito desde diferentes perspectivas utilizando diferentes modelos, y cada modelo es por tanto una abstracción semánticamente cerrada del sistema. Un modelo puede ser estructural, destacando la organización del sistema, o puede ser de un comportamiento, resaltando su dinámica.

¿Por qué modelamos?

La razón fundamental es que construimos modelos para comprender mejor el sistema que estamos desarrollando.

A través del modelado, conseguimos cuatro objetivos:

1. Ayudan a visualizar cómo es que queremos que sea un sistema.
2. Permiten especificar la estructura o el comportamiento de un sistema.
3. Proporcionan plantillas que nos guían en la construcción de un sistema.
4. Documentan las decisiones que hemos adoptado.

El modelado no es sólo para los grandes sistemas. Incluso el equivalente un software muy pequeño puede beneficiarse del modelado. Sin embargo, es absolutamente cierto que, cuanto más grande y complejo es el sistema, el modelado se hace más importante, por una simple razón:

Construimos modelos de sistemas complejos porque no podemos comprender el sistema en su totalidad.

A través del modelado, se reduce el problema que se está estudiando, centrándose sólo en un aspecto a la vez. Acometer un problema difícil dividiéndolo en una serie de subproblemas más pequeños que se pueden resolver.

El uso del modelado tiene una historia interesante en todas las disciplinas de ingeniería. Esa experiencia sugiere cuatro principios básicos de modelado:

1. La elección de qué modelos crear tiene una profunda influencia sobre cómo se acomete un problema y cómo se da forma a una solución.
2. Todo modelo puede ser expresado a diferentes niveles de precisión.
3. Los mejores modelos están ligados a la realidad.
4. Un único modelo no es suficiente. Cualquier sistema no trivial se aborda mejor a través de un pequeño conjunto de modelos casi independientes.

1.2. Modelado Orientado a Objetos

En el software hay varias formas de enfocar un modelo. Las dos formas más comunes son la perspectiva orientada a objetos y la perspectiva algorítmica. La visión tradicional del desarrollo de software toma una perspectiva algorítmica. En este enfoque, el bloque principal de construcción de todo el software es el procedimiento o función. Esta visión conduce a los desarrolladores a centrarse en cuestiones de control y descomposición de algoritmos grandes en otros más pequeños. No hay nada inherentemente malo en este punto de vista, salvo que tiende a producir sistemas frágiles. Cuando los requisitos cambian y el sistema crece, los sistemas construidos con un enfoque algorítmico se vuelven muy difíciles de mantener.

La visión actual del desarrollo de software toma una perspectiva orientada a objetos. En este enfoque, el principal bloque de construcción de todos los sistemas de software es el objeto o clase. Para explicarlo sencillamente, un objeto es una cosa, generalmente extraída del vocabulario del espacio del problema o del espacio de la solución; una clase es una descripción de un conjunto de objetos similares. Todo objeto tiene identidad (puede nombrarse o distinguirse de otra manera de otros objetos), estado (generalmente hay algunos datos asociados a él), y comportamiento (se le pueden hacer cosas al objeto, y él a su vez puede hacer cosas a otros objetos).

Por ejemplo, considérese una arquitectura sencilla de tres capas para un sistema de contabilidad, que involucre una interfaz de usuario, una capa intermedia y una base de datos. En la interfaz del usuario aparecerán objetos concretos, tales como botones, menús y cuadros de diálogo. En la base de datos aparecerán objetos concretos, tales como tablas que representarán entidades del dominio del problema, incluyendo clientes, productos y otras. En la capa intermedia aparecerán objetos tales como transacciones y reglas de negocio*, así como vistas de más alto nivel de las entidades del problema, tales como clientes, productos y pedidos.

Actualmente, el enfoque orientado a objetos forma parte de la tendencia principal para el desarrollo de software, simplemente porque ha demostrado ser válido en la construcción de sistemas en toda clase de dominios de problemas, abarcando todo el abanico de tamaños y complejidades. Más aún, la mayoría de los lenguajes actuales, sistemas operativos y herramientas son orientados a objetos. El desarrollo orientado a objetos proporciona la base fundamental para ensamblar sistemas a partir de componentes utilizando tecnologías como Java Beans o COM+.

Visualizar, especificar, construir y documentar sistemas orientados a objetos es exactamente el propósito de UML.

* Una regla del negocio es una expresión que define o restringe algún aspecto estructural o dinámico de una organización.

2. UML

UML es un lenguaje para visualizar, especificar, construir y documentar los artefactos de un sistema con gran cantidad de software.

2.1. UML es un lenguaje

Un lenguaje proporciona un vocabulario y las reglas para combinar palabras de ese vocabulario con el objetivo de posibilitar la comunicación. Un lenguaje de modelado es un lenguaje cuyo vocabulario y reglas se centran en la representación conceptual y física de un sistema. Un lenguaje de modelado como UML es por tanto un lenguaje estándar para los planos del software.

2.2. UML es un lenguaje para visualizar

Algunas cosas se modelan mejor textualmente; otras se modelan mejor de forma gráfica. En realidad, en todos los sistemas interesantes hay estructuras que trascienden lo que puede ser representado en un lenguaje de programación. UML es uno de estos lenguajes gráficos.

UML es algo más que un simple montón de símbolos gráficos. Más bien, detrás de cada símbolo en la notación UML hay una semántica bien definida. De esta manera, un desarrollador puede escribir un modelo en UML, y otro desarrollador, o incluso otra herramienta, puede interpretar ese modelo sin ambigüedad.

2.3. UML es un lenguaje para especificar

En este contexto, especificar significa construir modelos precisos, no ambiguos y completos. En particular, UML cubre la especificación de todas las decisiones de análisis, diseño e implementación que deben realizarse al desarrollar y desplegar un sistema con gran cantidad de software.

2.4. UML es un lenguaje para construir

UML no es un lenguaje de programación visual, pero sus modelos pueden conectarse de forma directa a una gran variedad de lenguajes de programación. Esto significa que es posible establecer correspondencias desde un modelo UML a un lenguaje de programación como Java, C++, Visual Basic o C#, o incluso a tablas en una base de datos relacional o al almacenamiento persistente en una base de datos orientada a objetos. Las cosas que se expresan mejor gráficamente también se representan gráficamente en UML, mientras que las cosas que se expresan mejor textualmente se plasman con el lenguaje de programación.

Esta correspondencia permite ingeniería directa: la generación de código a partir de un modelo UML en un lenguaje de programación. Lo contrario también es posible: se puede reconstruir un modelo en UML a partir de una implementación.

2.5. UML es un lenguaje para documentar

Una organización de software que trabaje bien produce toda clase de artefactos además de código ejecutable. Estos artefactos incluyen (aunque no se limitan a):

- Requisitos.
- Arquitectura.
- Diseño.
- Código fuente.
- Planificación de proyectos.
- Pruebas.
- Prototipos.
- Versiones.

UML cubre la documentación de la arquitectura de un sistema y todos sus detalles.

2.6. ¿Dónde puede utilizarse UML?

UML está pensado principalmente para sistemas con una gran cantidad de software. Ha sido utilizado de forma efectiva en dominios tales como:

- Sistemas de información de empresa.
- Bancos y servicios financieros.
- Telecomunicaciones.
- Transporte.

- Defensa / industria aeroespacial.
- Comercio.
- Electrónica médica.
- Ámbito científico.
- Servicios distribuidos basados en la web.

UML no está limitado al modelado de software. De hecho, es lo suficientemente expresivo para modelar sistemas que no son software, como flujos de trabajo (workflows) en el sistema jurídico, estructura y comportamiento de un sistema de vigilancia médica de un enfermo, y el diseño de hardware.

3. Evolución de UML

La metodología llamada UML (Unified Modeling Language - Lenguaje Unificado de Modelado) surge a mediados de los noventa de la unión de las tres mejores metodologías orientadas a objetos: el método de Booch, el método OOSE (Object Oriented Software Engineering - Ingeniería de Software Orientados a Objetos) de Jacobson y el método OMT (Object Modeling Technique - Técnica de Modelado de Objetos) de Rumbaugh. Cada uno de estos era un método completo, aunque todos tenían sus puntos fuertes y débiles, sus creadores decidieron unirse para incorporar a UML lo mejor de cada metodología.

Existen algunas herramientas CASE que emplean UML que son Rational Rose™ [1] y Visio™ [2].

[1] Disponible en WWW: <http://www-306.ibm.com/software/rational/>

[2] Disponible en WWW: <http://www.office.microsoft.com/visio/>

4. Elementos de UML

Como lenguaje, UML tiene una sintaxis y una semántica bien definidas. La parte más visible de la sintaxis de UML es su notación gráfica.

4.1. Elementos

- Elementos estructurales

Los elementos estructurales son los nombres de los modelos de UML. Estos incluyen clases, interfaces, colaboraciones, casos de uso, clases activas, componentes y nodos.

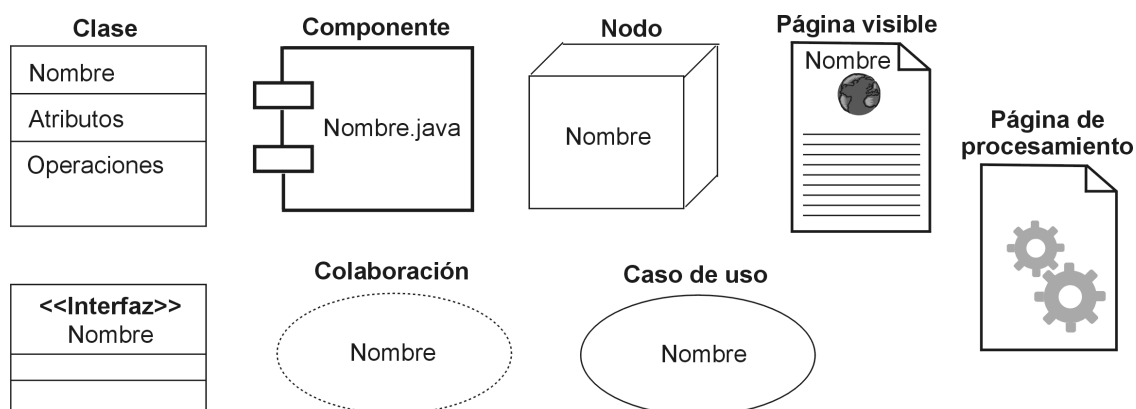


FIGURA 1. Elementos estructurales.

- Elementos de comportamiento

Los elementos de comportamiento son las partes dinámicas de los modelos de UML. Estos incluyen los diagramas de interacción.

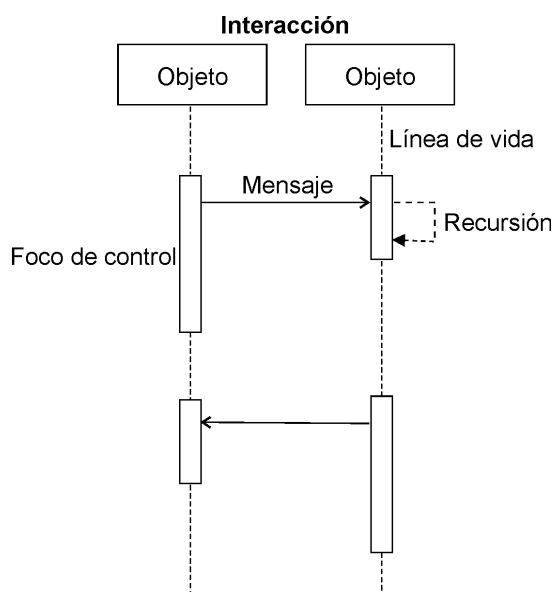


FIGURA 2. Elementos de comportamiento.

- Elementos de agrupación

Los elementos de agrupación son las partes organizativas de los modelos de UML. Estos incluyen los paquetes.

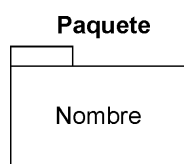


FIGURA 3. Elementos de agrupación.

- Elementos de anotación

Los elementos de anotación son las partes explicativas de los modelos de UML. Estos incluyen las notas.

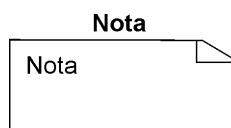


FIGURA 4. Elementos de anotación.

4.2. Relaciones

- Dependencia

Una dependencia es una relación semántica entre dos elementos, en la cual un cambio a un elemento (el elemento independiente) puede afectar a la semántica del otro elemento (elemento dependiente).

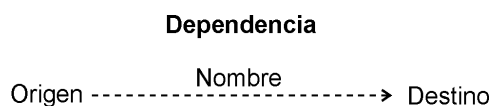


FIGURA 5. Dependencia.

- Asociación

Una asociación es una relación estructural que describe un conjunto de enlaces; un enlace es una conexión entre objetos.

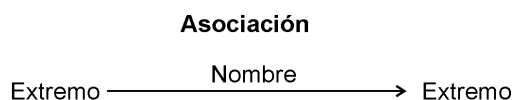


FIGURA 6. Asociación.

- Agregación

La agregación se denota con un diamante abierto. Esta relación denota que la clase agregada (la clase la clase con el diamante blanco que tocándola) es de

alguna manera él “todo”, y la otra clase en la relación es de algún modo una “parte” de ese todo.

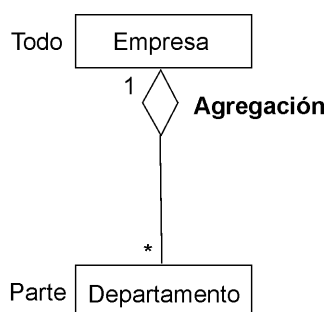


FIGURA 7. Agregación.

- Generalización

La generalización (herencia) es una relación de especialización/generalización en la cual los objetos del elemento especializado (el hijo) pueden sustituir a los objetos del elemento general (padre).

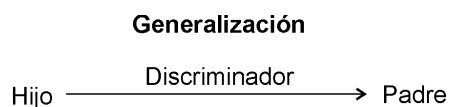


FIGURA 8. Generalización.

NOTA: La notación de UML presentada anteriormente no se encuentra completa, sólo se presenta la notación más utilizada. Se recomienda la bibliografía [3] si se quiere la notación completa de UML.

5. Modelos (Vistas)

Un modelo de UML es descompuesto en vistas que capturan diferentes perspectivas del modelo.

5.1. Vista de casos de uso

Esta vista tiene como propósito el entender la utilidad del sistema, en esta vista se ven actores y casos de uso, además de las interacciones entre ellos. Los diagramas en esta vista son los diagramas de caso de uso.

5.2. Vista lógica

La vista lógica captura los requerimientos funcionales del sistema, dentro de esta vista se encapsulan las clases y sus relaciones estáticas, en esta vista se utilizan además de los diagramas de clases, los diagramas de secuencia, los diagramas de colaboración y los diagramas de transición de estado.

5.3. Vista de componentes

Esta vista contiene información sobre el software ejecutable y componentes de la biblioteca del sistema, los diagramas utilizados en esta vista son los diagramas de componentes o de implantación.

5.4. Vista de despliegue

Esta vista tiene como propósito ver el estado de los nodos de proceso del sistema y los objetos, procesos, hilos y componentes que existen en ellos en tiempo de ejecución, esta vista sólo contiene los diagramas de despliegue.

Preguntas I

Responde las siguientes preguntas de forma individual y con tus propias palabras. La Tarea debe cumplir con la rúbrica indicada por el tutor.

1. ¿Cuál es el propósito de utilizar UML?
2. ¿Qué es un modelo?
3. ¿Por qué modelamos?
4. ¿Cuáles son los objetivos del modelado?
5. ¿Cuáles son los principios del modelado?
6. ¿Por qué UML es considerado un lenguaje?
7. ¿Por qué UML es considerado un lenguaje para visualizar?
8. ¿Por qué UML es considerado un lenguaje para especificar?
9. ¿Por qué UML es considerado un lenguaje para construir?
10. ¿Por qué UML es considerado un lenguaje para documentar?
11. ¿Dónde puede utilizarse UML?
12. ¿Dibuje la simbología que emplea UML para representar una clase, un objeto y un componente?
13. ¿Dibuje la simbología que emplea UML para representar un paquete, una asociación, una agregación y una dependencia?
14. ¿Cuáles son las vistas que componen un modelo en UML y describa cada una de ellas?

Material II

6. Diagramas en UML

Un diagrama es una representación gráfica de un conjunto de elementos, visualizando la mayoría de las veces como un grafo conexo de nodos (elementos) y arcos (relaciones). Los diagramas se dibujan para visualizar un sistema de diferentes perspectivas, de forma que un diagrama es una proyección de un sistema. Para todos los sistemas, excepto los más triviales, un diagrama representa una vista resumida de los elementos que constituyen un sistema. Por esta razón, UML incluye nueve de estos diagramas:

1. Diagrama de clases.
2. Diagrama de objetos.
3. Diagrama de casos de uso.
4. Diagrama de secuencia.
5. Diagrama de colaboración.
6. Diagrama de estados (*statechart*).
7. Diagrama de actividades.
8. Diagrama de componentes.
9. Diagrama de despliegue.

NOTA: Durante el desarrollo del proyecto no es necesario emplear todos los diagramas, depende del proyecto. La lista anterior no es una lista cerrada de diagramas. Las herramientas pueden utilizar UML para proporcionar otros tipos de diagramas, aunque estos nueve son, con mucho, los que con mayor frecuencia aparecerán en la práctica.

Cuando se modelan sistemas reales, sea cual sea el dominio del problema se puede dividir en dos partes: estática y dinámica. Muchas veces se dibujan los

misimos tipos de diagramas, porque representan vistas comunes de modelos comunes. Normalmente, las partes estáticas de un sistema (también llamados diagramas estructurales) se representarán mediante uno de los cuatro diagramas siguientes: diagramas de clases, objetos, componentes y despliegue. Los modelos estáticos representan clases, relaciones de herencia y relaciones de agregación. Por lo regular este tipo de diagramas son los primeros en hacerse. Los diagramas que muestran las partes dinámicas de un sistema son: diagramas de casos de uso, secuencia, colaboración, estados y actividades. A estos diagramas también se les conoce como diagramas de comportamiento.

La relación que existe entre los modelos estáticos y dinámicos es muy importante, ya que un modelo estático no puede ser probado con exactitud sin un modelo dinámico asociado a él. Los modelos dinámicos, por su parte, no representarán adecuadamente el estado estructural y el manejo de las dependencias.

6.1. Diagramas de casos de uso

Un diagrama de casos de uso muestra un conjunto de casos de uso y actores (un tipo especial de clases) y sus relaciones, son usados durante el análisis de un proyecto para identificar y particionar la funcionalidad del sistema. Los diagramas de casos de uso cubren la vista de casos de uso estática de un sistema. Estos diagramas son especialmente importantes en el modelado y organización del comportamiento de un sistema. Esta vista cubre principalmente el comportamiento del sistema (los servicios visibles externamente que proporciona el sistema en el contexto de su entorno).

Para crear un caso de uso, el analista debe primero identificar los diferentes tipos de personas de personas (o dispositivos) que se usan en el sistema o producto. Estos actores actualmente representan papeles ejecutados por personas (o dispositivos) cuando el sistema está en operación. Definido de una manera más

formal un actor es cualquier cosa que se comunique con el sistema o producto y que sea externo a él. Es importante observar que un actor y un usuario no son la misma cosa. Un usuario típico puede desempeñar un cierto número de papeles (roles) cuando usa el sistema, mientras que el actor representa una clase de entidades externas (a menudo, pero no siempre, las personas) que sólo desempeñan un único papel.

Los casos de uso se emplean para capturar el comportamiento deseado del sistema en desarrollo, sin tener que especificar cómo se implementa ese comportamiento. Los casos de uso proporcionan un medio para que los desarrolladores, los usuarios finales del sistema y los expertos del dominio lleguen a una comprensión común del sistema. Además, los casos de uso ayudan a validar la arquitectura y a verificar el sistema mientras evoluciona a lo largo del desarrollo. Conforme se desarrolla el sistema, los casos de uso son realizados por colaboraciones, cuyos elementos cooperan para llevar a cabo cada caso de uso. Los casos de uso bien estructurados denotan sólo comportamientos esenciales del sistema o de un subsistema, y nunca deben ser excesivamente genéricos ni demasiado específicos.

Un caso de uso describe qué hace un sistema (o un subsistema, una clase o una interfaz), pero no especifica cómo lo hace. Cuando se modela, es importante tener clara la separación de objetivos entre las vistas externa e interna. El comportamiento de un caso de uso se puede especificar describiendo un flujo de eventos de forma textual, lo suficientemente claro para que alguien ajeno al sistema lo entienda fácilmente. Cuando se escribe este flujo de eventos se debe incluir cómo y cuándo empieza y acaba el caso de uso, cuándo interactúa con los actores y qué objetos se intercambian, el flujo básico y los flujos alternativos o excepcionales del comportamiento.

Cuando se modela la vista de casos de uso estática de un sistema, normalmente se emplearán los diagramas de casos de uso de una de las dos formas siguientes:

- Modelar el contexto de un sistema. Implica dibujar una línea alrededor de todo el sistema y asegurar qué actores quedan fuera del sistema e interactúan con él. Aquí, se emplearán los diagramas de casos de uso para especificar los actores y el significado de sus roles.
- Modelar los requisitos de un sistema. Implica especificar qué debería hacer el sistema (desde el punto de vista externo), independientemente de cómo se haga.

Documentación

Uno de los problemas con el mantenimiento de la documentación es que cuando un solo requerimiento cambia, este puede afectar muchos lugares en el texto de la especificación funcional. Por lo tanto, algunas veces el costo de la redundancia de la información en una especificación funcional puede ser muy alto, causando problemas de mantenimiento significativos. La meta de los casos de uso y sus relaciones es para administrar las descripciones textuales con una especificación funcional. Mediante la estructuración los casos de uso y propiamente sus relaciones, podemos crear especificaciones funcionales que nunca necesitan ser cambiadas en mas de un lugar. Para proyectos muy grandes, esto puede ser una ganancia significativa.

A continuación se muestra un ejemplo de un diagrama de casos de uso para modelar el comportamiento de un teléfono móvil. El diagrama contempla dos actores: *Usuario* y *Red telefónica*. El actor *Usuario* puede *Realizar llamada telefónica*, *Recibir llamada telefónica* y *Usar agenda*. Mientras que el actor *Red telefónica* puede únicamente *Realizar llamada telefónica* y *Recibir llamada telefónica*.

Podemos ver en el diagrama, los casos o funciones *Realizar llamada telefónica* y *Recibir llamada telefónica*, extienden de *Realizar llamada de conferencia* y *Recibir llamada adicional* respectivamente, que no es más que un proceso superior del cual dependen.

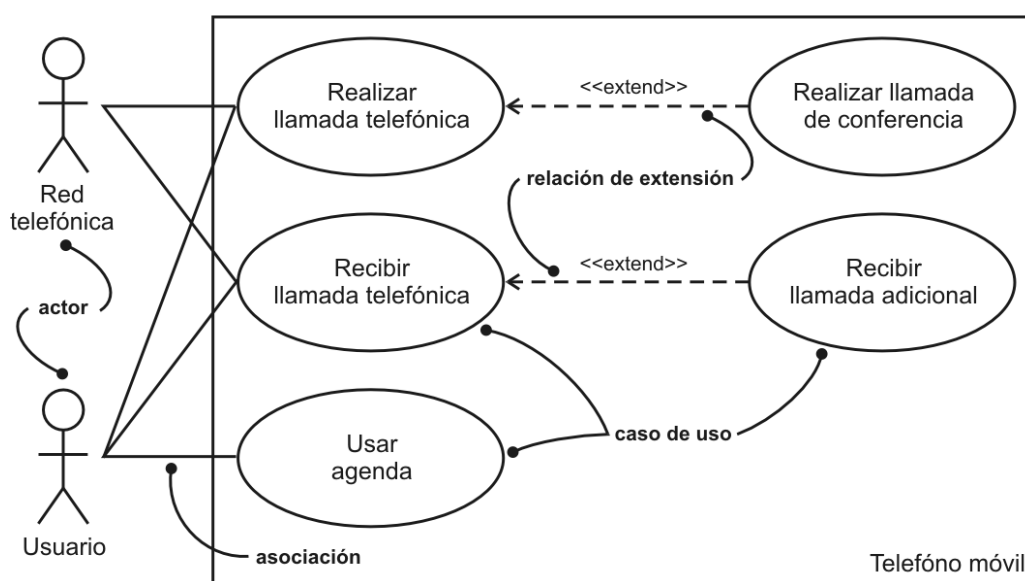


FIGURA 9. Diagrama de casos de uso de un teléfono móvil.

6.2. Diagramas de secuencia

Tanto los diagramas de secuencia como los diagramas de colaboración son un tipo de diagramas de interacción. Un diagrama de interacción muestra una interacción, que consta de un conjunto de objetos y sus relaciones, incluyendo los

mensajes que pueden ser enviados entre ellos. Los diagramas de interacción cubren la vista dinámica de un sistema. Los diagramas de secuencia y de colaboración describen el flujo de los mensajes entre objetos, se enfocan en el orden en que los mensajes son enviados. Estos son muy útiles para la descripción del flujo procedural a través de varios objetos. También son muy útiles para encontrar condiciones de competencia en los sistemas concurrentes. Los diagramas de secuencia tienen dos características que los distinguen de los diagramas de colaboración.

En primer lugar, está la línea de vida. La línea de vida de un objeto es la línea discontinua vertical que representa la existencia de un objeto a lo largo de un período de tiempo. La mayoría de los objetos que aparecen en un diagrama de interacción existirán mientras dure la interacción, así que los objetos se colocan en la parte superior del diagrama, con sus líneas de vida dibujadas desde arriba hasta abajo. Pueden crearse objetos durante la interacción. Sus líneas de vida comienzan con la recepción del mensaje estereotipado como *create*. Los objetos pueden destruirse durante la interacción. Sus líneas de vida acaban con la recepción del mensaje estereotipado como *destroy* (además se muestra la señal visual de una gran X que marca el final de sus vidas). Véase el siguiente diagrama.

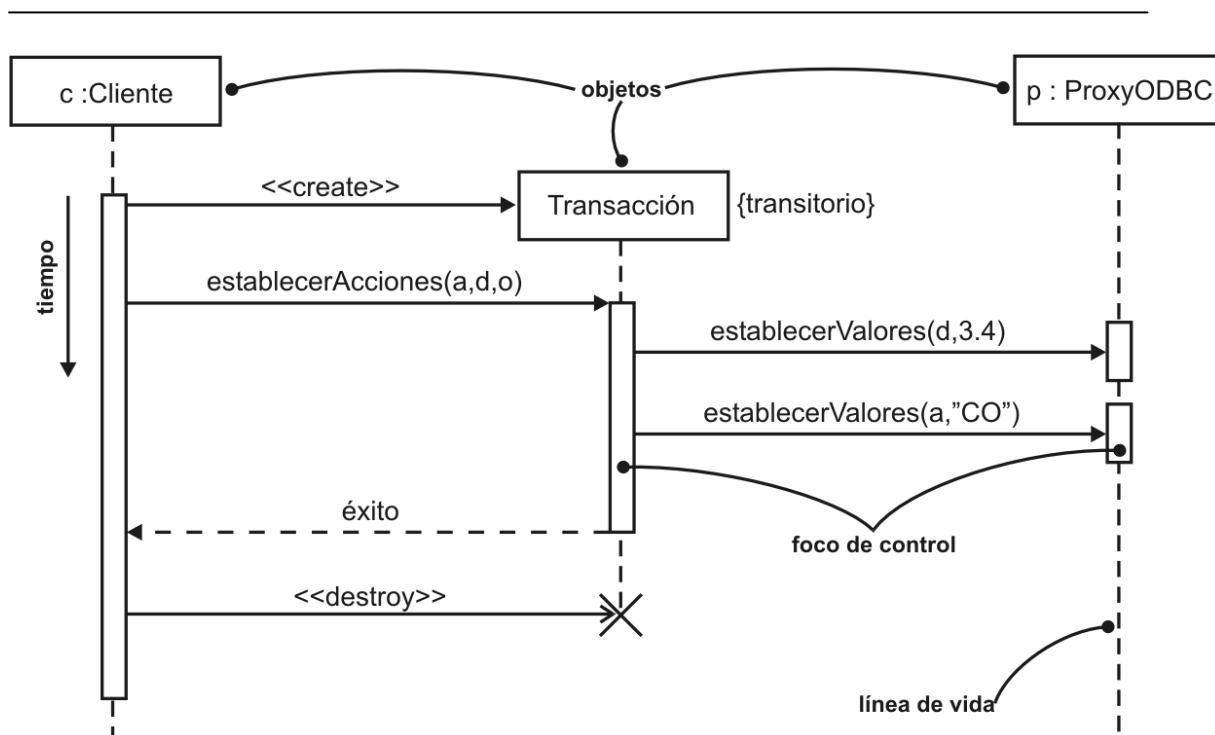


FIGURA 10. Diagrama de secuencia.

En segundo lugar, está el foco de control. El foco de control es un rectángulo delgado y estrecho que representa el período de tiempo durante el cual un objeto ejecuta una acción, bien sea directamente o a través de un procedimiento subordinado. La parte superior del rectángulo se alinea con el comienzo de la acción; la inferior se alinea con su terminación (y puede marcarse con un mensaje de retorno). También puede mostrarse el anidamiento de un foco de control (que puede estar causado por recursión, una llamada a una operación propia, o una llamada desde otro objeto) colocando otro foco de control ligeramente a la derecha de su foco padre (esto se puede hacer a cualquier nivel de profundidad). Si se quiere ser especialmente preciso acerca de dónde se encuentra el foco de control, también se puede sombrear la región del rectángulo durante la cual el método del objeto está ejecutándose (y el control no ha pasado a otro objeto).

Por ejemplo, en el siguiente diagrama de secuencia se especifica el flujo de control para iniciar una simple llamada telefónica entre dos partes. A este nivel de abstracción existen cuatro objetos involucrados: dos *Interlocutores* (*a* y *b*), una *Central* de teléfonos y la materialización de la *Conversación* (*c*) entre ambas partes.

La secuencia comienza cuando un *Interlocutor* (*a*) emite una señal (*descolgarAuricular*) al objeto *Central*. A su vez, la *Central* llama a *darTonoDeLlamada* sobre éste *Interlocutor*, y el *Interlocutor* itera sobre el mensaje *marcarDigito*. Nótese que este mensaje tiene una marca temporal (*marcando*) que se utiliza en una restricción de tiempo (su *tiempoDeEjecucion* debe ser menor de 30 segundos). El objeto *Central* se llama así mismo con el mensaje de *enrutarLlamada*. A continuación crea un objeto *Conversación* (*c*), al cual delega el resto del trabajo. El objeto *Conversación* (*c*) llama al *Interlocutor* (*b*), el cual envía asincrónicamente el mensaje *descolgarAuricular*. Entonces, el objeto *Conversación* indica a la *Central* que debe *conectar* la llamada, y luego a los dos objetos *Interlocutor* que pueden *conectar*, tras lo cual pueden intercambiar información, como se indica en la nota adjunta.

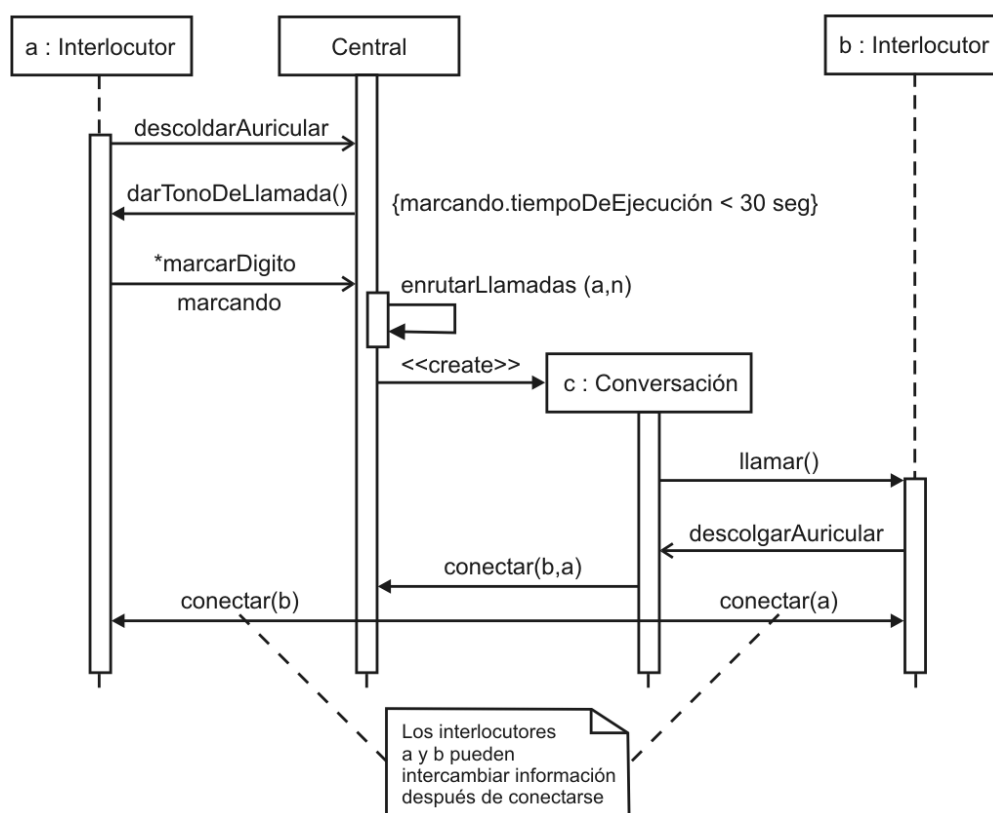


FIGURA 11. Diagrama de secuencia que muestra el flujo de control de una llamada telefónica.

6.3. Diagramas de colaboración

Un diagrama de colaboración es un diagrama de interacción que resalta la organización estructural de los objetos que envían y reciben mensajes. Los diagramas de secuencia y los diagramas de colaboración son isomorfos, es decir, que se puede tomar uno y transformarlo en el otro. Un diagrama de colaboración es una forma de representar interacción entre objetos, alterna al diagrama de secuencia. A diferencia de los diagramas de secuencia, pueden mostrar el contexto de la operación (cuáles objetos son atributos, cuáles temporales, etc.) y ciclos de ejecución.

La secuencia se indica numerando los mensajes (1, 2, 3, et.), sin embargo el hacer esto hace más difícil la visualización de la secuencia, pero la distribución espacial del diagrama permite mostrar otras cosas mejor, como la vinculación entre objetos o la sobreposición de paquetes u otra información.

Para representar el anidamiento, se utiliza la numeración decimal de Dewey (1 es el primer mensaje; 1.1 es el primer mensaje dentro del mensaje 1; 1.2 es el segundo mensaje dentro del mensaje 1; etc.). El anidamiento se puede representar a cualquier nivel de profundidad. Nótese también que, a través del mismo enlace, se pueden mostrar varios mensajes (posiblemente enviados desde distintas direcciones), y cada uno tendrá un número de secuencia único.

La mayoría de las veces se modelarán flujos de control simples y secuenciales. Sin embargo, también se pueden modelar flujos más complejos, que impliquen iteración y bifurcación. Una iteración representa una secuencia repetida de mensajes. Para modelar una iteración, el número de secuencia de un mensaje se precede de una expresión iteración, como en $* [i : = 1 . . n]$ (o sólo $*$ si se quiere indicar iteración pero no se desea especificar los detalles). Una iteración indica que el mensaje (y cualquier mensaje es anidado) se repetirá de acuerdo con la expresión dada. Análogamente, una condición presenta un mensaje cuya ejecución depende de la evaluación de una expresión booleana. Para modelar una condición, el número de secuencia de un mensaje se precede de una cláusula de condición, como $[x > 0]$. Los distintos caminos alternativos de bifurcación tendrán el mismo número de secuencia, pero cada camino debe ser distinguible de forma única por una condición que no se solape con las otras.

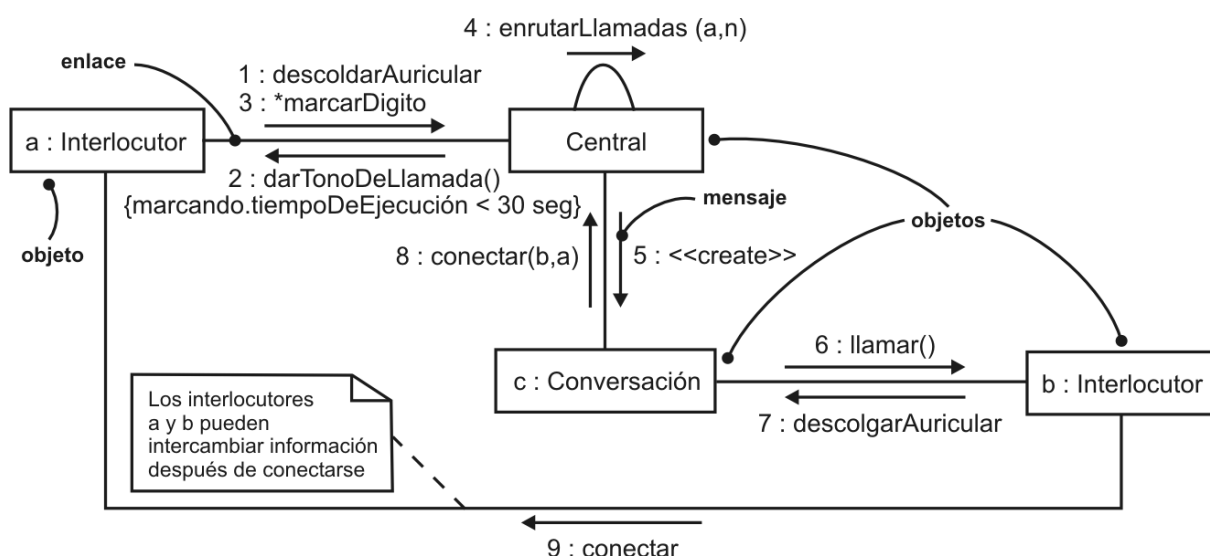


FIGURA 12. Diagrama de colaboración.

6.4. Diagramas de clase

Un diagrama de clases muestra un conjunto de clases, interfaces y colaboraciones, así como sus relaciones. Estos diagramas son los diagramas más comunes en el modelado de sistemas orientados a objetos. Los diagramas de clases se utilizan para modelar la vista de diseño estática de un sistema. Los diagramas de clases que incluyen clases activas cubren la vista de procesos estática de un sistema. El elemento fundamental de un diagrama de clases es un icono que representa la clase. Una clase contiene atributos y operaciones.

Esta vista soporta principalmente los requisitos funcionales de un sistema, los servicios que el sistema debe proporcionar a sus usuarios finales. Cuando se modela la vista de diseño estática de un sistema, normalmente se utilizarán los diagramas de clases de una de estas tres formas:

- Modelar el vocabulario de un sistema. Implica tomar decisiones sobre qué abstracciones son parte del sistema en consideración y cuáles caen fuera de sus límites. Los diagramas de clases se utilizan para especificar abstracciones y sus responsabilidades.

- Modelar colaboraciones simples. Una colaboración es una sociedad de clases, interfaces y otros elementos que colaboran para proporcionar un comportamiento cooperativo mayor que la suma de todos los elementos.
- Modelar un esquema lógico de base de datos. Se puede pensar en un esquema como en un plano para el diseño conceptual de una base de datos.

En el siguiente diagrama de clases, se muestra un conjunto de clases extraídas de un sistema de información de una universidad. Esta figura es una extensión de un diagrama anterior, y ahora se muestran las clases a un nivel suficientemente detallado para construir una base de datos física.

Comenzando por la parte inferior y a la izquierda del diagrama, se encuentran las clases *Estudiante*, *Curso* y *Profesor*. Hay una asociación entre *Estudiante* y *Curso*, que especifica que los estudiantes asisten a los cursos y cada curso puede tener cualquier número de estudiantes.

Las seis clases se han marcado como persistentes, indicando que sus instancias se han concebido para almacenarse en una base de datos u otra forma de almacenamiento persistente. Este diagrama también muestra los atributos de las seis clases. Todos los atributos son de tipos primitivos. Cuando se modela un esquema, generalmente una relación con cualquier tipo no primitivo se modela mediante agregaciones explícitas en vez de con atributos.

Dos de estas clases (*Universidad* y *Departamento*) muestran varias operaciones para manipular sus partes. Estas operaciones se incluyen porque son importantes para mantener la integridad de los datos (añadir o eliminar un *Departamento*, por ejemplo, tendrá algunos efectos en la cadena). Hay muchas operaciones que se podrían considerar para estas dos clases y para el resto, como consultar los prerrequisitos de un curso para asignarle un estudiante. Estas son más bien reglas

de negocio en vez de operaciones para integridad de la base de datos, y por ello se deberán colocar a un nivel mayor de abstracción que éste esquema.

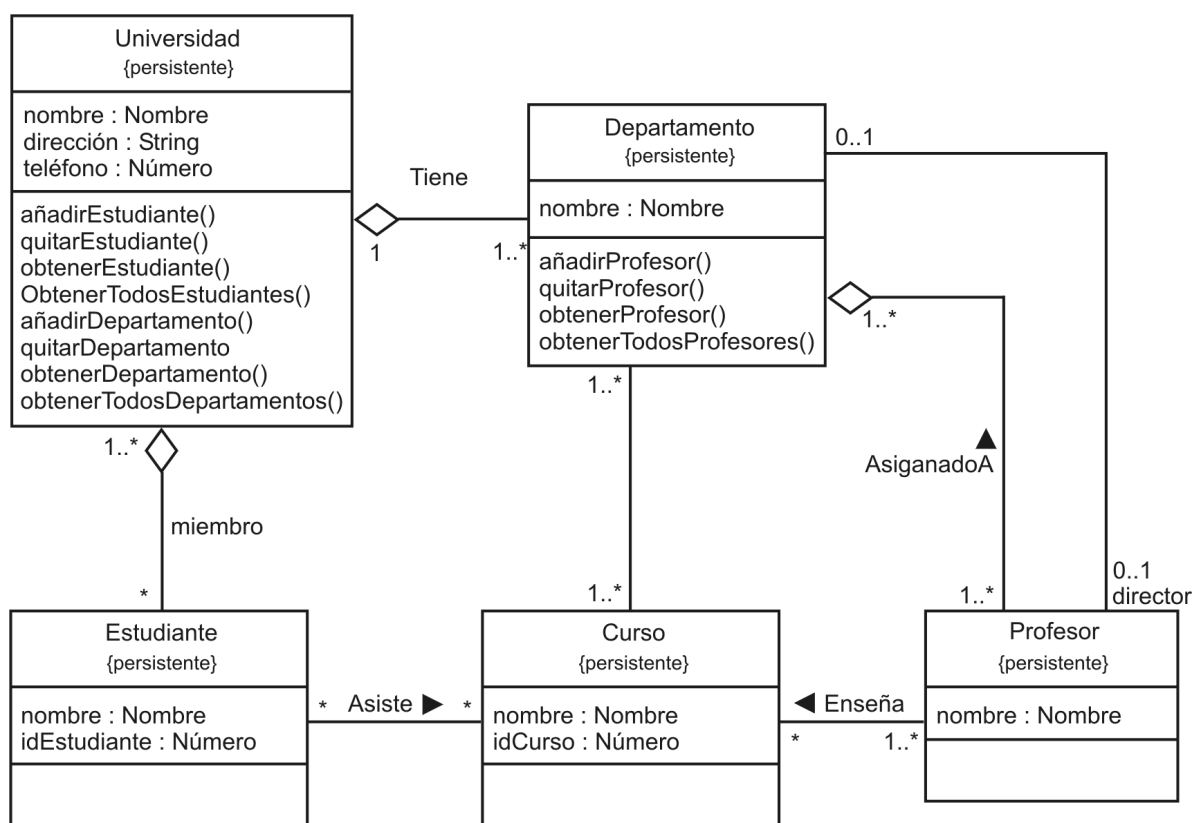


FIGURA 13. Diagrama de clase.

6.5. Diagrama de objetos

Un diagrama de objetos muestra un conjunto de objetos y sus relaciones. Los diagramas de objetos representan un momento concreto de las relaciones de instancias de los elementos encontrados en los diagramas de clases. Estos diagramas cubren la vista de diseño estática o la vista de procesos estática de un sistema como lo hacen los diagramas de clases, pero desde la perspectiva de casos reales o prototípicos. Esta vista sustenta principalmente los requisitos funcionales de un sistema (o sea, los servicios que debe proporcionar el sistema a

sus usuarios finales). Los diagramas de objetos permiten modelar estructuras de datos estáticas.

Al modelar la vista de diseño estática o la vista de procesos estática de un sistema, normalmente los diagramas de objetos se utilizan de una forma para modelar estructuras de objetos.

Para modelar una estructura de objetos:

- Hay que identificar el mecanismo que se desea modelar. Un mecanismo representa alguna función o comportamiento de la parte del sistema que se está modelando, que resulta de la interacción de una sociedad de clases, interfaces y otros elementos.
- Para cada mecanismo, hay que identificar las clases, interfaces y otros elementos que participan en esta colaboración; identificar también las relaciones entre estos elementos.
- Hay que considerar un escenario en el que intervenga este mecanismo. También hay que congelar este escenario en un momento concreto, y representar cada objeto que participe en el mecanismo.
- Hay que mostrar el estado y valores de los atributos de cada uno de esos objetos, si son necesarios para comprender el escenario.
- Análogamente, hay que mostrar los enlaces entre esos objetos, que representarán instancias de asociaciones entre ellos.

Por ejemplo, en el siguiente diagrama de objetos, se representa un conjunto de objetos extraídos de la implementación de un robot autónomo. Éste diagrama se

centra en algunos de los objetos implementados en el mecanismo utilizado por el robot para calcular un modelo del mundo en el que se mueve. Hay muchos más objetos implicados en un sistema de ejecución, pero éste diagrama se centra sólo en aquellas abstracciones implicadas directamente en la creación de esta vista del mundo.

Como indica el diagrama, un objeto representa al propio robot (*r*, una instancia de *Robot*), y *r* se encuentra actualmente en estado de *movimiento*. Este objeto tiene un enlace con *m*, una instancia de *Mundo*, que representa una abstracción del modelo del mundo del robot. Este objeto tiene un enlace con un multiobjeto que consiste en instancias de *Elemento*, que representa entidades que el robot ha identificado, pero aún no ha asignado en su vista del mundo. Estos elementos se marcan como parte del estado global del robot.

En ese instante, *m* está enlazado a dos instancias de *Area*. Una de ellas (*a2*) se muestra con sus propios enlaces a tres objetos *Pared* y un objeto *Puerta*. Cada una de estas paredes está etiquetada con su anchura actual cada, y cada una se muestra enlazada a sus paredes vecinas. Como sugiere este diagrama de objetos, el robot ha reconocido el área que lo contiene, que tiene paredes en tres lados y una puerta en el cuarto.

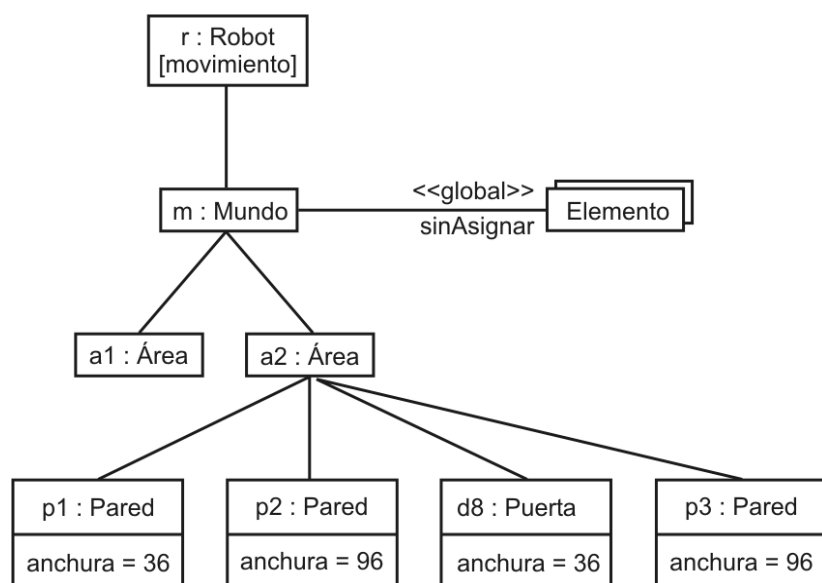
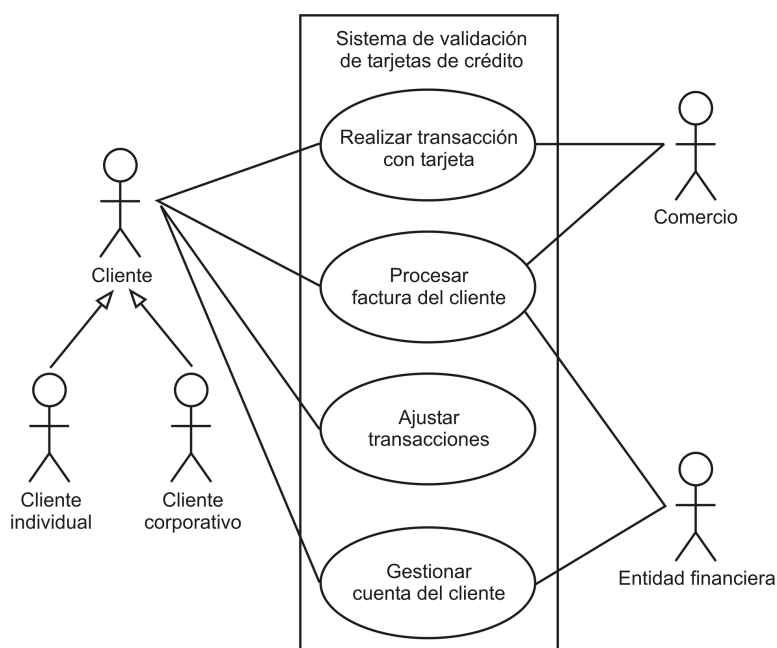


FIGURA 14. Diagrama de objetos.

Preguntas II

Responde las siguientes preguntas de forma individual y con tus propias palabras. La Tarea debe cumplir con la rúbrica indicada por el tutor.

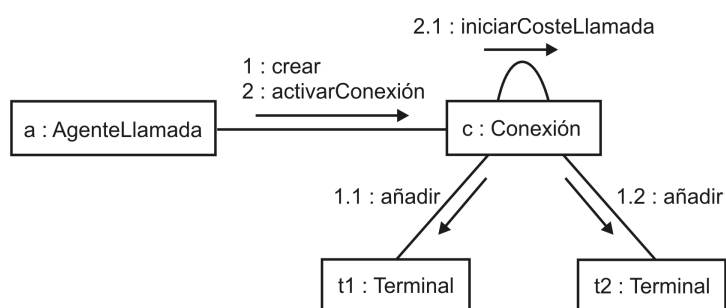
15. ¿Qué es un diagrama en UML?
16. ¿Qué es y para qué se emplea un diagrama de casos de uso?
17. Describe el siguiente diagrama de casos de uso.



18. ¿Qué es y para qué se emplea un diagrama de secuencia?

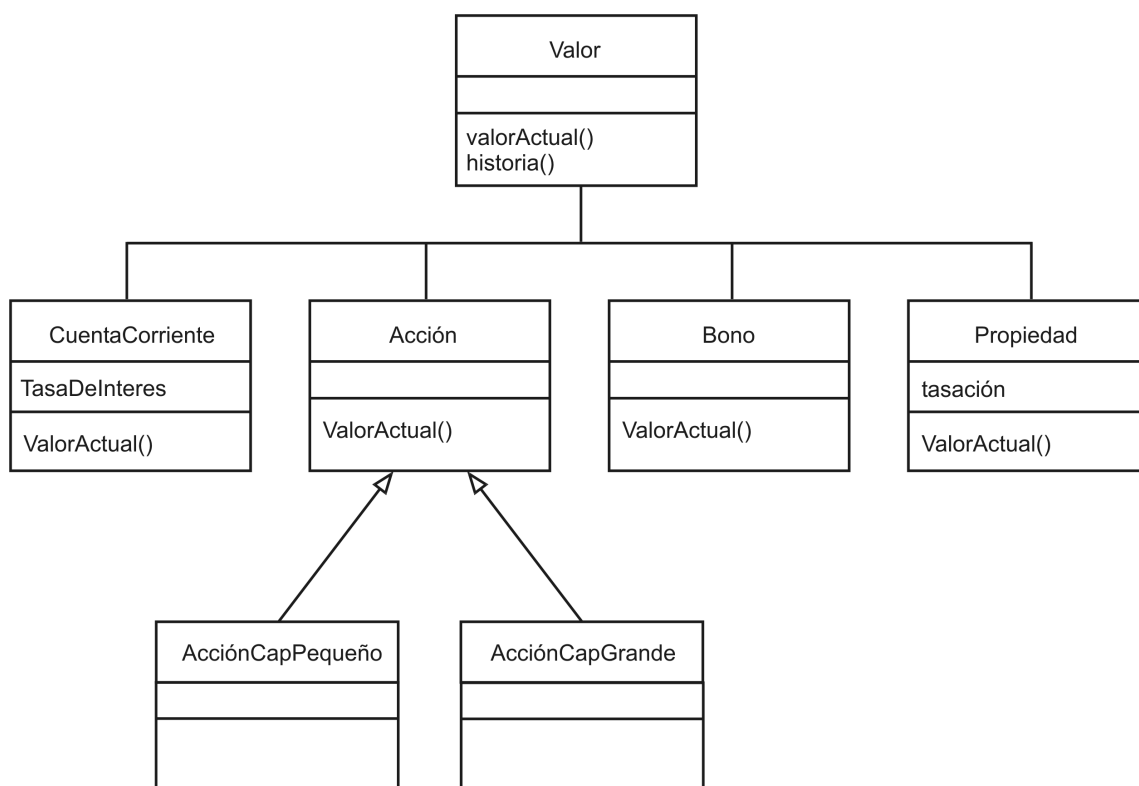
19. ¿Qué es y para qué se emplea un diagrama de colaboración?

20. Describe el siguiente diagrama de colaboración.



21. ¿Qué es y para qué se emplea un diagrama de clases?

22. Describe el siguiente diagrama de clases



23. ¿Qué es y para qué se emplea un diagrama de objetos?

Material III

6.6. Diagrama de estados

Un diagrama de estados muestra una máquina de estados, que consta de estados, transiciones, eventos y actividades. Los diagramas de estados cubren la vista dinámica de un sistema. Son especialmente importantes en el modelado del comportamiento de una interfaz, una clase o una colaboración y resaltan el comportamiento dirigido por eventos de un objeto, lo cual es especialmente útil en el modelado de sistemas reactivos.

La mejor forma de especificar el comportamiento de un sistema de estas características es modelar sus estados estables (por ejemplo: *Inactivo*, *Montado*, *Activo*, *Comprobando*, etc.), los eventos que producen un cambio de estado y las acciones que ocurren en cada cambio de estado.

Un diagrama de estados muestra una máquina de estados, destacando el flujo de control entre estados. Una *máquina de estados* es un comportamiento que especifica las secuencias de estados por las que pasa un objeto a lo largo de su vida en respuesta a eventos, junto con sus respuestas a esos eventos. Un *estado* es una condición o situación en la vida de un objeto durante la cual satisface alguna condición, realiza alguna actividad o espera algún evento. Un *evento* es la especificación de un acontecimiento significativo que ocupa un lugar en el tiempo y en el espacio. En el contexto de las máquinas de estados, un evento es la aparición de un estímulo que puede activar una transición de estado. Una *transición* es una relación entre dos estados que indica que un objeto que este en el primer estado realizará ciertas acciones y entrará en el segundo estado cuando ocurra un evento especificado y se satisfagan unas condiciones específicas. Una *actividad* es una ejecución no atómica en curso, dentro de una máquina de estados. Una *acción* es una computación atómica ejecutable que produce un cambio en el estado del modelo o la devolución de un valor. Gráficamente, un diagrama de estados es una colección de nodos y arcos.

A continuación se muestra un diagrama de estados perteneciente a un fax:

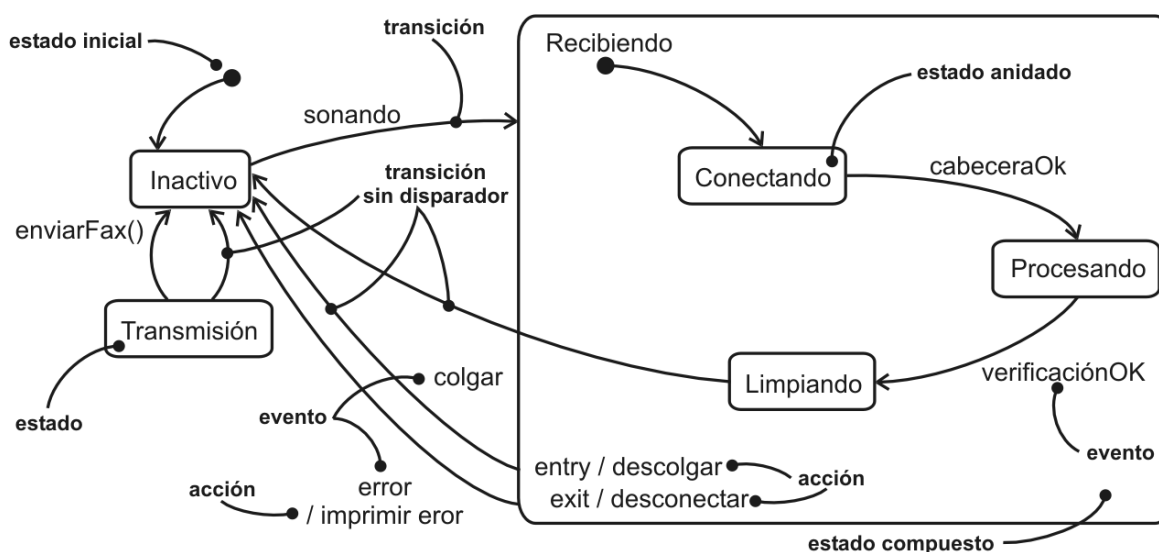


FIGURA 15. Diagrama de estados.

Partiendo del *estado inicial* el fax está en estado de espera hasta que se envía o se recibe información. Si se recibe un fax, se activa un sonido y empieza el proceso de recepción. Primero se realiza la conexión tanto del fax transmisor como del receptor. Enseguida se lee el encabezado del fax, seguido por el cuerpo del mensaje y se imprimen ambos. Si la recepción fue exitosa, se envía un mensaje de recibido al fax transmisor y se corta la conexión.

Si se quiere enviar un fax, se descuelga el auricular, se coloca el papel con el mensaje en el rodillo, se marca el número del fax receptor, se envía la información y se cuelga el auricular. Si el envío fue exitoso se recibe un mensaje de confirmación y se corta la conexión.

6.7. Diagrama de actividades

Un diagrama de actividades es un tipo especial de diagrama de estados que muestra el flujo de actividades dentro de un sistema. Los diagramas de actividades cubren la vista dinámica de un sistema. Son especialmente

importantes al modelar el funcionamiento de un sistema y resaltan el flujo de control entre objetos. Véase el siguiente diagrama.

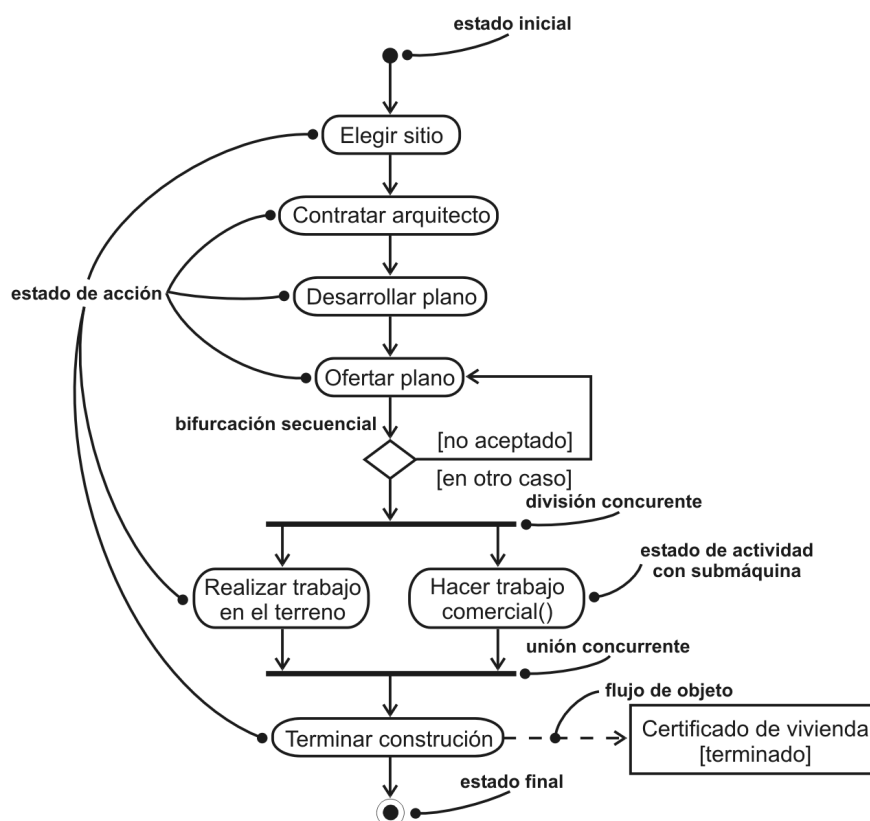


FIGURA 16. Diagrama de actividades.

Los diagramas de actividades no son sólo importantes para modelar los aspectos dinámicos de un sistema, sino también para construir sistemas ejecutables a través de ingeniería directa e inversa.

Un diagrama de flujo muestra el flujo de actividades. Una actividad es una ejecución atómica en curso, dentro de una máquina de estados. Las actividades producen finalmente alguna acción, que está compuesta de computadoras atómicas ejecutables que producen un cambio en el estado del sistema o la devolución de un valor. Las acciones incluyen llamadas a otras operaciones, envío

de señales, creación o destrucción de objeto o simples cálculos, como la evaluación de una expresión. Gráficamente, un diagrama de actividades es una colección de nodos y arcos.

Por ejemplo, en el contexto de la clase *Linea*, se tiene en el siguiente diagrama de actividades, donde se especifica el algoritmo de la operación *Intersección*, cuya signatura incluye un parámetro (*l*, un parámetro *Intersección* de la clase *Linea*) y un valor retorno (de la clase *Punto*). La clase *Linea* tiene dos atributos interesantes: *pendiente* (que contiene la pendiente de la línea) y *delta* (que contiene el desplazamiento de la línea respecto del origen).

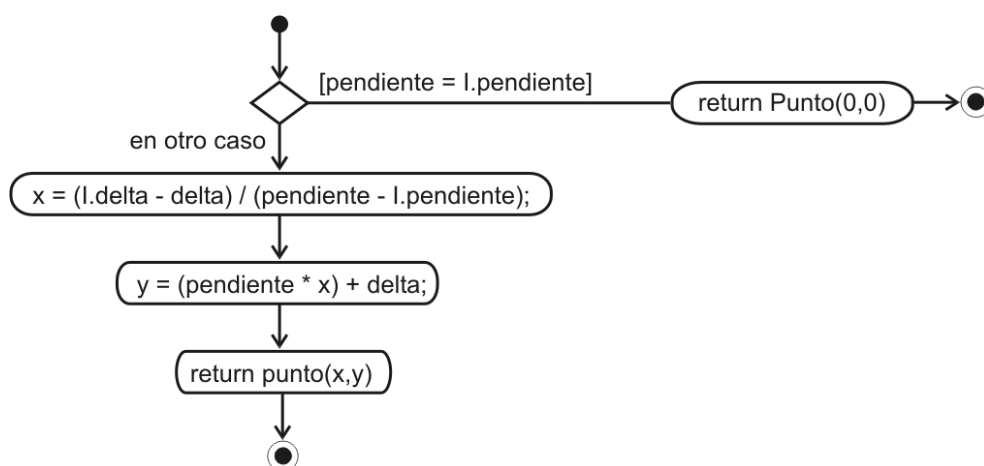


FIGURA 17. Diagrama de actividades.

El algoritmo de la operación es sencillo, como se muestra en el diagrama de actividades. En primer lugar, hay una guarda que comprueba si la *pendiente* de la línea actual es la misma *pendiente* del parámetro *l*. Si es así, las líneas no se cortan y se devuelve el *Punto* en (0,0). En otro caso, la operación calcula el valor de *x* para el punto de intersección y a continuación, el valor de *y*; *x* e *y* son los objetos locales a la operación. Por último, se devuelve un *Punto* en (*x*,*y*).

6.8. Diagrama de componentes

Un diagrama de componentes muestra la organización y las dependencias entre un conjunto de componentes. Los diagramas de componentes cubren la vista de implementación estática de un sistema. Se relacionan con los diagramas de clases en que un componente se corresponde, por lo común, con una o más clases, interfaces o colaboraciones. Los diagramas de componentes se utilizan de cuatro maneras:

- Modelar código fuente. Con la mayoría de los lenguajes de programación orientados a objetos actuales, el código se produce utilizando entornos integrados de desarrollo, que almacenan el código fuente en archivos. Los diagramas de componentes se pueden utilizar para modelar la gestión de configuraciones de estos archivos, los cuales representan los componentes obtenidos como productos de trabajo. Véase el diagrama de la figura 18.
- Modelar versiones ejecutables. Una versión es un conjunto de artefactos relativamente consistente y completo que se entrega a un usuario interno o externo. En el contexto de los componentes, una versión se centra en las partes necesarias para entregar un sistema en ejecución. Cuando se modela una versión mediante diagramas de componentes, se están visualizando, especificando y documentando las decisiones acerca de las partes físicas que constituyen el software (es decir, sus componentes de despliegue). Véase el diagrama de la figura 19.
- Modelar bases de datos físicas. Una base de datos física puede ser vista como la realización concreta de un esquema, y que pertenece al mundo de los bits. Los esquemas ofrecen un API para la información persistente; el modelo de una base de datos física representa el almacenamiento de esa información en las tablas de una base de datos relacional o las páginas de una base de datos

orientada a objetos. Los diagramas de componentes se utilizan para representar estos y otros tipos de base de datos físicas. Véase el diagrama de la figura 20.

- Modelar sistemas adaptables. Algunos sistemas son bastante estáticos; sus componentes entran en escena, participan en la ejecución y desaparecen. Otros sistemas son más dinámicos, e implican agentes móviles o componentes que migran con el propósito de equilibrar la carga o la recuperación de fallos. Los diagramas de componentes se utilizan, junto a algunos de los diagramas de UML, para modelar el comportamiento, con el fin de representar a estos tipos de sistemas.

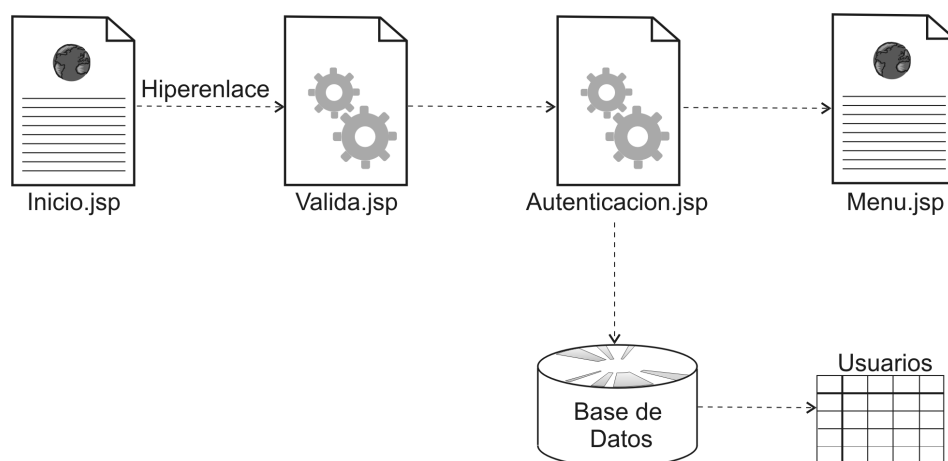


FIGURA 18. Diagrama de componentes de un sistema de autenticación.

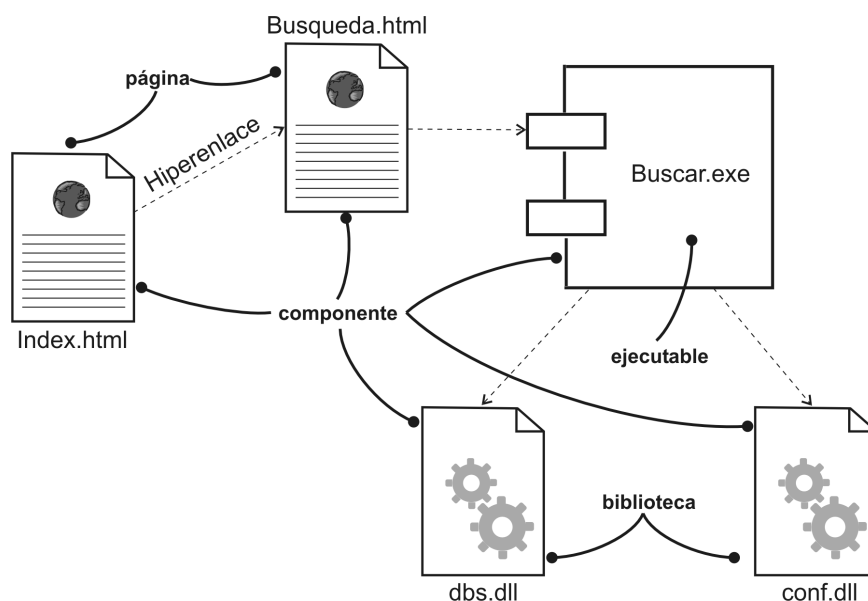


FIGURA 19. Diagrama de componentes de un archivo ejecutable en Internet.

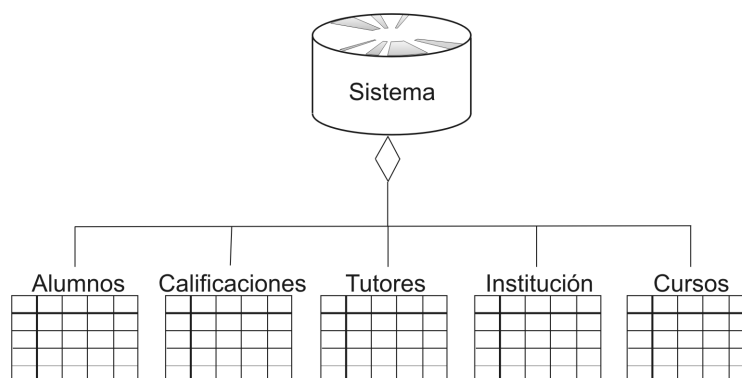


FIGURA 20. Diagrama de componentes de una BD que contiene la información de una Institución educativa.

6.9. Diagrama de despliegue

Un diagrama de despliegue muestra la configuración de nodos de procesamiento en tiempo de ejecución y los componentes que residen en ellos. Los diagramas de despliegue cubren la vista de despliegue estática de una arquitectura. Se relacionan con los diagramas de componentes en que un nodo incluye, por lo

común, uno o más componentes. Los diagramas de despliegue se utilizan de tres maneras:

- Modelar sistemas empotrados. Un sistema empotrado es una colección de hardware con gran cantidad de software que interactúa con el mundo físico. Los sistemas empotrados involucran software que controla dispositivos como motores, actuadores y pantallas y que, a su vez, están controlados por estímulos externos tales como entradas de sensores, movimientos y cambios de temperatura. Los diagramas de despliegue se pueden utilizar para modelar los dispositivos y los procesadores que comprenden un sistema empotrado. Véase el diagrama de la figura 21.
- Modelar sistemas cliente servidor. Un sistema cliente/servidor es una arquitectura muy extendida que se basa en hacer una clara separación de intereses entre la interfaz de usuario del sistema (que residen en el servidor). Los sistemas cliente/servidor son un extremo del espectro de los sistemas distribuidos y requieren tomar decisiones sobre la conectividad de red de los clientes a los servidores y sobre la distribución física de los componentes del software del sistema a través de los nodos. La topología de tales sistemas se puede modelar mediante diagramas de despliegue. Véase el diagrama de la figura 22.
- Modelar sistemas completamente distribuidos. En el otro extremo del espectro de los sistemas distribuidos se encuentran aquellos que son ampliamente, si no totalmente, distribuidos y que, normalmente, incluyen varios niveles de servidores. Tales sistemas contienen a menudo varias versiones de los componentes de software, algunos de los cuales pueden incluso migrar de nodo en nodo. El diseño de tales sistemas requiere tomar decisiones que permitan un cambio continuo de la topología del sistema. Los diagramas de

despliegue se pueden utilizar para visualizar la topología actual del sistema y la distribución de componentes, para razonar sobre el impacto de los cambios en esa topología.

Por ejemplo en el diagrama de la figura 21, se muestra el hardware de un simple robot autónomo. Se puede ver un nodo (Placa base Pentium) estereotipado como un procesador. Rodeando a este nodo hay cinco dispositivos, cada uno estereotipado como un dispositivo y representado con un icono que ofrece una señal visual clara de su equivalente en el mundo real.

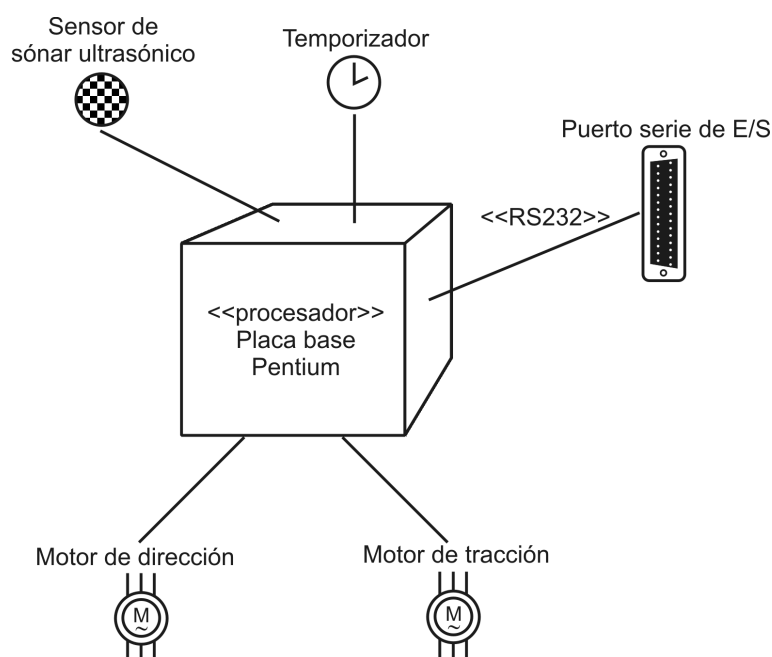


FIGURA 21. Diagrama de despliegue de un sistema empujado.

En el siguiente diagrama de la figura 22, se muestra la topología de un sistema de recursos humanos, que sigue una arquitectura cl3sica cliente/servidor. Este diagrama describe expl3citamente la divisi3n cliente/servidor mediante los paquetes denominados *clientes* y *servidores*.

El paquete cliente contiene dos nodos (*consola* y *terminal*), ambos estereotipados y distinguibles visualmente. El paquete servidor contiene dos tipos de nodos (*servidor de cache* y *servidor*), y ambos han sido adornados con algunos de los componentes que residen en ellos. También puede notarse que *servidor de cache* y *servidor* han sido marcados con multiplicidades explícitas, que especifican cuantas instancias de cada uno se esperan en una configuración de despliegue particular. Por ejemplo, este diagrama indica que podría haber dos o más *servidores de cache* en cualquier instancia desplegada del sistema.

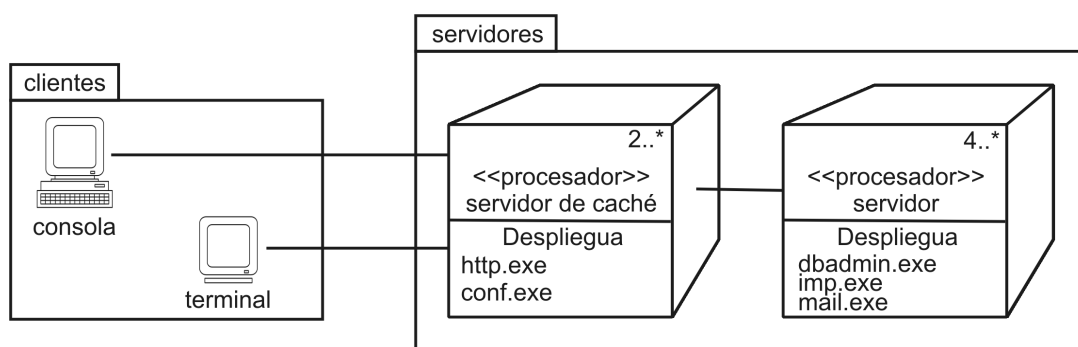


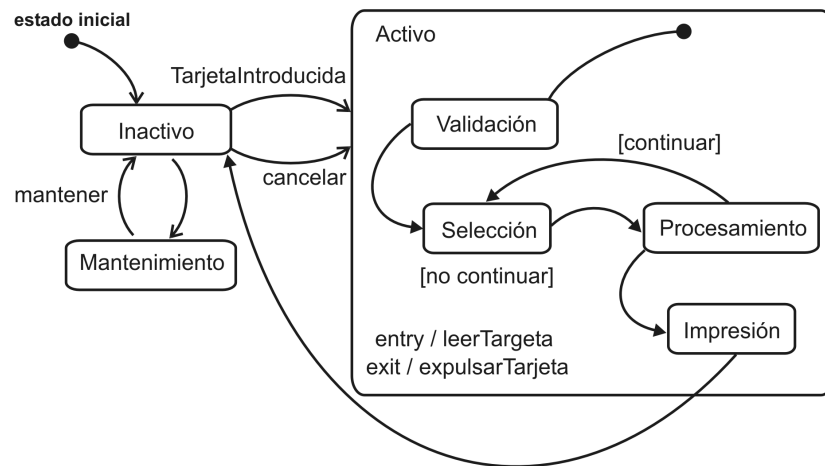
FIGURA 22. Diagrama de despliegue de un sistema cliente/servidor.

Preguntas III

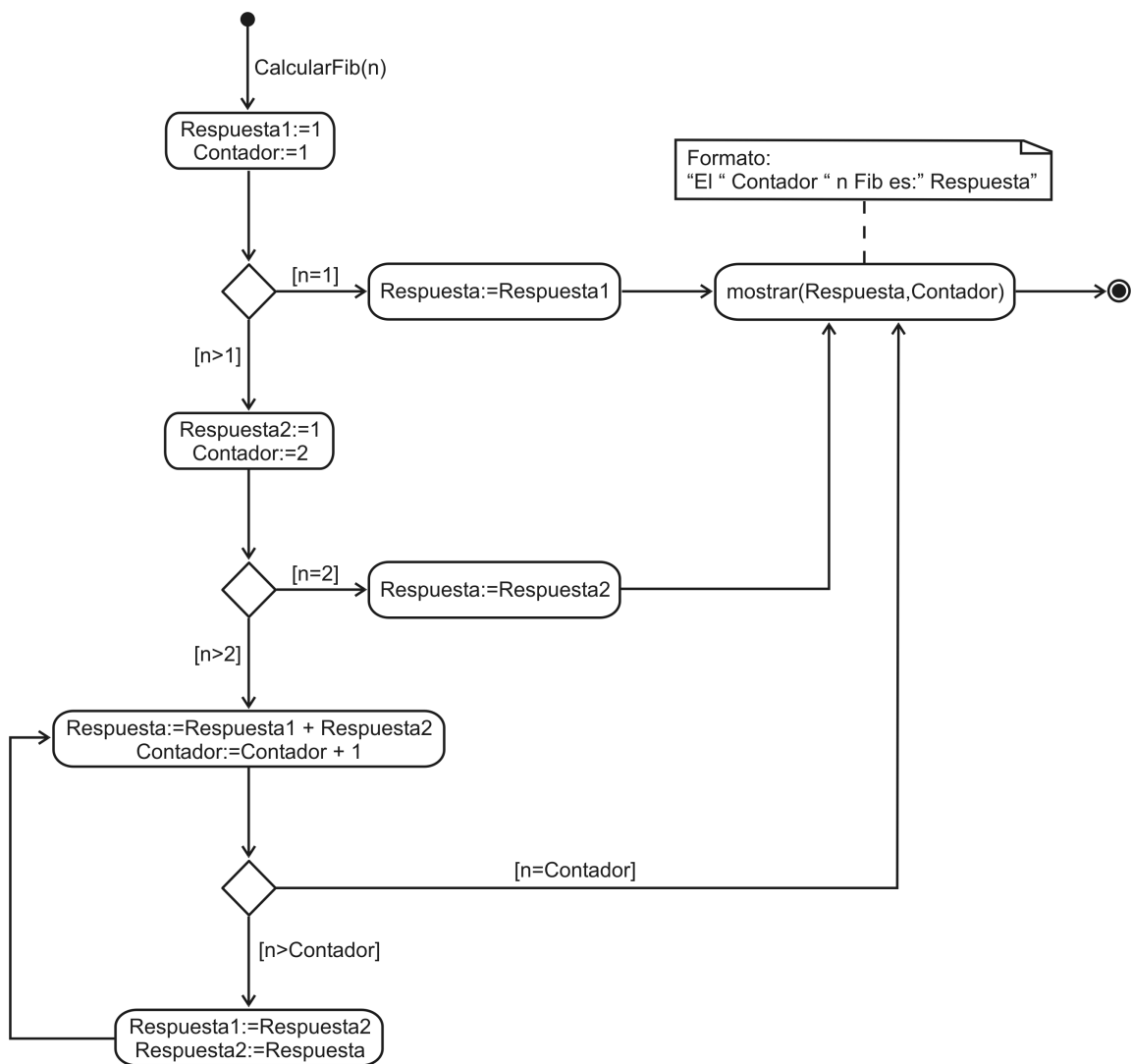
Responde las siguientes preguntas de forma individual y con tus propias palabras. La Tarea debe cumplir con la rúbrica indicada por el tutor.

24. ¿Qué es y para qué se emplea un diagrama de estados?

25. Describe el siguiente diagrama de estados.

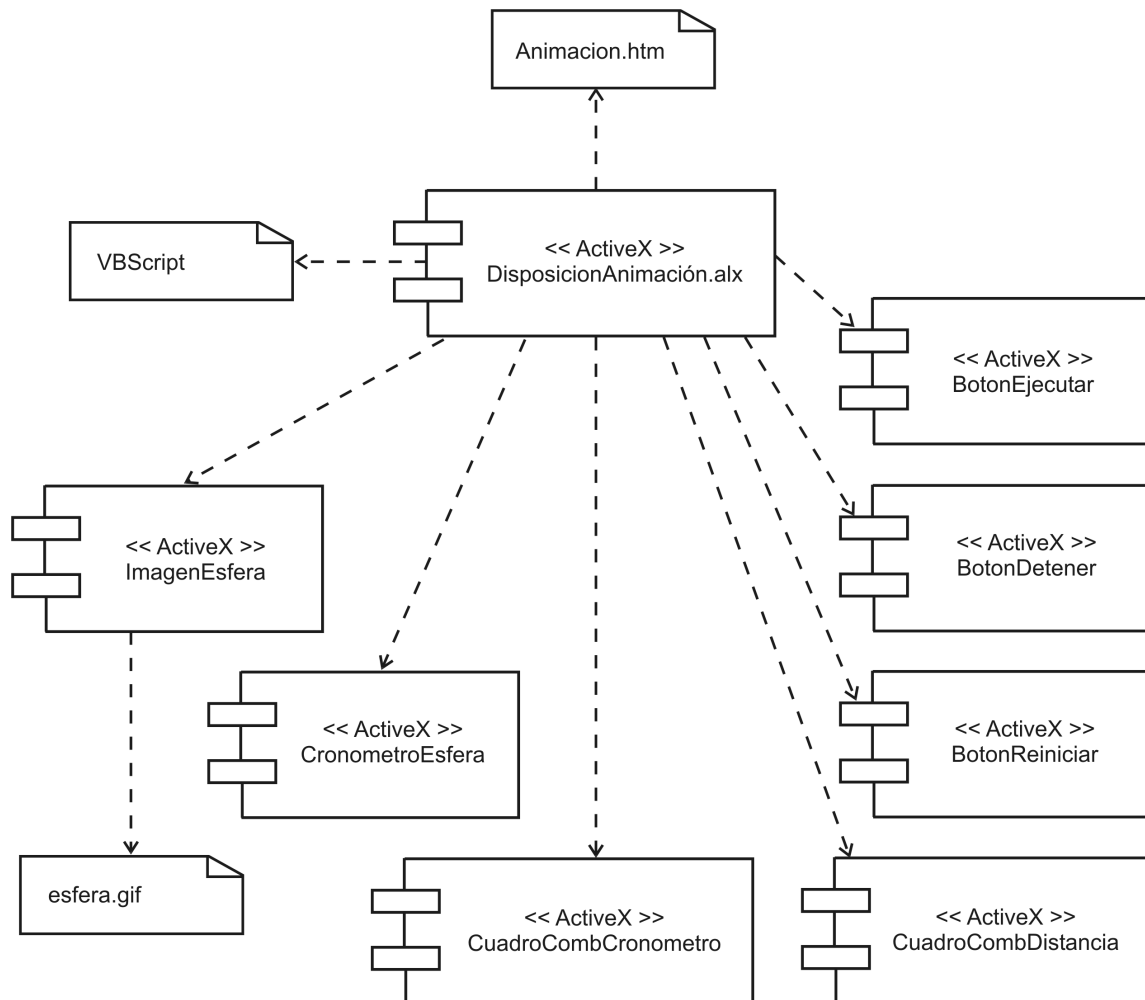


26. ¿Qué es y para qué se emplea un diagrama de actividades?
27. Analiza el siguiente diagrama de actividades que representa las operaciones de cálculo para el enésimo número de Fibonacci. En base al diagrama, realiza el programa con el lenguaje que hayas elegido para trabajar (C# o Java).



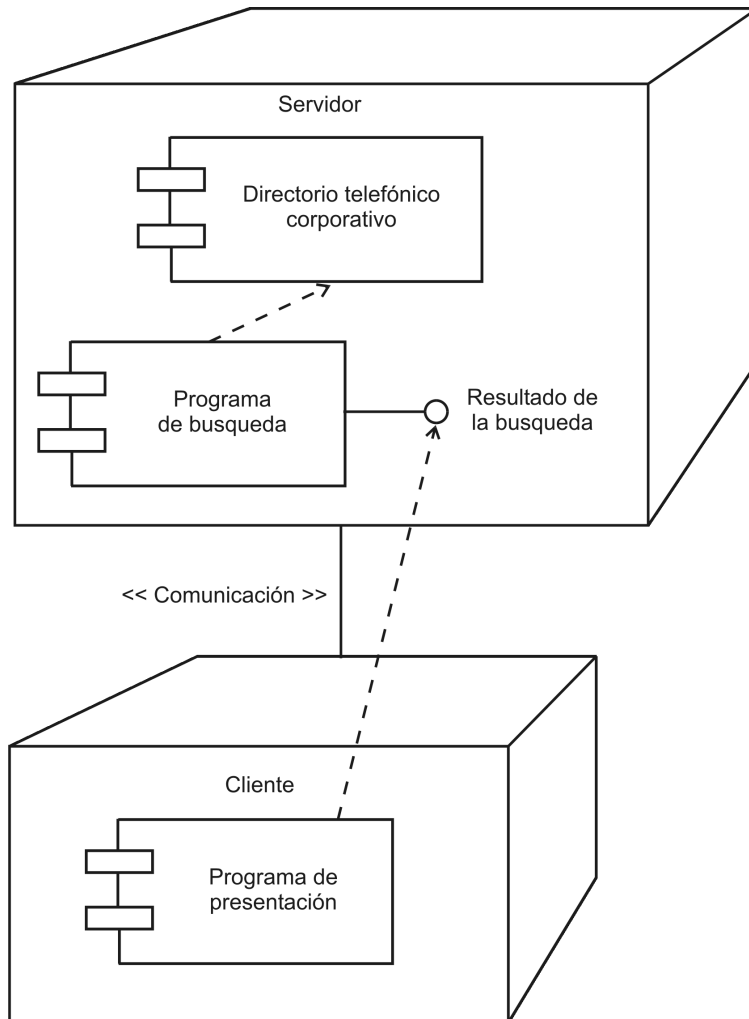
28. ¿Qué es y para qué se emplea un diagrama de componentes?

29. Describe el siguiente diagrama de componentes para una página web con componentes ActiveX.



30. ¿Qué es y para qué se emplea un diagrama de despliegue o distribución?

31. Describe el siguiente diagrama de distribución.



5. Modelo de Diseño

Objetivos Específicos:

- Describirá los fundamentos de UML
- Identificará el uso de UML, sus vistas y los diagramas asociados

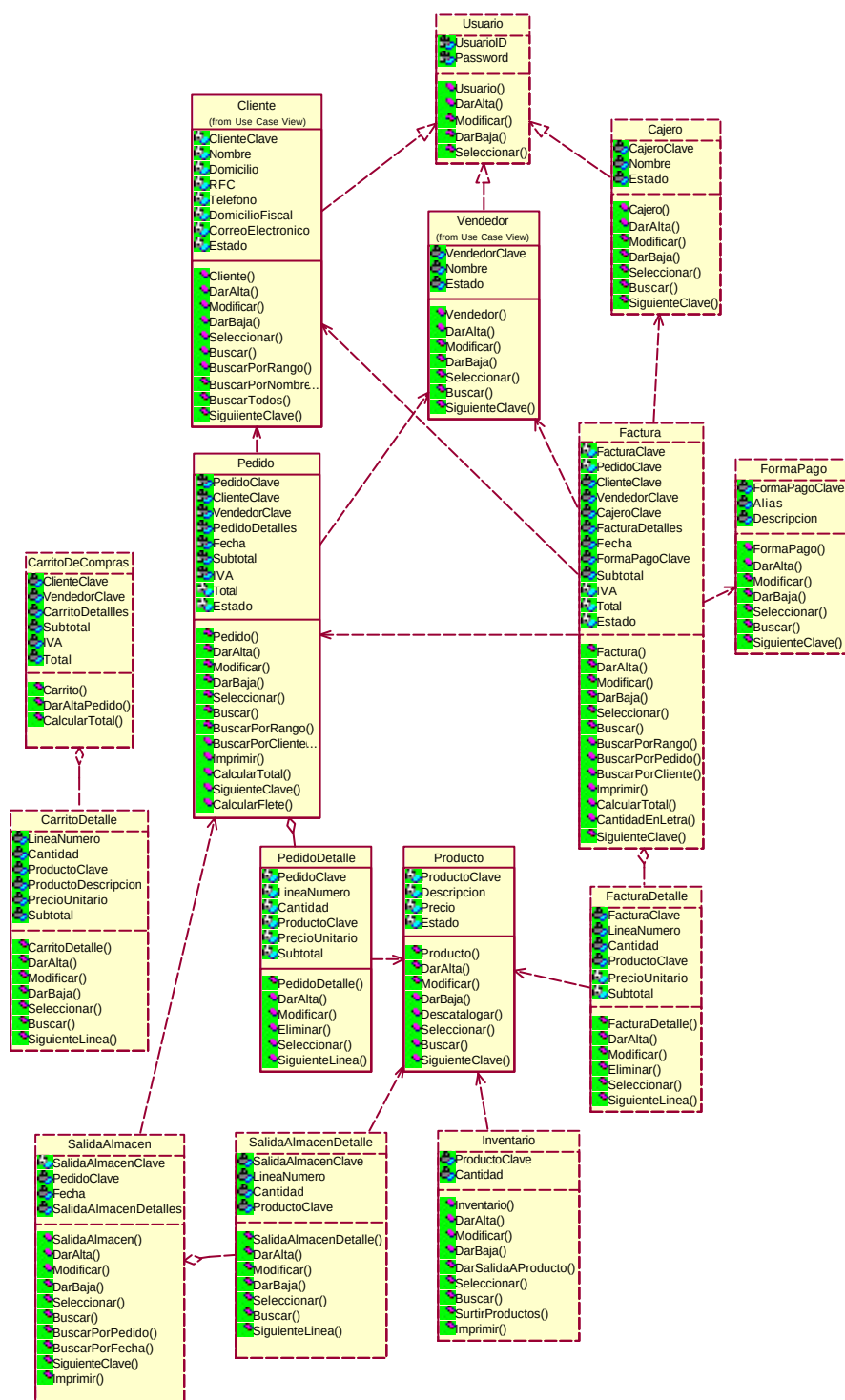
5.1. Diagrama de Clases

Ejercicio

1. Interprete el diagrama dado por el escenario
2. Si el tutor se lo solicita, realice la programación asociada al diagrama.

Escenario

Programación Orientada a Objetos

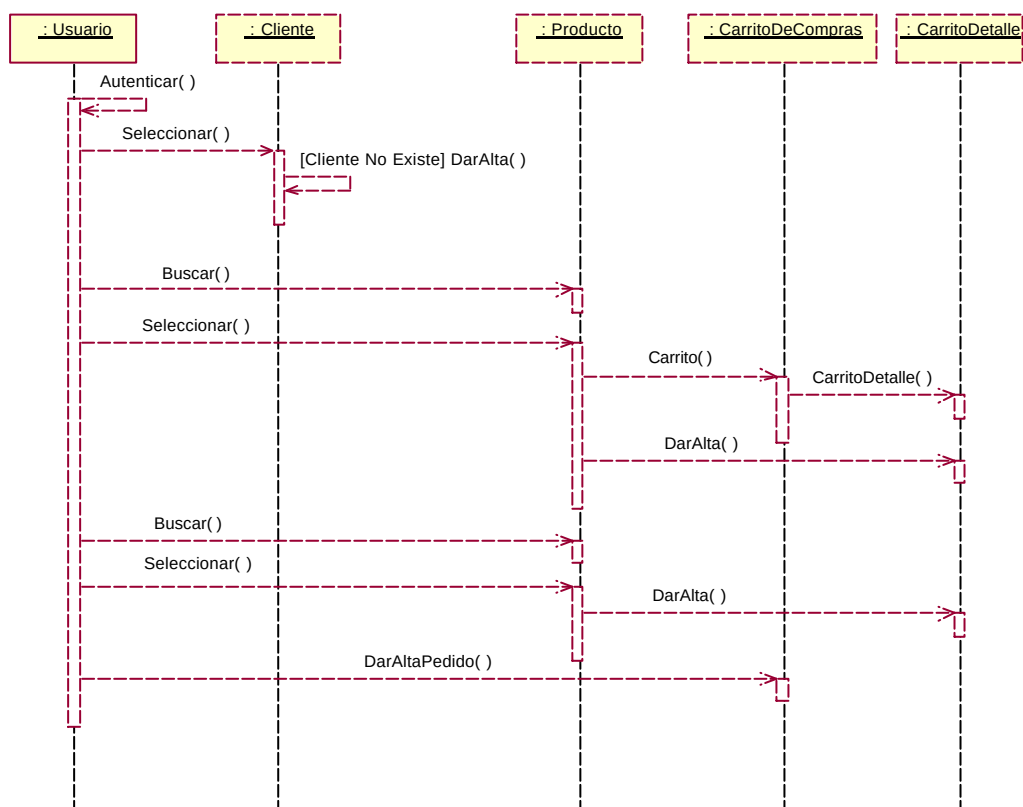


5.2. Diagrama de Secuencia

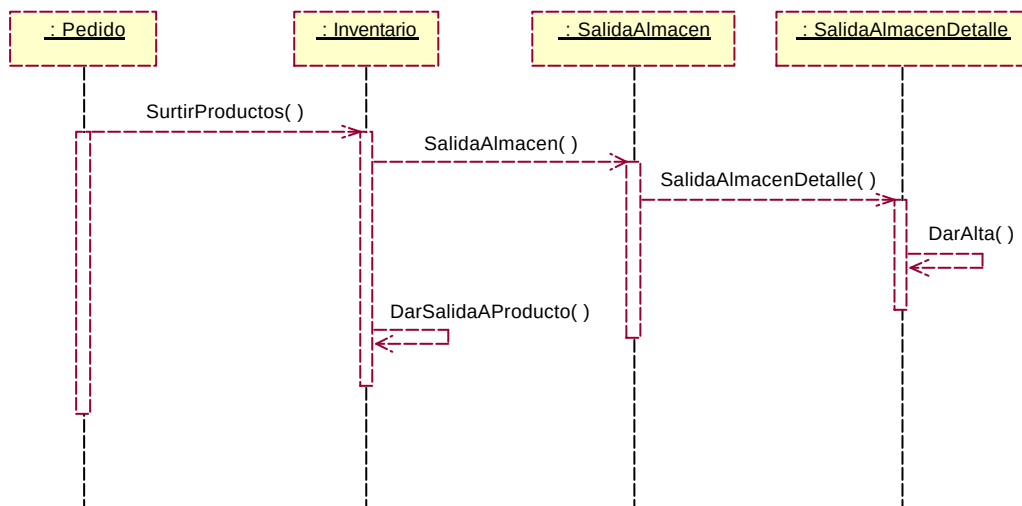
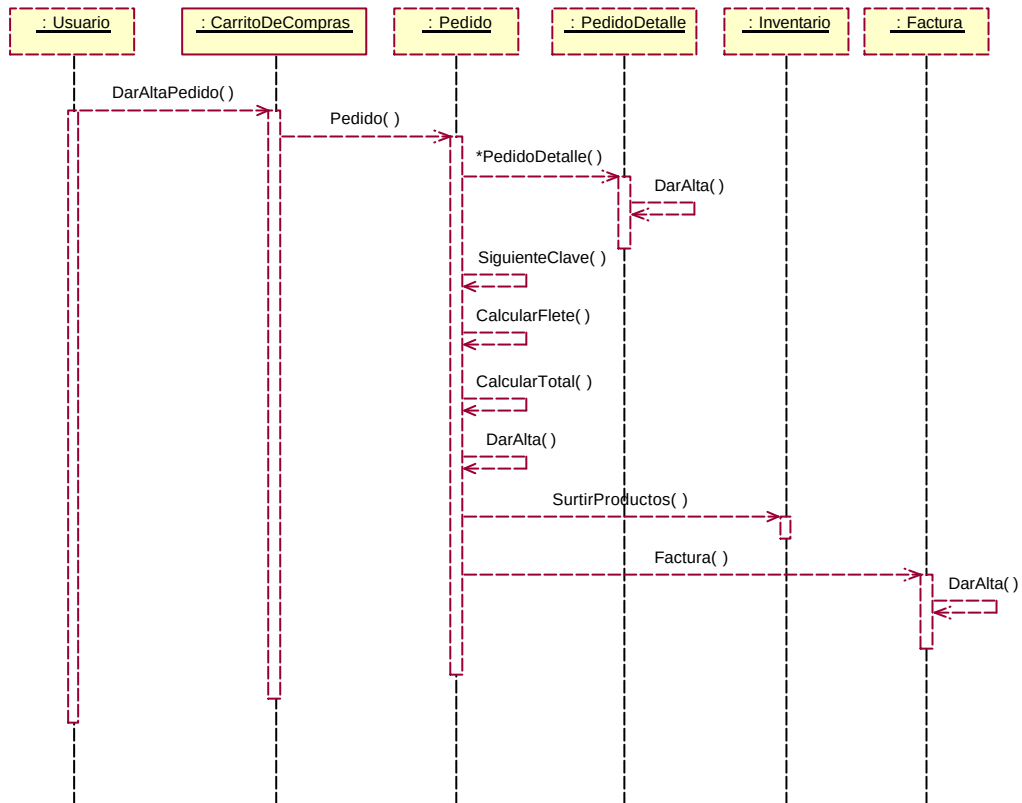
Ejercicio

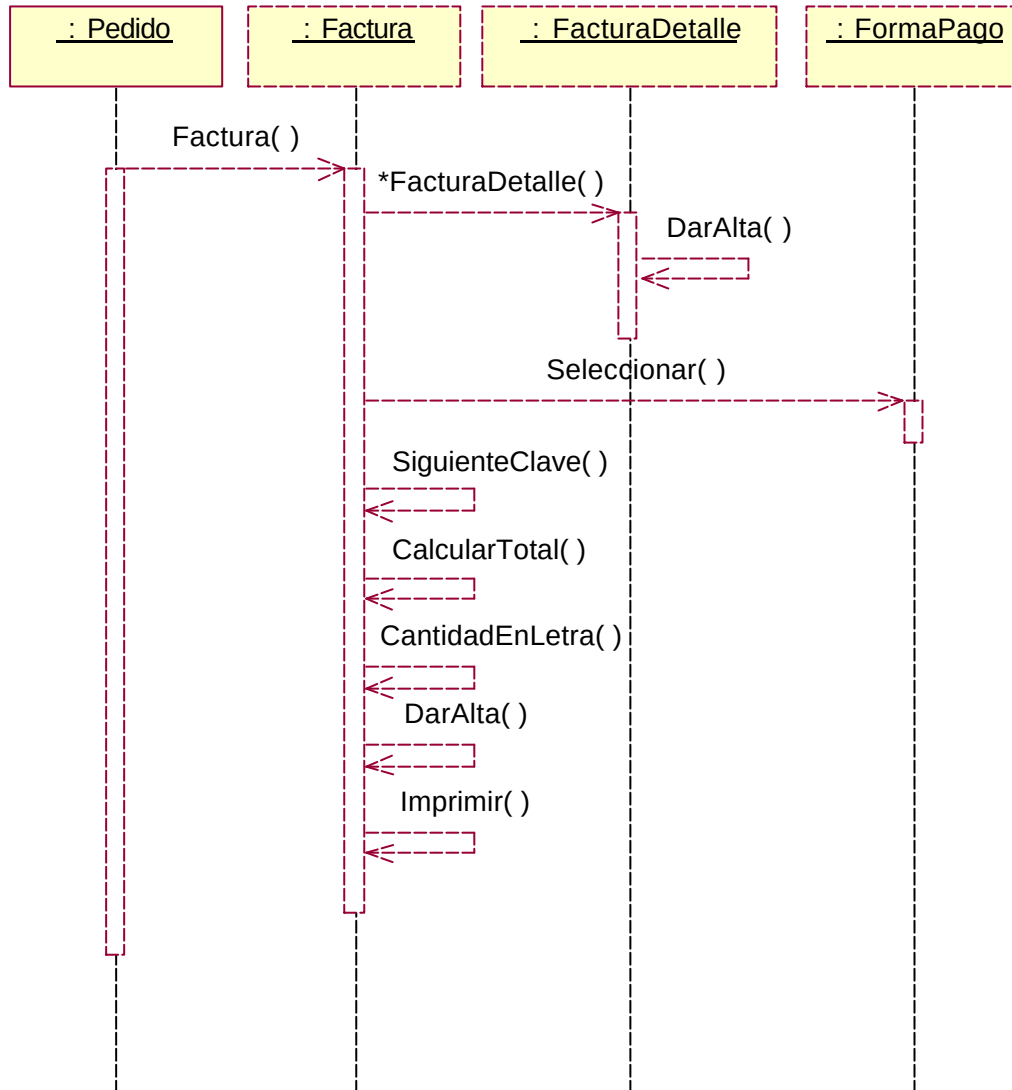
1. Interprete el diagrama dado por el escenario
2. Si el tutor se lo solicita, realice la programación asociada al diagrama.

Escenario



Programación Orientada a Objetos





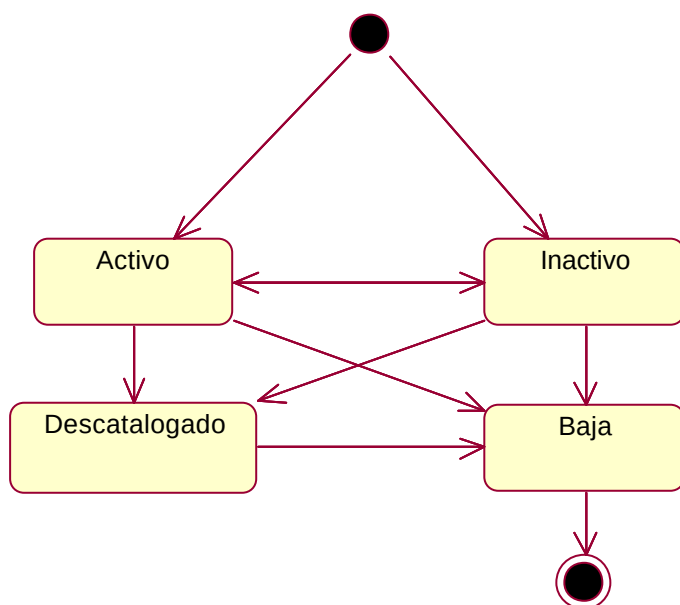
5.3. Diagrama de Estado

Ejercicio

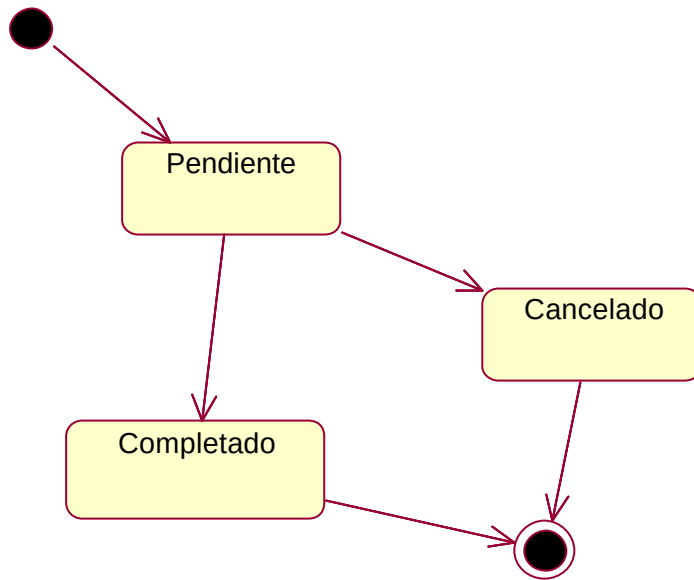
1. Interprete el diagrama dado por el escenario
2. Si el tutor se lo solicita, realice la programación asociada al diagrama.

ESCENARIO

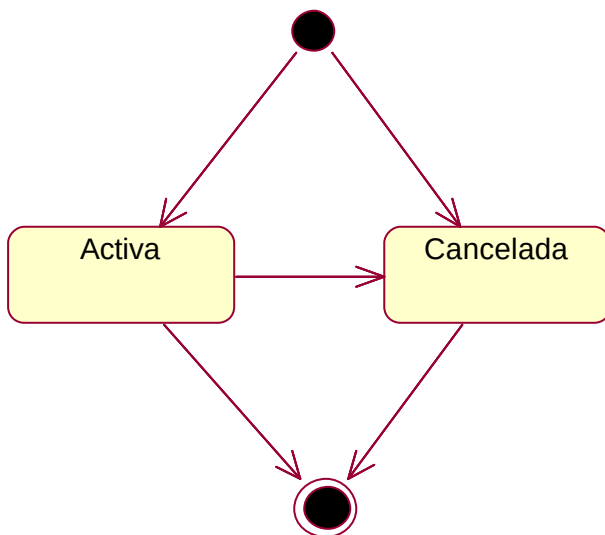
PRODUCTO



PEDIDO



FACTURA

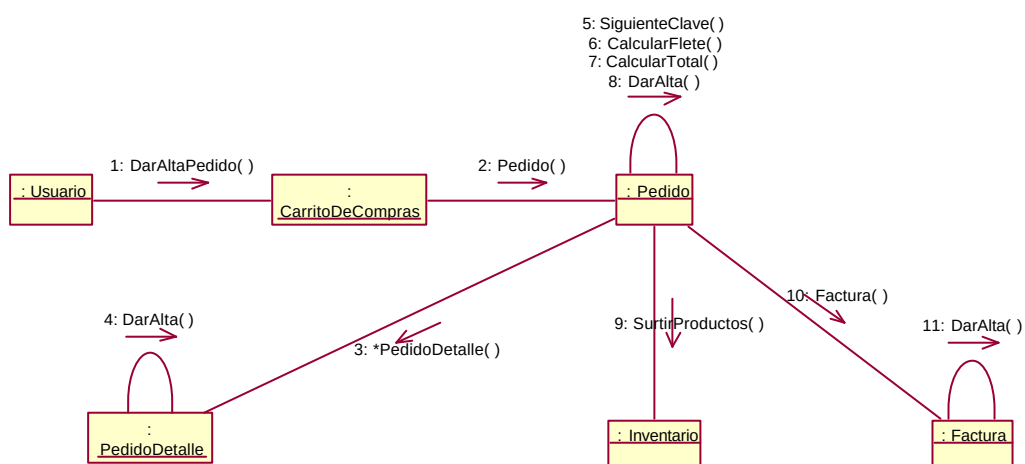


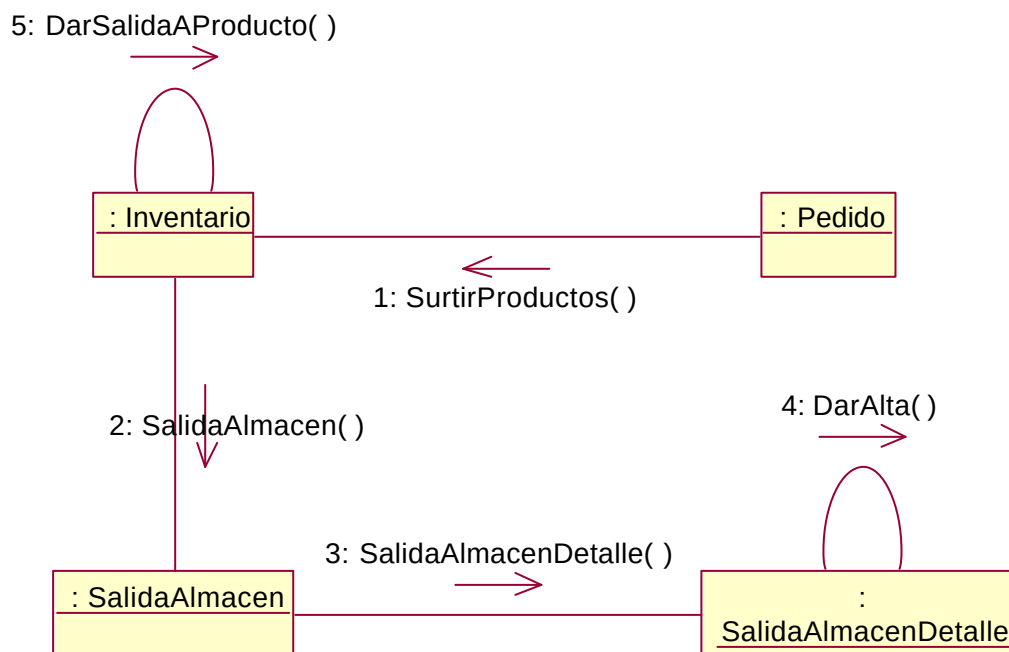
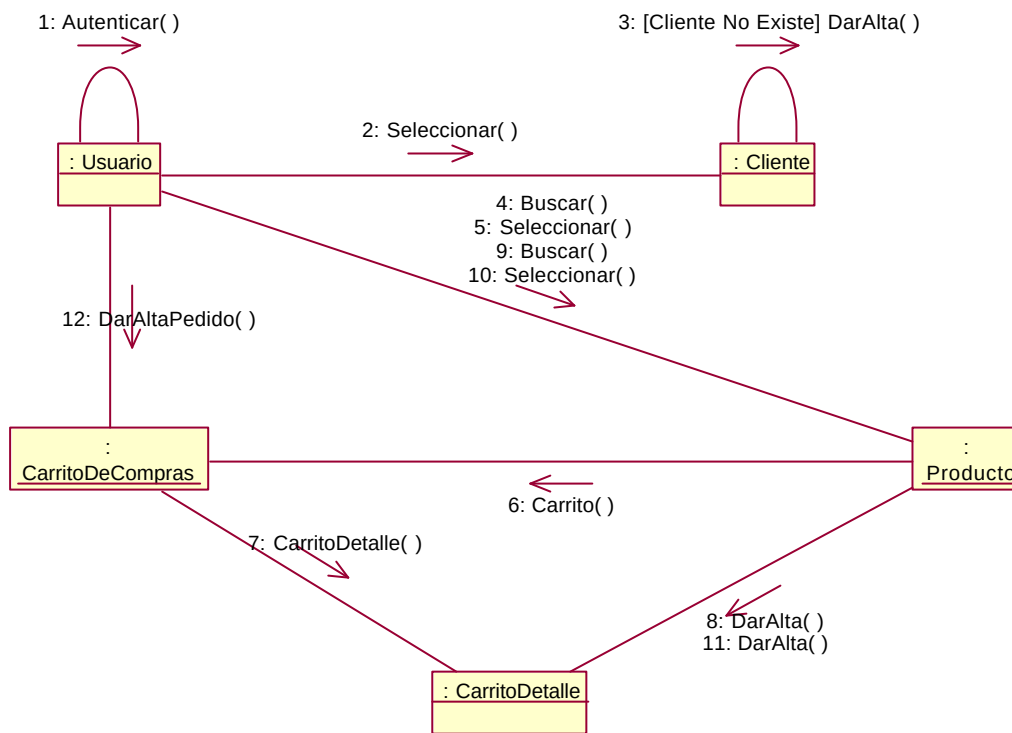
5.4. Diagrama de Colaboración

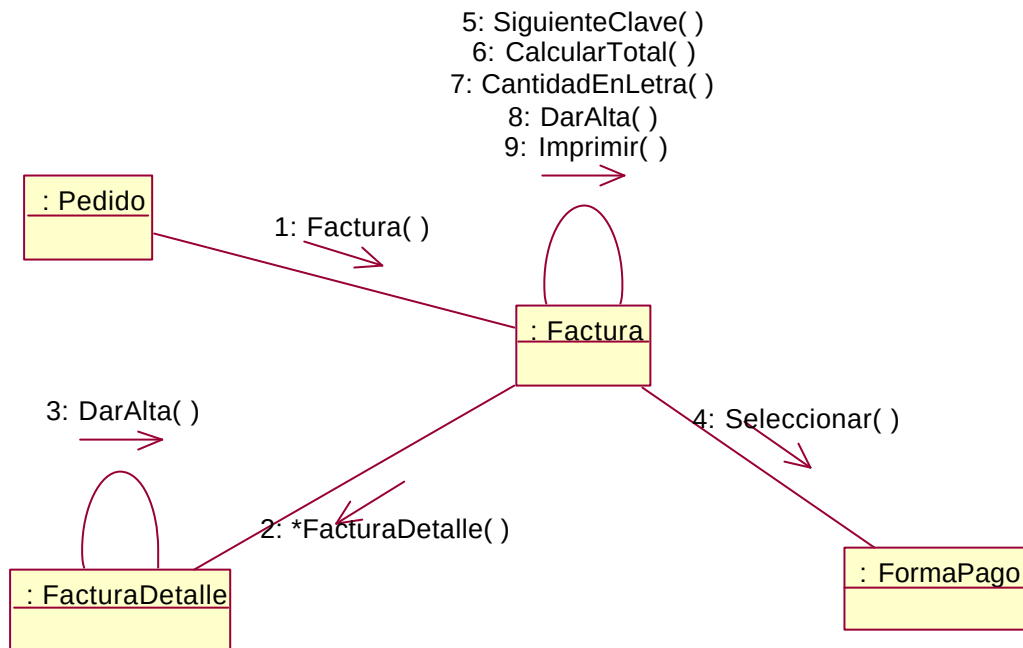
Ejercicio

1. Interprete el diagrama dado por el escenario
2. Si el tutor se lo solicita, realice la programación asociada al diagrama.

ESCENARIO







5.5. Diagrama de Actividad

Ejercicio

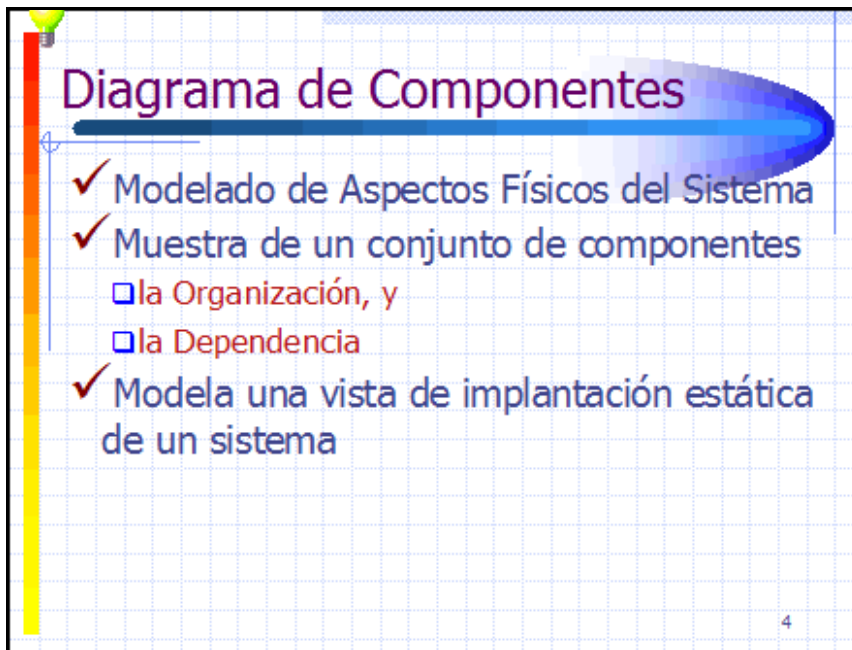
Interprete el diagrama dado por el escenario. Modele el proceso de venta y facturación a través de flujos de trabajo con diagramas de actividad.

6. Modelo de Implantación o Distribución

Objetivo Específico:

- Reconocerá los elementos representados en los diferentes diagramas empleados en el modelo de diseño (vista lógica y de proceso) de UML para programar en distintos lenguajes de programación las representaciones de los diagramas y evaluar tales implementaciones.

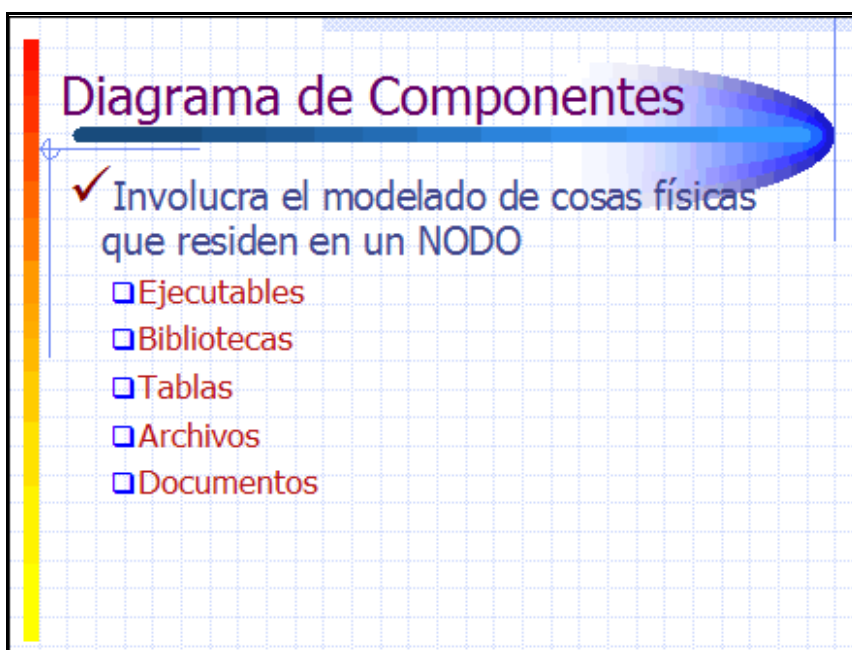
6.1. Diagrama de Componentes (vista de implementación)



El aspecto físico del sistema corresponde a la forma en que podemos apreciar físicamente los componentes de un sistema, como por ejemplo, un archivo en el sistema operativo.

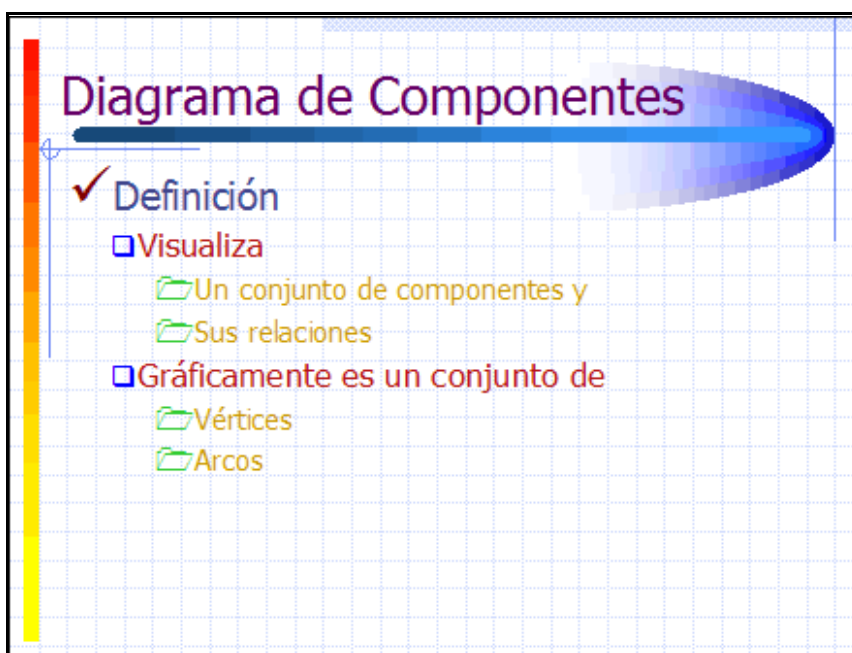
Dada la naturaleza distribuida de una aplicación, donde ésta se conforma de uno o más componentes; se hace necesario apreciar visualmente la organización y dependencia entre éstos.

Tal apreciación es estática, es decir, se conserva a través del tiempo y se enfoca más a la relación estructural entre los componentes.



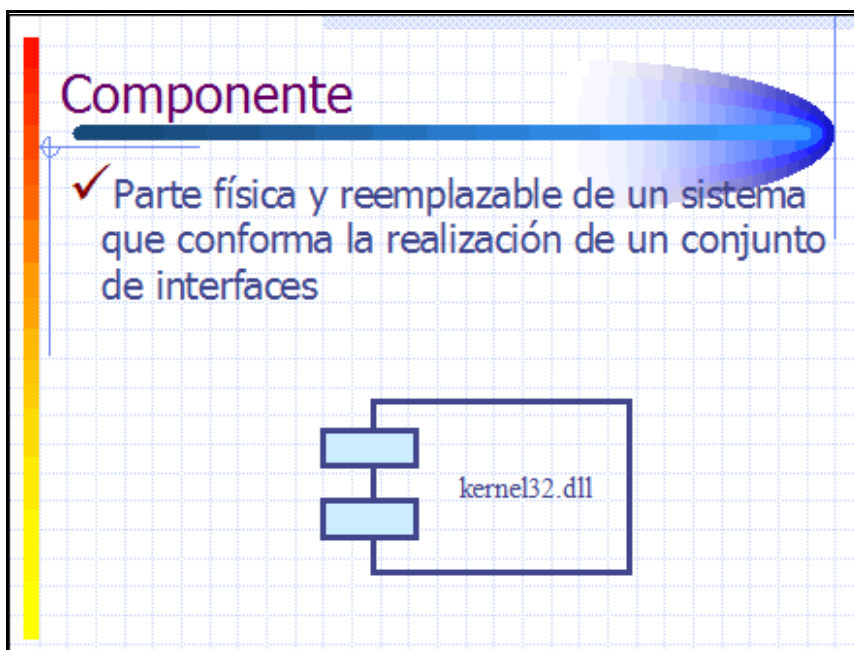
Un nodo es un elemento de procesamiento que participa o contiene uno o más componentes de una aplicación.

En forma general, el tiempo de componentes son elementos de software que pueden apreciarse como existentes en un sistema de cómputo o nodo tales como: ejecutables, bibliotecas, tablas, archivos o documentos entre otros.



Un diagrama de componentes visualiza un conjunto de componentes y la relación entre ellos.

Desde el punto de vista gráfico, el diagrama se conforma por un conjunto de vértices o íconos que representan los componentes que participan en la aplicación o solución de software; y de un conjunto de líneas o arcos que los unen y establecen las relaciones estructurales entre ellos.



Un componente es una parte física y reemplazable de un sistema y que conforma la realización de un conjunto de interfaces.

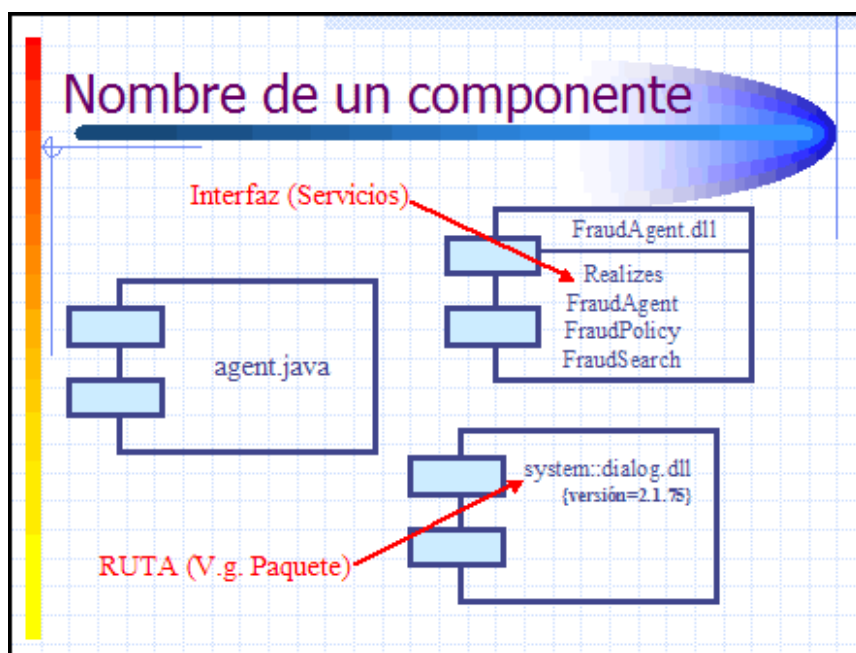
Es una parte física identificable, y aún a pesar que es un elemento de software, puede apreciarse su existencia individual como por ejemplo un archivo que será ejecutado o biblioteca de encadenamiento dinámico que se localizará en la memoria.

Es reemplazable bajo una óptica de “caja negra”, puede modificarse su estructura y comportamiento interno mientras no modifique la forma con la que se relaciona con otros componentes.

Realiza un conjunto de interfaces, ya que un componente expone conjuntos de métodos a ejecutar y brindan servicios a quién se los solicite.

Desde otro punto de vista y en el caso donde aplique, un componente empaqueta una o más clases o interfaces de clase, y tendrá tantas interfaces como clases o interfaces de clase contenga. Puede darse el caso recurrente donde un

componente contenga una sola clase y por tanto exponga una sola interfaz; pero no siempre es así, un componente puede contener varias clases y por tanto exponer varias interfaces.



El símbolo general utilizado para representar un componente se construye con 3 rectángulos; de éstos, un rectángulo es el principal y los dos restantes son adornos cuyo objetivo es representar una especie de “enchufe” intentando representar su naturaleza física y reemplazable dentro de una aplicación o solución distribuida.

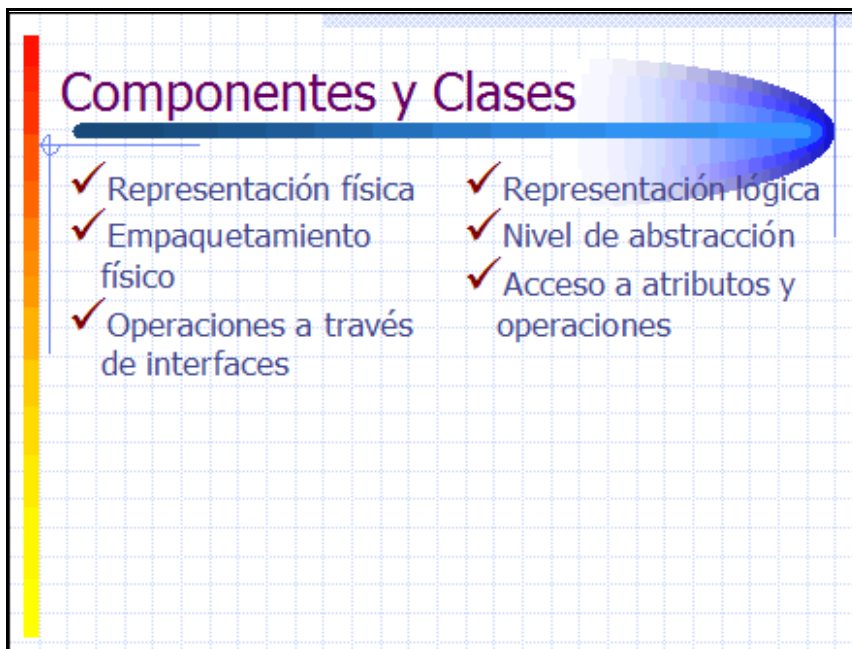
El rectángulo principal contiene en su interior el nombre del componente.

Existen algunas variantes. Una de ellas consiste en mostrar el conjunto de interfaces que expone el componente; para ello, se divide con una línea horizontal en dos secciones, la primera sección contiene el nombre del componente y la segunda, a través de la palabra reservada “realizes” el nombre de las interfaces que contiene o realiza el componente.

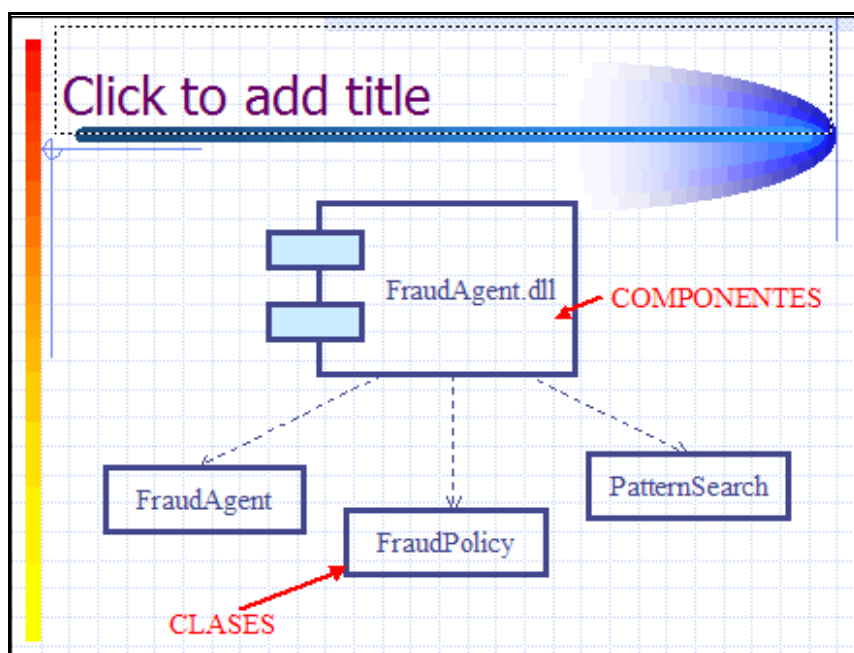
Por otro lado, puede agregarse información adicional (o “adornments”) que proporcionen den mayor semántica de organización o identificación del componente. Por ejemplo, puede indicarse el nombre del paquete al que pertenece el componente con un prefijo: “<nombre del paquete>::”. Otro ejemplo, mostrando información encerrada entre llaves como puede ser la versión del componente.



Desde el punto de vista de utilización, los componentes y clases pueden tener nombre, realizan un conjunto de interfaces, participan en relaciones de dependencia, generalización y asociación; pueden anidarse, es decir, un componente puede contener uno o más componentes ó una clase puede contener una o más clases (agregación); pueden tenerse instancias de estos y participar en interacciones con otros elementos de su mismo tipo.



El hecho de comparar las semejanzas entre componentes y clases no significa que tengan la misma naturaleza. Dado que un componente contiene o empaqueta una o más clases. El componente tiene una apreciación física, mientras que la clase es de tipo lógico. Una clase puede existir sin la existencia de componentes, a excepción de un único ejecutable que contenga todas las clases participantes en una aplicación. Sin embargo, un componente no puede existir sin al menos una clase que contener. Es importante recalcar que una misma aplicación puede estructurarse en componentes, donde estos contienen las clases que participan en la aplicación, o exclusivamente de clases. Si está estructurada en componentes, alguno de estos puede sustituirse sin afectar al resto; sin embargo, si una aplicación no utiliza componentes, al afectar una clase, toda la aplicación tendrá que ser regenerada o compilada.



Una forma de poder diagramar las interfaces que realiza o contiene un componente es utilizando la relación de dependencia entre el símbolo de componente y el de clase.

Un componente depende de las clases que empaqueta o contiene.

Componentes e Interfaces

✓ Interfaz

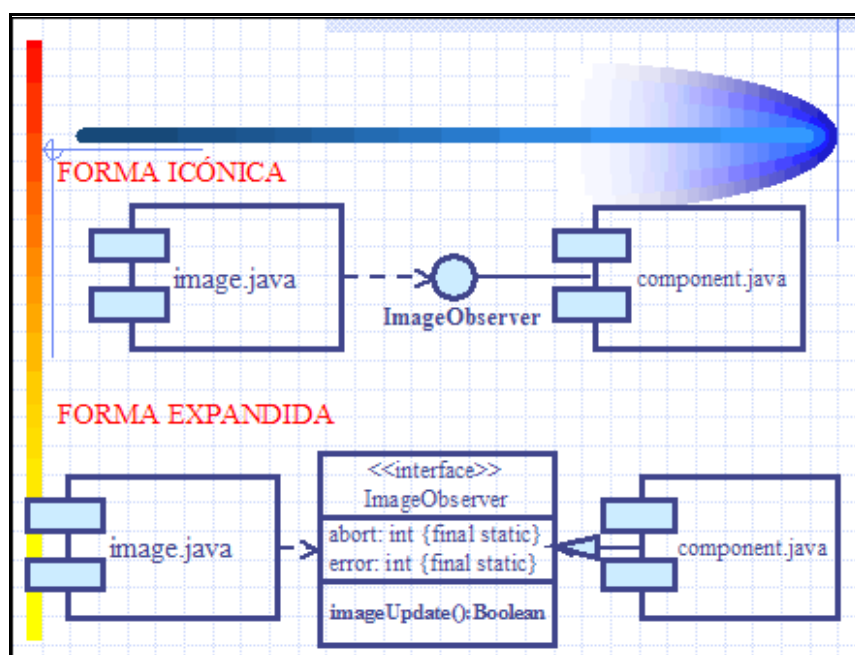
- Colección de Operaciones que es utilizada para especificar un servicio de una clase o componente.
- Son el pegamento entre los diferentes componentes (se requiere infraestructura basada en componentes del sistema operativo)
 - ⇒ COM+
 - ⇒ CORBA
 - ⇒ Enterprise Java Beans

Tanto las clases como los componentes ofrecen interfaces.

Una interfaz, en general, es una colección de operaciones que es utilizada para especificar un servicio (de una clase o componente).

El concepto de interfaz se utiliza básicamente de dos formas. Una forma general, en la que una interfaz es lo que una clase expone hacia su exterior. La otra forma, mucho más estricta y por definición, es una clase sin atributos y que sólo contiene la firma de los métodos más no su implementación.

Actualmente, la forma de comunicar los componentes es mediante un tipo de tecnología denominada “middleware de objetos”, que obedece a estándares de comunicación entre objetos. Como por ejemplo existen estándares de middleware como COM+, CORBA, J2EE, etc.

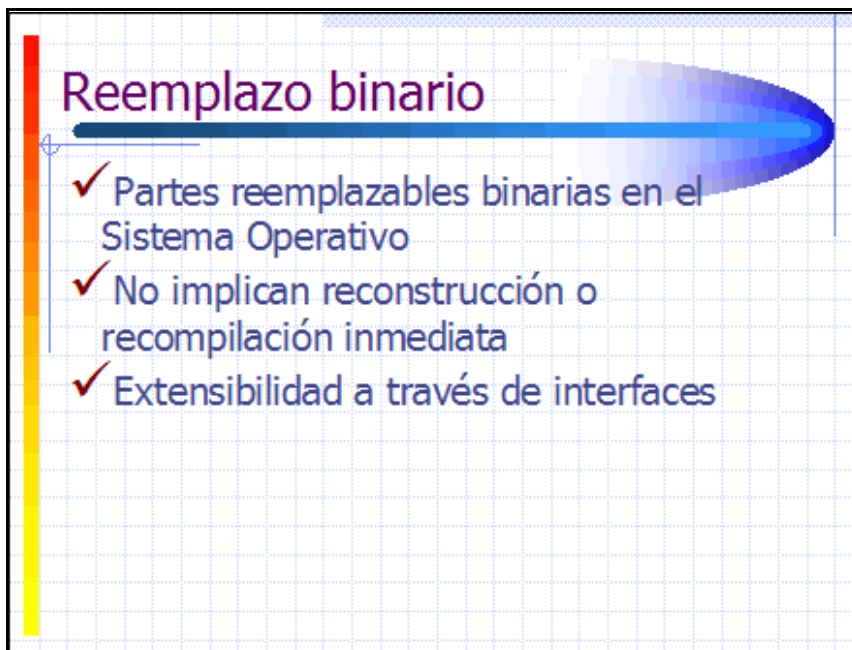


Existen 2 formas para modelar las interfaces que ofrece un componente:

- Forma icónica
- Forma expandida
- La forma icónica consiste en modelar la interfaz como una circunferencia y el nombre de la interfaz en la parte inferior bajo la circunferencia. Se traza una línea recta hacia el componente que implementa la interfaz y una relación de dependencia del componente que utiliza la interfaz.

La forma expandida consiste en representar la interfaz como una clase con el estereotipo <<interface>> (recordemos que una interfaz de clase no contiene atributos –aunque podrían contener constantes- y sólo la firma de los métodos).

Con una flecha de realización, se unen el símbolo de la clase y el componente que lo implementa. Al igual que la forma icónica, el componente que lo utiliza se une con el símbolo de la clase a través de la flecha de dependencia.



Precisamente, el reemplazo binario es una de las características importantes de un componente. Al ser físicamente una pieza de software compilada, obedeciendo a determinadas reglas en la exposición de sus interfaces, puede ser reemplazada por la misma pero modificada internamente; sin afectar o tener que recompilar toda la aplicación.



Hasta el momento nos hemos referido a los componentes como aquellos necesarios para crear una aplicación distribuida ejecutable.

Sin embargo, en UML existen categorías adicionales que abarcan a un conjunto de componentes de otra índole. UML considera 3 categorías de componentes:

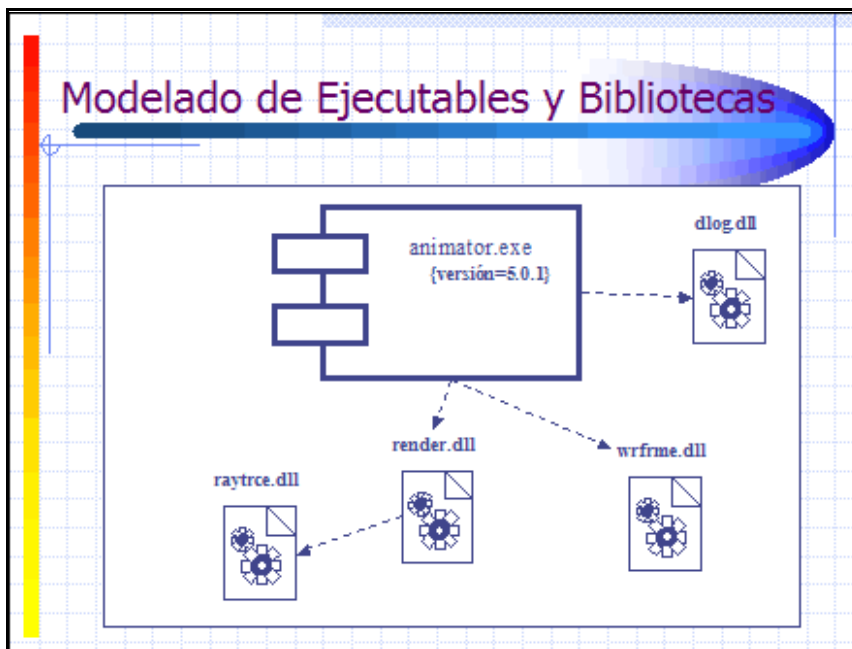
1. De implantación: Son aquellos necesarios y suficientes para generar sistemas ejecutables como archivos ejecutables, bibliotecas de encadenamiento dinámico o equivalentes, tablas de datos, páginas web estáticas o dinámicas, modelos de objetos, otros ejecutables, etc.
2. Productos de trabajo: Archivos fuente o de datos.
3. De ejecución: Aquellos que existen sólo en ambiente de ejecución.



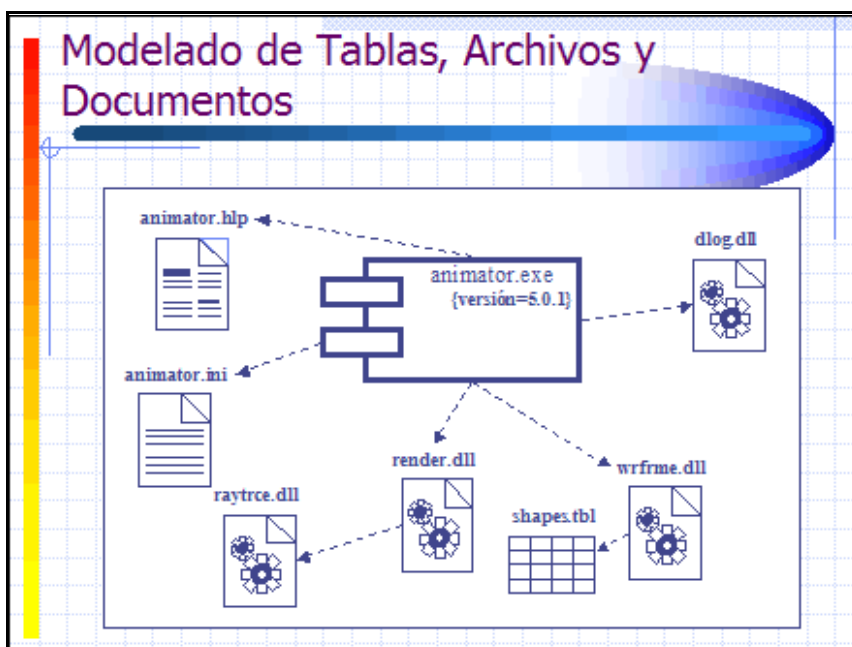
Aunque existe un símbolo general para componente, en UML se nos indica adicionalmente una serie de símbolos para expresar el tipo de componente del que se esté refiriendo. Sin embargo, siempre podrá ser usado el símbolo único para componentes.

Los símbolos adicionales para componentes son:

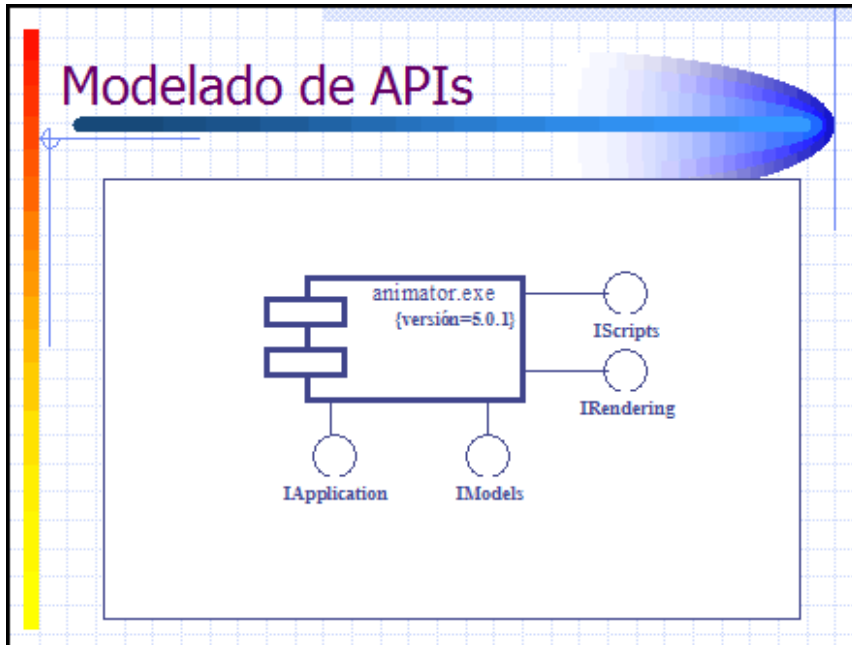
- Biblioteca: Se utiliza un rectángulo a manera de hoja de papel con la esquina superior derecha doblada, conteniendo un par de engranes en la parte interior.
- Tabla: Se utiliza un retícula dentro de un rectángulo, siendo más ancho en su parte horizontal.
- Archivo: Se utiliza un rectángulo a manera de hoja de papel con la esquina superior derecha doblada, conteniendo líneas horizontales en su parte interior
- Documento: Se utiliza un rectángulo a manera de hoja de papel con la esquina superior derecha doblada, conteniendo líneas horizontales en su parte interior a manera de 2 columnas.



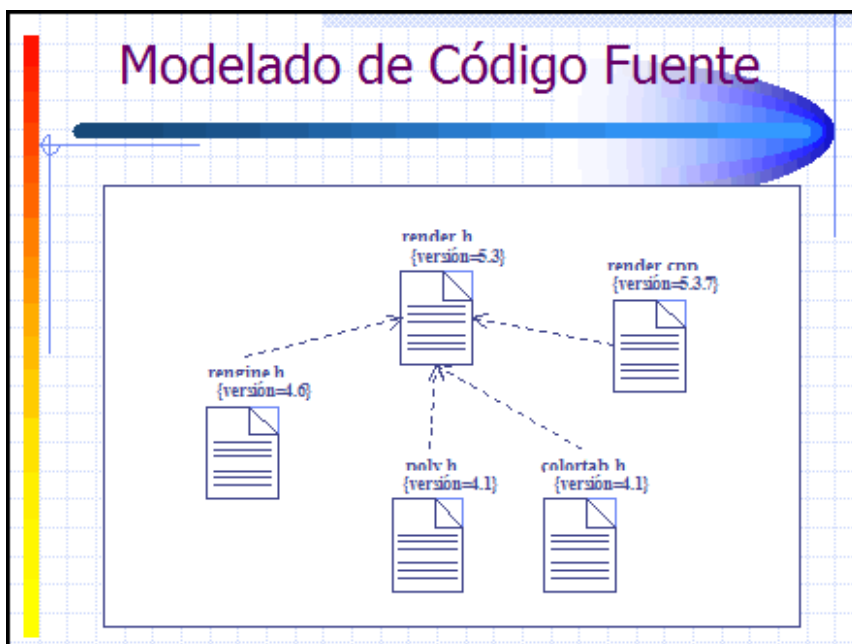
En este ejemplo se expresa la relación de dependencia de un ejecutable con los dll's que utiliza.



Este ejemplo es el mismo que el anterior, expresando adicionalmente la dependencia de otros elementos como un archivo de inicio, una tabla de la base de datos y un documento de ayuda.

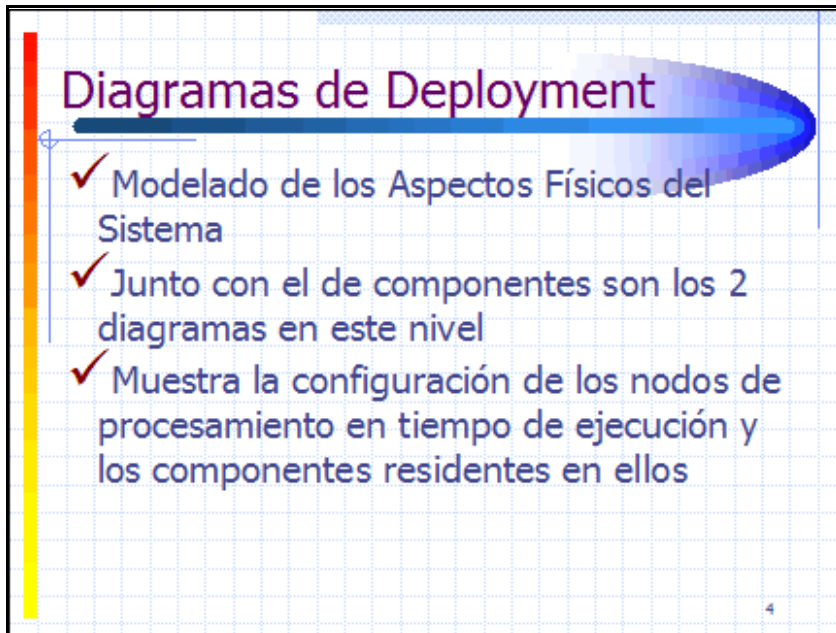


Se cuenta con una forma de expresar las interfaces que implementa con componente en cuestión.



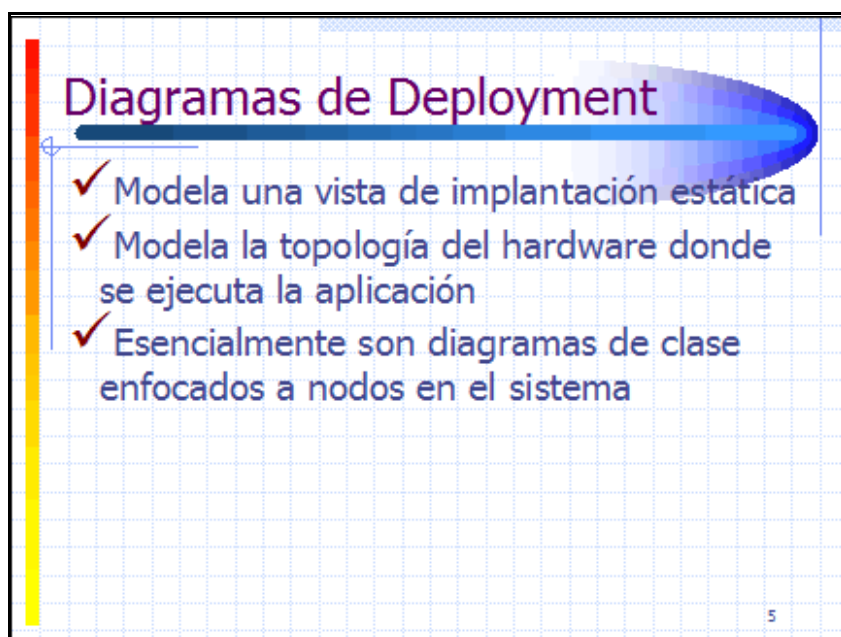
En UML, el diagrama de componentes, también nos sirve para expresar la relación de dependencia existente entre archivos fuente.

6.2. Diagrama de Distribución (vista de distribución)

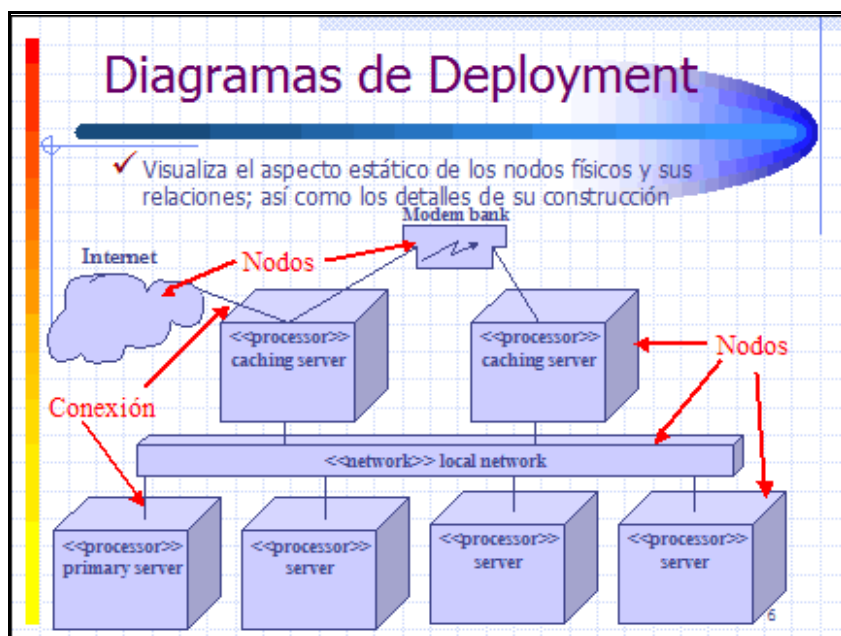


El diagrama de distribución o “Deployment” modela aspectos físicos del sistema, mostrando la configuración de nodos de procesamiento en tiempo de ejecución de la aplicación distribuida, así como los componentes que residen en ellos.

Junto con el diagrama de componentes, son los 2 diagramas del modelo de implantación o físico del sistema.

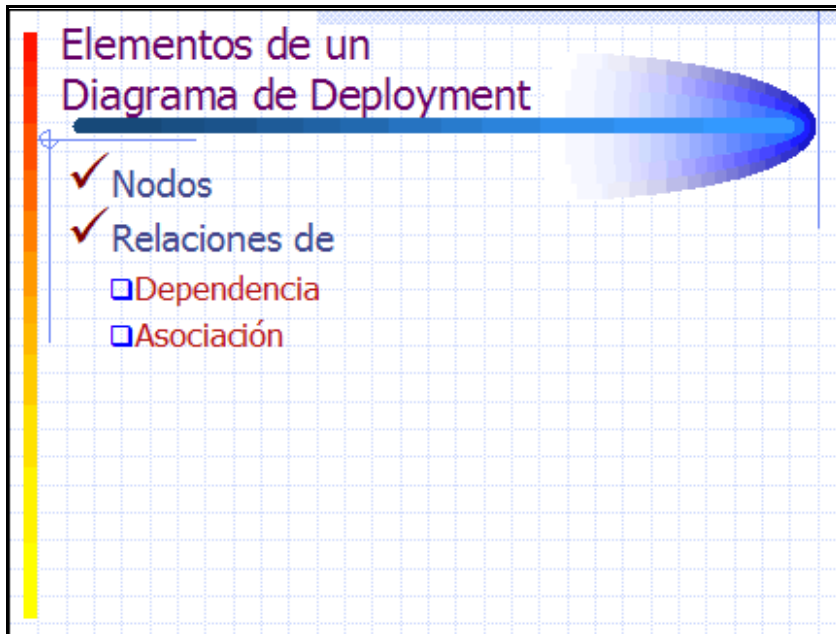


Un diagrama de distribución es de tipo estático, es decir expresa una relación estructural u organizacional y ésta no cambia a través del tiempo. El diagrama de distribución se enfoca en los nodos involucrados en el sistema, tanto en su estructura interna como la relación que existe con otros nodos.



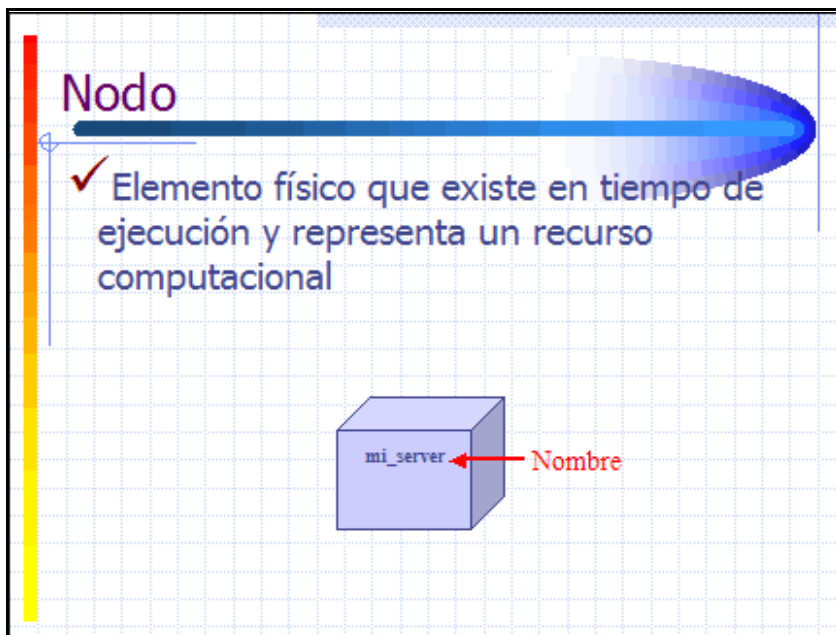
En el ejemplo, se modela una red de área local, los servidores dentro de ésta y su conexión a Internet. Todos estos elementos son modelados como nodos.

Asimismo, se expresa la relación entre ellos a través de líneas denominadas conexiones.



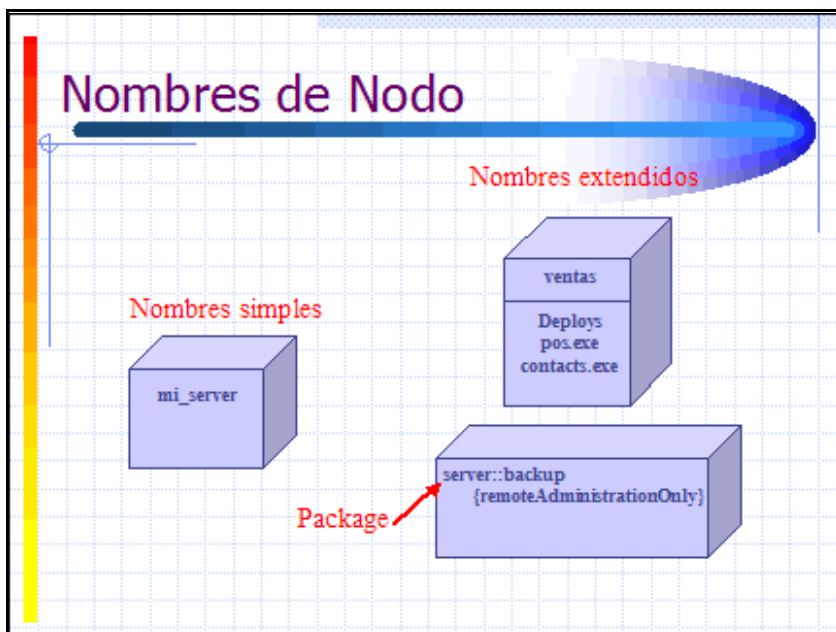
Los elementos utilizados en un diagrama de distribución son:

- Nodos
- Relaciones entre nodos de dependencia y asociación.



Un NODO es un elemento físico que existe en tiempo de ejecución y representa un recurso computacional.

Se representan con una caja con ancho tridimensional, colocando el nombre del nodo en la parte central del rectángulo frontal.

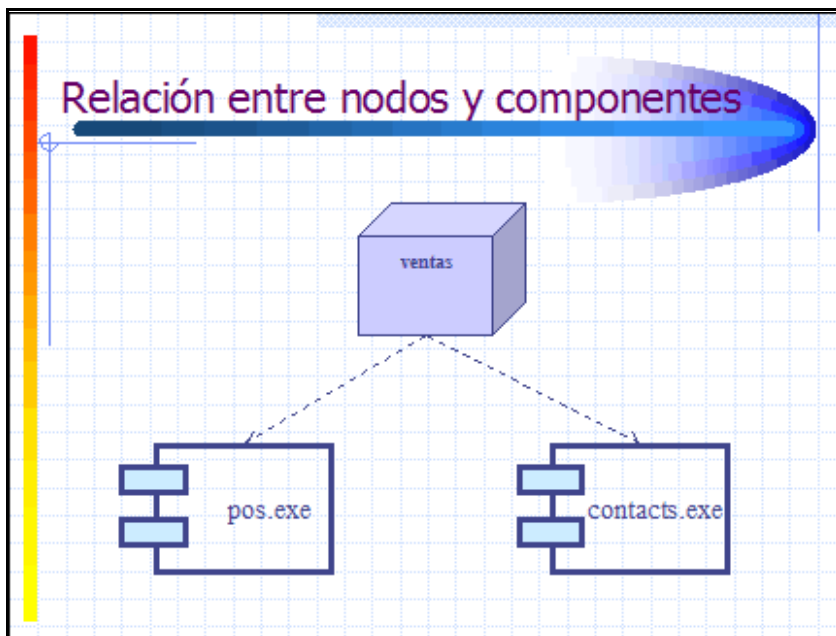


La representación más simple del nombre de un nodo es colocarlo al centro del rectángulo frontal. Sin embargo se cuentan con representaciones extendidas.

Puede dividirse en rectángulo con una línea horizontal para representar dos secciones. La sección superior contendría el nombre del nodo. La sección inferior esta encabezada con la palabra “Deploys” (distribuye) y continua con el nombre de los componentes que contiene.

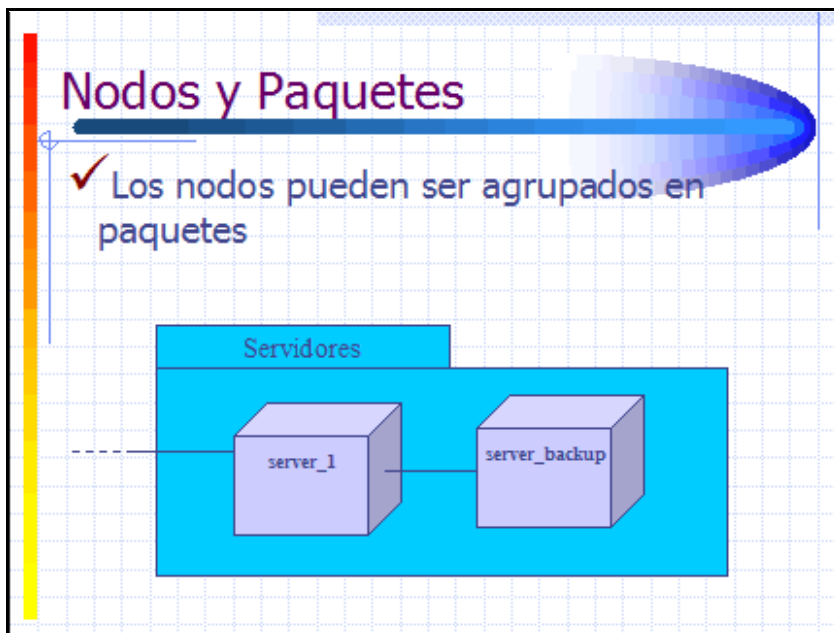
Por otro lado, adicionalmente, también se puede indicar a qué paquete pertenece el nodo con el prefijo “<nombre de paquete>:.”

También puede expresarse información adicional denominada como adornos (“adornments”) encerrándola entre llaves “{“ y “}” por cada característica.

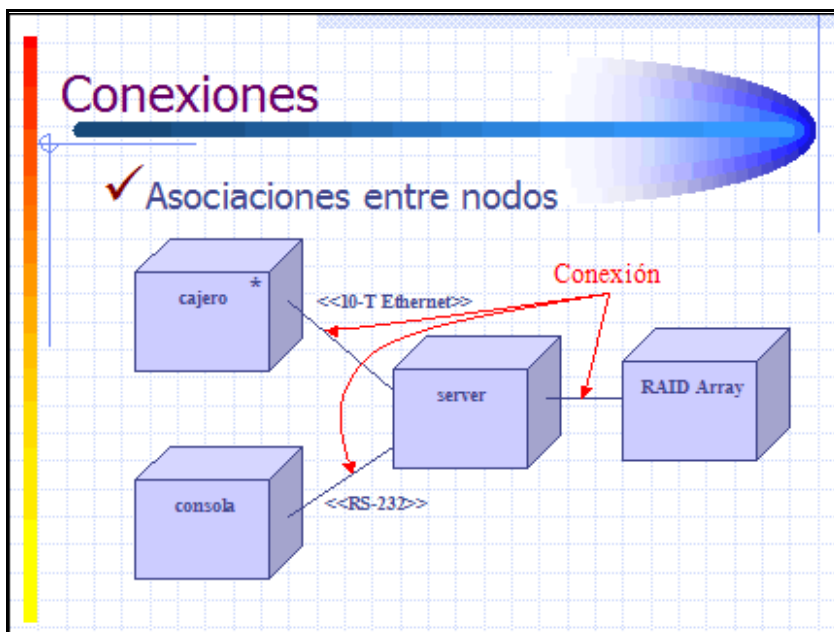


Otra forma de indicar la contención de componentes en un nodo es a través de la relación de dependencia.

Esta forma equivale a aquella que utiliza la palabra reservada “Deploys” en el símbolo de nodo.

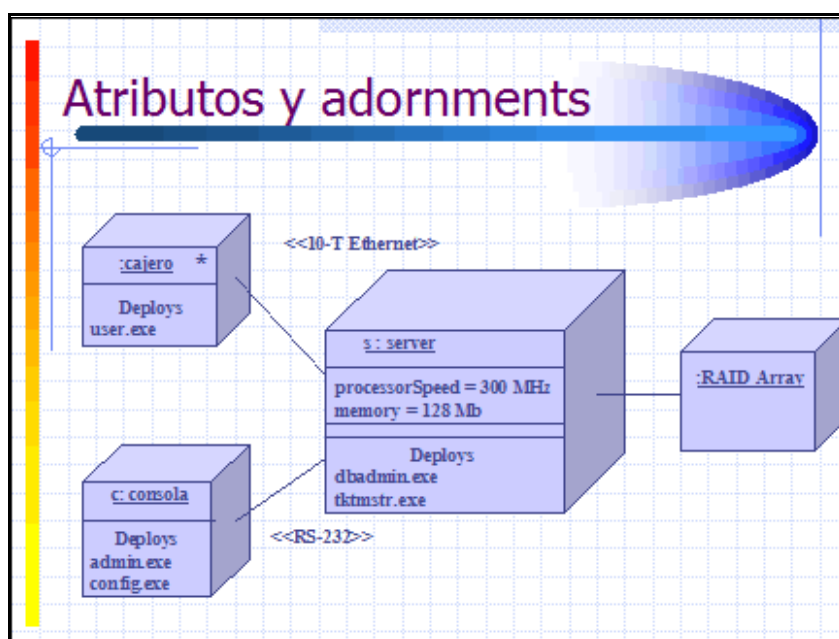


Los nodos pueden ser agrupados con el símbolo de paquete. Esta representación equivale a aquella que tiene el prefijo “<nombre de paquete>::” antes del nombre del nodo.

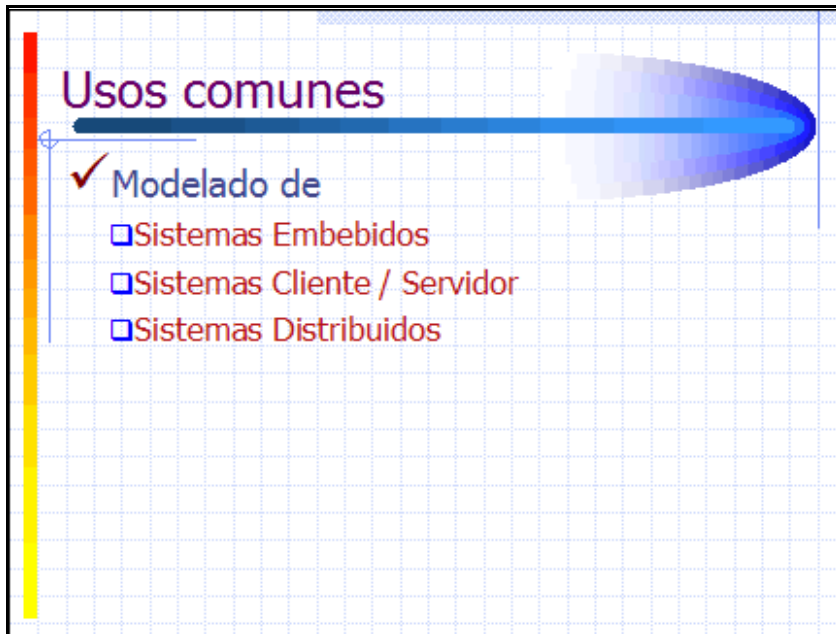


Las conexiones entre nodos expresan a través de la relación de asociación. Puede utilizarse un estereotipo indica en la conexión (con la simbología de estereotipo “<<información>>” que incrementa la semántica asociada a la conexión).

Cuando un nodo tiene un asterisco en la parte superior derecha expresa una relación 1 a muchos, siendo “muchos” del lado con el nodo que tiene el asterisco.



El rectángulo frontal puede dársele el tratamiento como el utilizado en una clase u objeto, por lo que pueden expresarse atributos y operaciones. La sección de “Deploys” es la cuarta sección cuando esto se hace.



Las aplicaciones del diagrama de distribución son particularmente notables cuando se utiliza en el modelado de sistemas incrustados, cliente/servidor y distribuidos.